Index

1. Introduction
2. Linked Lists
3. Stack and Queue
4. Searching and BSTs
5. Heaps
6. Comparison-Based Sorting
7. Non-comparison-Based Sorting
8. Hashing
9. Tree Optimisations - Balancing and Union-Find
10. Graphs
11. Miscellaneous - Randomisation

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 1

Introduction

Welcome to CS 213

Course Logistics

- Every week
  - Three lectures
  - One lab
  - Exercises for practice
- Learning Management System: Bodhitree
  - Lab submissions
  - Slides will be available soon after/before lecture
- Evaluation plan (subject to change)
  - Evaluation for CS 213:
    - Participation (10%)
    - Two quizzes (15% x 2)
    - Two exams (25% + 35%)
  - Evaluation for CS 293:
    - Weekly lab evaluation (20%)
    - Two lab quizzes (15% x 2)
    - Two exams (20% + 30%)
- More logistics in the coming days!

Academic Integrity

Do not copy, consult or share

in quizzes, exams and evaluated lab submissions!

Do not copy or use LLMs in evaluated lab submissions.

You can consult references provided, and ask TAs for guidance/hints.

But blindly copying defeats the purpose of the exercises.

All code submitted should be written by yourself

Any suspected cheating will be reported to D-ADAC

What is in store in CS 213/293?

Data Structures and (related) Algorithms

- In 2017, Instagram switched from "BTrees" to "LSM Trees", for storing activity feed
- What are these "trees"? And why did they matter?
  - BTrees data structure, invented in 1970, is a gold standard in databases. (We will see this later in the course.)
  - LSM Trees (invented in 1996) is specialised for exploiting higher sequential write-speeds available in hardware. Turned out to be an important consideration at Instagram's subsequent scale.

Data Structures and (related) Algorithms

- In almost every program dealing with large amounts of data, a careful choice of data structures is crucial for efficiency
  - Many "standard" data structures are available as part of libraries
- In this course:
  - What, why and how of many standard data structures
    - What: lists, queues, trees, hash tables, ...
      Why: Their efficiency features/limitations
      How: Their internal workings
  - Some algorithms for data manipulation (sorting, compression, ...)

A Quick CS 101 Recap

A C++ Program

```
#include <iostream>
int main() {
  std::cout << "Hello Data Structures!" << std::endl;
  return 0;
}
```

Another C++ Program




```
#include <iostream>
struct node {
  std::string word;
  node* next = nullptr;
  void print() {
    std::cout << word << " ";
    if(next) next->print();
    else std::cout << std::endl;
  }
};
```

```
int main() {
  node* first = new node{"Hello"};
  first->next = new node{"World!"};
  first->print();
  return 0;
}
```

Let's Get Started

With a Little Experiment

Store and Retrieve

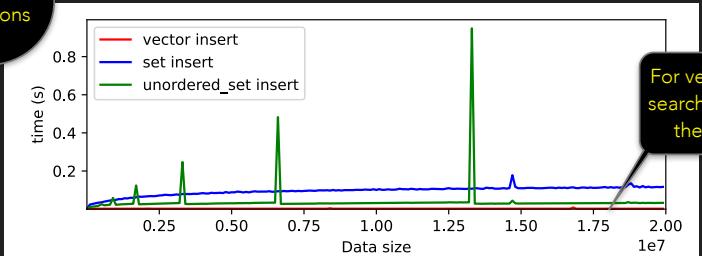
- Task: Store a bunch of integers in a data structure, and later given a query, check if it was already stored
- C++ provides convenient data structures for this, with a "find" method
  - std::vector
  - std::set
  - std::unordered_set
- Let's find out how they each perform

Store and Retrieve

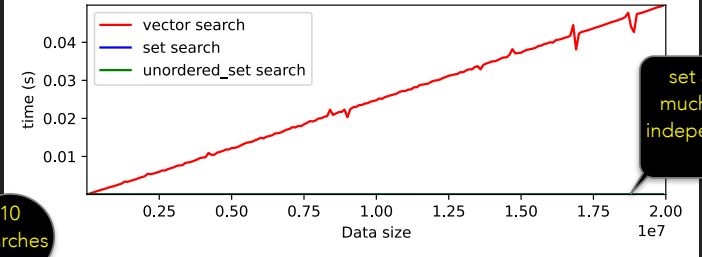
```
std::vector<int> v;
std::set<int> s;
std::unordered_set<int> u;
...
// in a loop
  v.push_back(item);
  s.insert(item);
  u.insert(item);
...
std::find(v.begin(), v.end(), query); //iterate through the range
s.find(query);
u.find(query);
```

Store and Retrieve

100000 insertions



For vector, insert is very fast, but search gets slower and slower as the number of items grows



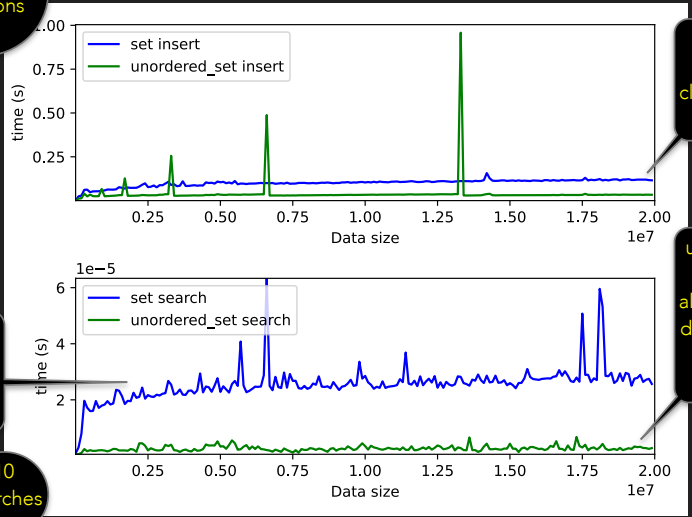
set and unordered_set have much faster search, seemingly independent of how big the data set is

10 searches

Store and Retrieve

100000 insertions

A closer look at set vs. unordered_set



set insert is also slowly climbing with the data size

unordered_set insert and search both take time almost independent of the data-size (except for insert times spiking at various data sizes)

set search is slowly climbing with the data size

10 searches

Data Structures and (related) Algorithms

- What is under the hood of std::set and std::unordered_set, and why do they behave so differently from each other?
- Looking ahead:
  - std::set is a “balanced binary search tree”
  - std::unordered_set is a “hash table”

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 2

Asymptotics

Measuring the Trends

Course Logistics

- For the Lab today, find out group you are assigned to at https://bit.ly/A26-DSA-Mapping (needs IITB Google Suite login)



Recall Our Experiment

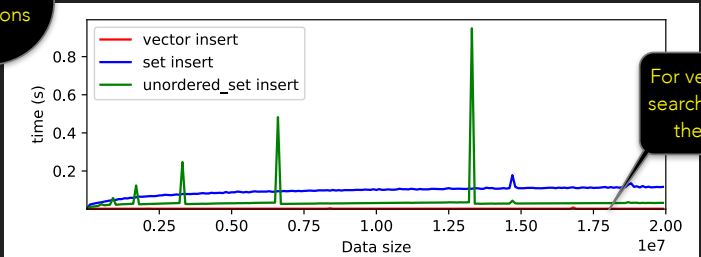
Store and Retrieve



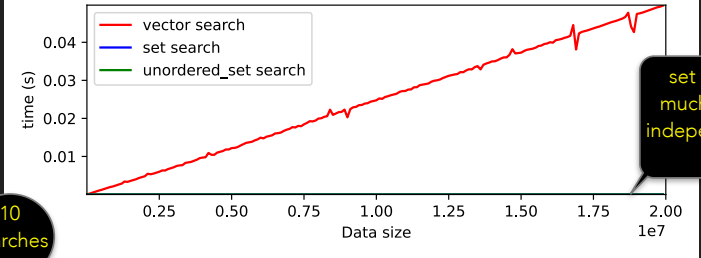

```
std::vector<int> v;
std::set<int> s;
std::unordered_set<int> u;
...
// in a loop
  v.push_back(item);
  s.insert(item);
  u.insert(item);
...
std::find(v.begin(), v.end(), query); //iterate through the range
s.find(query);
u.find(query);
```

Store and Retrieve

100000 insertions



For vector, insert is very fast, but search gets slower and slower as the number of items grows



set and unordered_set have much faster search, seemingly independent of how big the data set is

10 searches

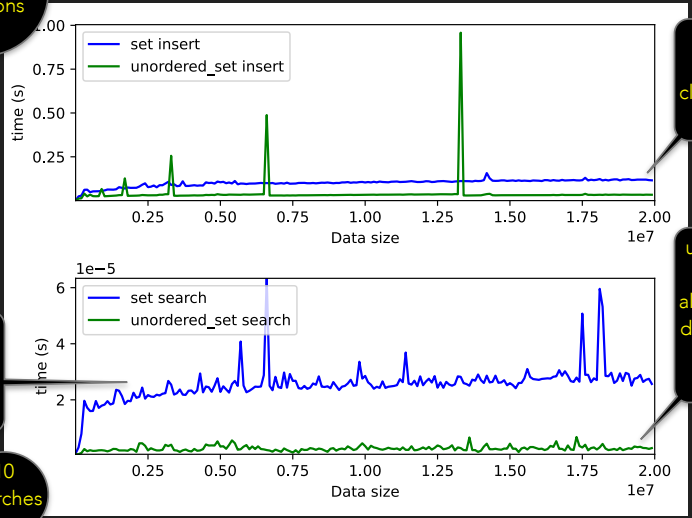
Recall

Store and Retrieve

Recall

100000 insertions

A closer look at set vs. unordered_set



set insert is also slowly climbing with the data size

unordered_set insert and search both take time almost independent of the data-size (except for insert times spiking at various data sizes)

set search is slowly climbing with the data size

10 searches

Measuring Behaviour

- What is under the hood of std::set and std::unordered_set, and why do they behave so differently from each other?
- But first: we need some vocabulary to discuss their behaviour
  - Asymptotic Worst-Case upper bounds
- Let us unpack that

Asymptotic Behaviour

Asymptotic Behaviour

- Imagine a quantity, that starts growing starting from 1 item on day 0, in one of three ways
  - Each day, increases by a million: 1, 1000001, 2000001, ...
  - On day n, increases by 10000n: 1, 10001, 30001, 60001, ...
  - Each day, doubles: 1, 2, 4, 8, ...

$A(n) = 10^6 n + 1$

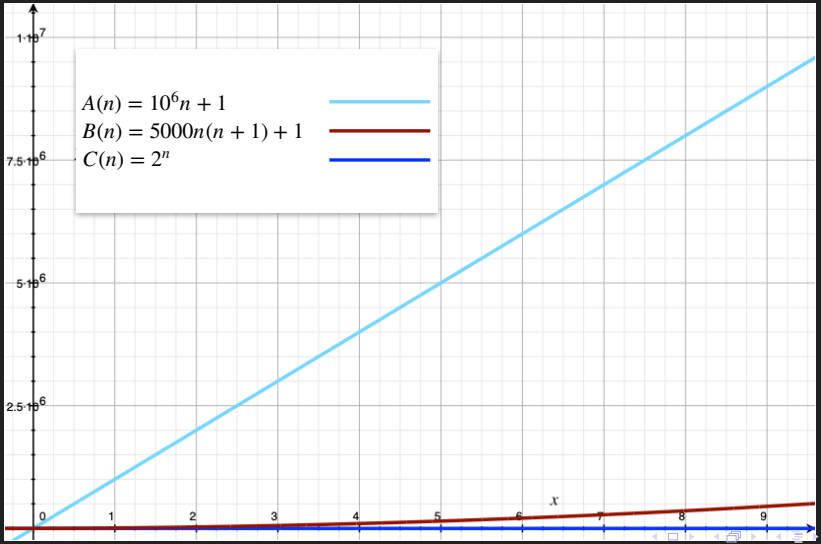
$B(n) = 10000n(n+1)/2 + 1$

$C(n) = 2^n$

- After a week, which rule will result in the largest amount? After a month?

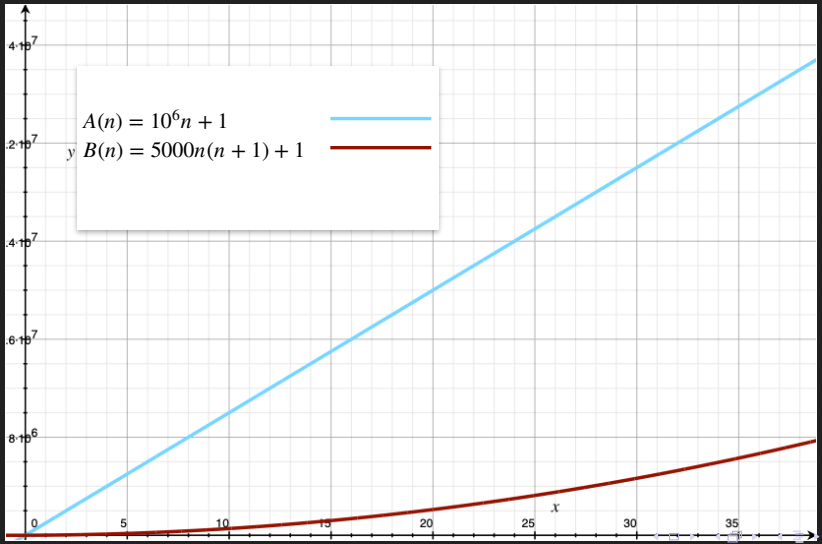
Asymptotic Behaviour

The first couple of weeks



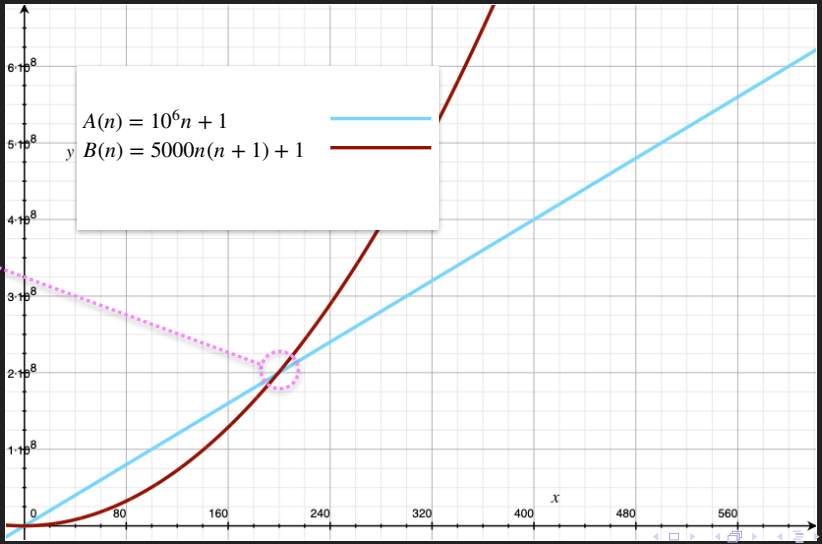
Asymptotic Behaviour

A vs. B:
First month



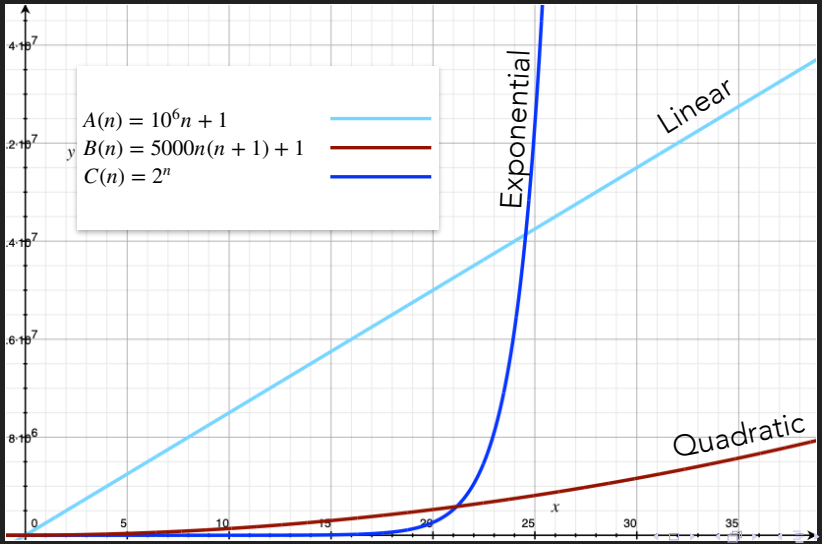
Asymptotic Behaviour

A vs. B:
200 days later



Asymptotic Behaviour

Back to the first month



Eventually,

Exponential
>> Quadratic
>> Linear

Asymptotic Behaviour

- *We say* f(n) has an asymptotically linear upper bound, if there is a scaling factor $c > 0$ and a threshold $k > 0$, such that for all $n > k$, $f(n) \leq c.n$
  - The "Big-Oh" notation: $f(n) = O(n)$
- And a quadratic upper bound, or $f(n) = O(n^2)$, if $\exists c, k$ s.t. $\forall\ n>k$, $f(n) \leq c.n^2$
- More generally, $f(n) = O(g(n))$ if $\exists c, k$ s.t. $\forall\ n>k$, $f(n) \leq c.g(n)$
- E.g., Suppose $f(n) = 5n^2 + 2n + 1$.
  - Then $f(n) = O(n^2)$. $5n^2 + 2n + 1 \leq 6n^2$ once $2n+1 \leq n^2$, or $n \geq 3$
  - But f(n) is not O(n).

If $5n^2 + 2n + 1 \leq cn$ for some c, then $n^2 \leq cn$. But this holds only for $n \leq c$, and not all $n \geq$ any k.

Asymptotic Behaviour

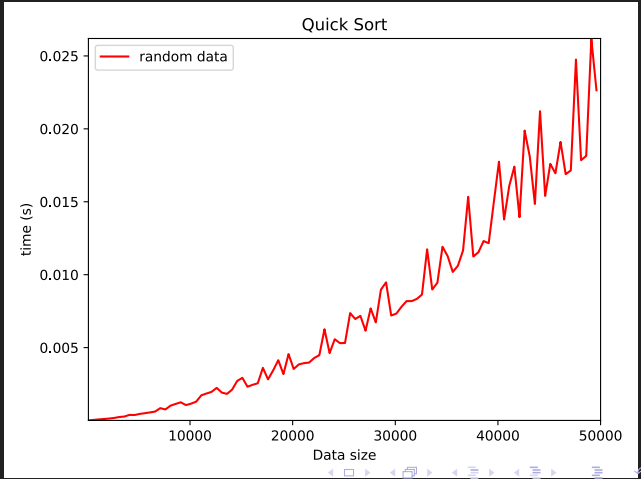
- We are typically interested in the question of scaling (to very large inputs)
  - E.g., If processing a query involving million data points takes 1s, how long will it take to process one involving 10 million data points?
    - Will it take about the same, 10x more, or 100x more, or something else?
    - Depends on how the processing algorithm scales.
    - If processing time (as a function of input data size n) is O(n), then roughly 10 seconds. If it is $O(n^2)$, roughly 100 seconds.

Asymptotic Behaviour

- $O(\cdot)$ is an upper bound
  - E.g., $f(n) := 5n^2 + 2n + 1 = O(n^2)$. But it is also $O(n^3)$, $O(n^4)$ etc.
  - If $f(n) = O(g(n))$ and $g(n)=O(f(n))$, we write $f(n) = \Theta(g(n))$
    - Equivalently, $f(n) = \Theta(g(n))$ iff there exist positive constants α, β, k s.t. for all $n \geq k$, $\alpha.g(n) \leq f(n) \leq \beta.g(n)$
- But what would f be?
  - Running time of an algorithm (in some abstract machine). But on which input? Worst-Case!

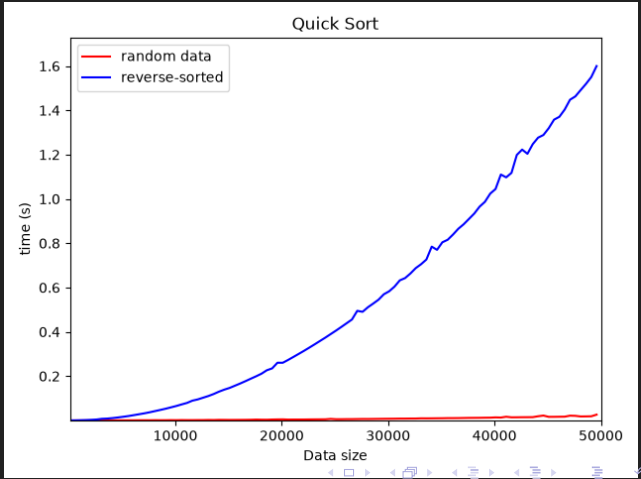
An Experiment

- QuickSort is a popular sorting algorithm, the basic form for which we'll experiment with
- On random data
  - Not O(n)
  - Turns out to be Θ(n log n)



An Experiment

- QuickSort is a popular sorting algorithm, the basic form for which we'll experiment with
- On random data
  - Not O(n)
  - Turns out to be Θ(n log n)
- On reverse sorted data, takes *much* longer
  - $\Theta(n^2)$



Asymptotic Worst-Case Time

- Asymptotic: What is the trend as data grows
- Worst-Case: Algorithms can perform wildly differently on different kinds of inputs. We should ideally be prepared for the worst-case.
  - Calculate O(.) upper bounds or Θ(.) bounds *analytically.*
- Some examples we will encounter: O(1), O(log n), O(n), O(n log n), $O(n^2)$
  - std::set has Θ(log n) insertion and search (where n is the current set size)
  - std::unordered_set has Θ(1) search; insert can occasionally be Θ(n).
  - std::vector has Θ(1) insert and Θ(n) search.

Data-Structures & Algorithms

with

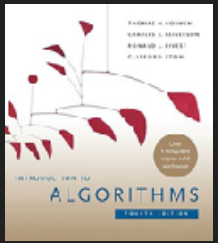
Manoj Prabhakaran

Lecture 3

Linked Lists

Course Logistics

- Bodhi Tree
  - We'll use this as a portal for all course-related activities
  - Discussion forum, announcements, labs, lecture slides
- Reference textbook: CLRS
  - We don't need everything in the book
  - And we'll cover a few things not in the book



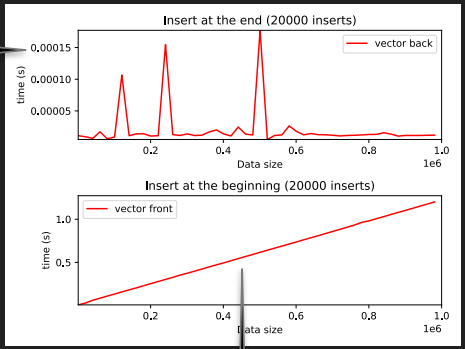
Arrays

- We saw std::vector has Θ(1) insertion (and Θ(n) search) time
- But that is only if you insert at the back of the vector
  - In std::vector, `v.push_back(x)`
- What happens if you insert elsewhere?
- A vector uses a contiguous array of memory locations to store its elements
- To insert at location i, you need to physically move the memory contents at locations i, i+1, i+2, … to make room for the new element
  - O(n-i) time, if n is the size of the vector
  - Can we really see this?

Inserting into an Array

```
std::vector<int> v1, v2;
...
// in a loop
  ...
  v1.insert(v1.begin(), 1); // add as v[0]
  v2.insert(v2.end(), 1); // v2.push_back(1)
...
```

microseconds vs 1 second!



Θ(1) vs Θ(n)

An Alternative to Arrays

- Linked lists: Maintain an ordering of the items, while still allowing O(1) insertion to the front and the back
- A list of "nodes" which do not have to be in contiguous memory locations in order
- Each node has a pointer to the next one in the list
  - The last node has `nullptr` as next

O(1) insertion

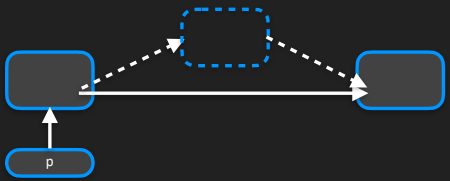
```
struct node { int val; node* next = nullptr; }; // a node in the list
int main() {
  node *head, *tail;
  head = tail = new node{0}; //a list with a single node
  // insert at the end
  tail->next = new node{1}; //attach a new node at the back
  tail = tail->next;        //point tail to the new node
  // insert at the beginning
  head = new node{-1,head};
}
```

```
node* tmp = new node;
tmp->val = -1;
tmp->next = head;
head = tmp;
```

O(1) insertion

- Can insert not just to the front and back, but anywhere in a list
  - Once we have a position in the list to insert into, insertion itself takes a fixed number of operations, irrespective of the size of the list

```
// p points to a node; to insert a new node with val x after that node
p->next = new node{x,p->next};
if(p==end) end = p->next;
```

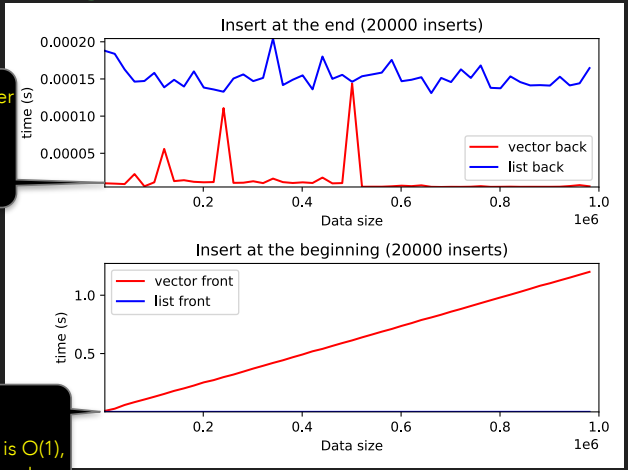



```
node* tmp = new node;
tmp->val = x;
tmp->next = p->next;
p->next = tmp;
```

List vs. Array Insertion

```
std::vector<int> v1, v2;
std::list<int> L1, L2;
...
// in a loop
  ...
  v1.insert(v1.begin(), 1);
  v2.insert(v2.end(), 1);
  L1.push_front(1);
  L2.push_back(1);
...
```

Vector is an order of magnitude faster for push_back



List insertion is O(1), at either end

O(1) Deletions

Demo

- In a list, deletions are also O(1)
- Need the link to the node before the one being deleted




```
// To remove p->next
p->next = p->next->next; // OK?
```

- Problem: Memory leak!

```
// The proper way to remove p->next
node* tmp = p->next; p->next = p->next->next; delete tmp;
if (tail==tmp) tail = p;  // To be strictly safe, do this before delete
```

Data-Structures & Algorithms

with

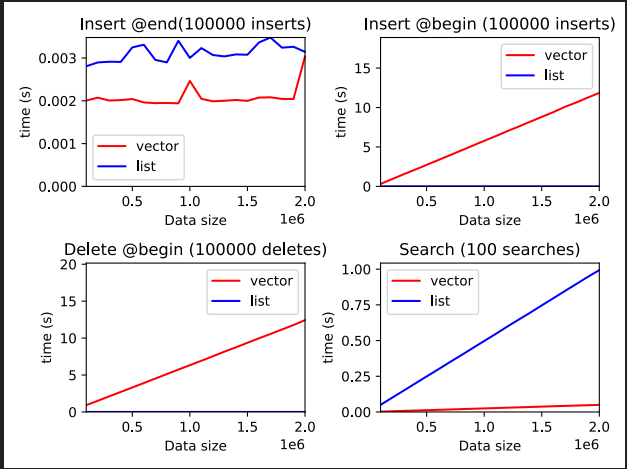
Manoj Prabhakaran

Lecture 4

Linked Lists: Doubly-Linked Lists

Recap: Lists vs. Arrays

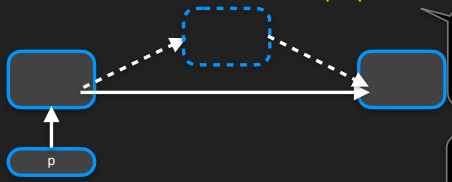
- Two simple data structures with Θ(n) searches
  - Concretely, searches in an array are faster (e.g., by an order of magnitude)
- Lists allow Θ(1) insertion and deletion anywhere in the list, including the beginning, but it is Θ(n) for vectors
  - Aside: insert/delete in vector is much faster than search, due to hardware support of bulk moves



O(1) Insertion and Deletion in a List

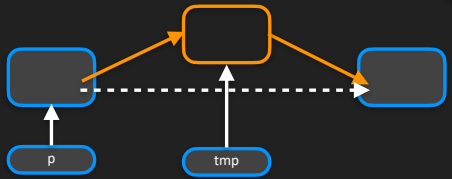
- Given a pointer to the node after which the insertion/deletion should be

```
// to insert a new node with val x after the node p points to
p->next = new node{x,p->next};
if(p==tail) tail = p->next;
```



Inserting as the head needs slightly different code

```
// to remove p->next, without memory leak
node* tmp = p->next;
p->next = p->next->next;
if (tmp==tail) tail = p;
delete tmp;
```



Deleting the head needs slightly different code

Why Linked Lists

- O(1) insertion and deletion
  - Once we have the location where insertion/deletion happens
  - No existing data nodes are moved during insertion/deletion (except the one deleted)
- Insertion/deletion will not invalidate a node pointer already at hand (unless the node it is pointed to is the one deleted)
  - Can hold search results for a long time, with intervening insertions/deletions (of other nodes)

Why Arrays

- If only insertions at the end needed, arrays are better than lists
  - An important feature of arrays: random access. Directly access the ith element in O(1) time
    - In a list, need to start from the beginning and sequentially proceed to the ith element, in O(n) time
  - Arrays have less space overhead, faster sequential access (no pointer to follow), faster push_back (no memory allocation each time), and guaranteed to be contiguous in physical memory, which allows random access and also leads to better hardware performance ("cache-friendly")

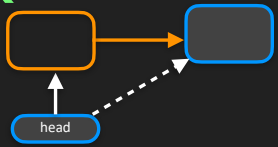
Packaging a List

- To work with a list (in particular to allow push_back and push_front), we need to maintain two pointers, to the front and the back of the list
- Why not make another struct which maintains these two pointers internally?
- E.g., `struct list { node* head; node* tail; /*functions*/};`
- push_back and push_front will automatically update tail/head pointers. User code doesn't have to maintain them (or any pointers)
- When a list object goes out of scope its destructor can deallocate all memory: `list::~list() { while (head) pop_front(); }`

pop_front and pop_back

- Popping from the front of a list:

```
void pop_front() {
  if (!head) return;
  node* tmp = head;
  head = head->next;
  if (tmp==tail) tail = nullptr;
  delete tmp;
}
```



Almost the same code as deleting p->next, but with p->next replaced with head.

- How about pop_back? We need the pointer to the node before tail (so that we can make tail point to it) without O(n) search.
  - Maintain one more pointer in struct list? But how do we update that then?
  - Idea: Every node maintains a pointer to the previous node as well.

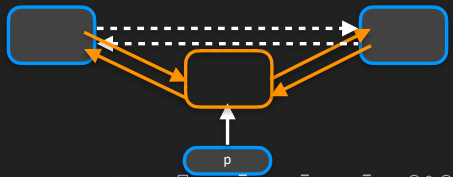
Doubly Linked List

- ```
  struct node {
    int val; node* next = nullptr; node* prev = nullptr;
  };
  ```

- Now, to delete a node, enough to have a pointer to that node itself

```
void list::erase(node* p) { // assumes p is a non-null pointer
  if(p==head) head=p->next; else p->prev->next = p->next;
  if(p==tail) tail=p->prev; else p->next->prev = p->prev;
  delete p;
}
```

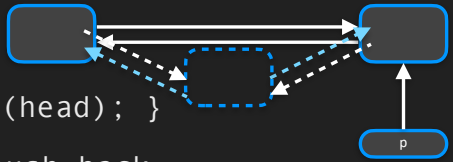
- ```
  void list::pop_front() { erase(head); }
  void list::pop_back()  { erase(tail); }
  ```



Doubly Linked List: Insert

- `void list::insert_before(node* p, int x) { // p non-null`

```
  node* tmp = new node{x,p,p->prev};
  if(p==head) head=tmp; else p->prev->next = tmp;
  p->prev = tmp;
}
```

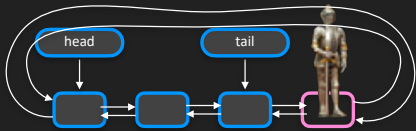


- `void list::push_front() { insert_before(head); }`
- Can symmetrically define `insert_after` and `push_back`
  - Or can do `insert_before(p->next, x)` unless p is `nullptr`
  - Cannot use `insert_before` for `push_back`
- Can avoid special cases using a “sentinel” node (like in std::list)

Sentinel Node

- The code for list operations become cleaner/more uniform if we use a sentinel "end" node, beyond the tail
- The prev field of sentinel will point to the tail. No separate tail pointer.
  - Avoids separate check for inserting at tail (which is now just a standard insert before the sentinel)
- Sentinel's next field can store the head. No separate head pointer.
- head's prev can point to sentinel: avoids special cases for head
- Keep just the sentinel node itself in the list

```
struct list { node sentinel; /*functions*/};
```



Data-Structures & Algorithms

with

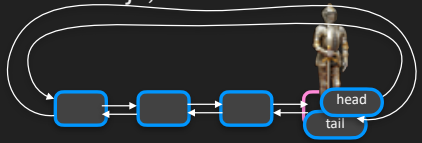
Manoj Prabhakaran

Lecture 5

Linked Lists: Some Cool Tricks!

Recap: Doubly-Linked List w/ Sentinel Node

- `struct node { int val; node *next, *prev; };`
  `struct list { node sentinel; /*functions*/};`
- Insert/Erase code is simplified, and without if-else conditions
  - Per list, one "`val`" field in the sentinel is wasted
  - A significant issue only if a large number of *small* lists are being used



Recap: Doubly-Linked List w/ Sentinel Node

```
struct node { int val; node *next, *prev; };
struct list {
  node sentinel; // sentinel.next is head, sentinel.prev is tail
  list() { sentinel.next = sentinel.prev = &sentinel;} // empty list. no nullptr
  node* begin() { return sentinel.next; } // begin points to head
  node* end() { return &sentinel; }  // end points beyond tail, to the sentinel itself
  void insert(int x, node* p) { // insert before. for insert_after can use insert(x,p->next)
    node* tmp = new node{x,p,p->prev};
    p->prev->next = tmp;  // no special case for p being head
    p->prev = tmp;
  }
  void erase(node* p) {  // assumes p a non-nullptr
    if(p==end()) return; // sentinel cannot be removed
    p->prev->next = p->next;
    p->next->prev = p->prev;
    delete p;
  }
  // Exercise: ~list(), find(), size(), reverse(), join(), ... Add members as needed.
};
```



In std::list

- Very similar to our doubly-linked list with a sentinel. Needn't be circular.
- Doesn't expose a separate node struct. Instead of node* uses a type called `std::list::iterator`
  - Returned by `begin()` and `end()`
  - Supports `*`, `->`, `++`, `--`, `==`, `!=`
    - ⚠ Effects of `--L.begin()`, `++L.end()` and `*(L.end())` undefined
  - `std::find(L.begin(),L.end(),x)` returns an iterator to where the search ended — a node in the list with value x, or otherwise `L.end()`

`*it` instead of `p->val`. `it->field` instead of `p->val.field`

`++it` and `--it` instead of `p=p->next`, `p=p->prev`

Some Clever Algorithms

- Today
  - An algorithm for efficiently finding a cycle in a (malformed) linked list
  - An algorithm for detecting intersection of two linked lists
  - Lists reversible in O(1) time
- First:
  - Merging two sorted linked lists (a useful subroutine in the mergesort algorithm)

Merging Sorted Lists



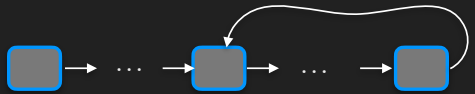
- Problem: Given two sorted (singly) linked lists, L1, L2, merge them into a single sorted list, using O(1) additional space, and without moving any nodes (so that pointers to nodes remain valid after merging)
- Idea: "detach" a node from the front of one of the two lists (whichever has the smaller value) and "attach" it to the back of a merged list

```
struct node { int val; node* next = nullptr; };
struct list {
   node* head=nullptr; node* tail = nullptr;
   bool empty() { return (head==nullptr); }
   node* detach_front(); // doesn't delete detached node
   void attach_back(node* p); // doesn't make a new node
};
```

Exercise

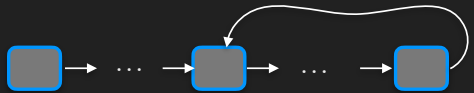
Cycle Detection

- Recall a singly-linked list
- What if, when creating the list, by mistake, a node points to the head?
  - This is a (singly-linked) circular linked list
  - How can you check if this has happened?
    - Easy: Traverse the list until we hit a nullptr or the head again
- A trickier situation: what if a node points to some previous node (not necessarily the head)?
  - Don't know which node will repeat



Cycle Detection

- With $\Theta(n)$ extra space, can solve it easily
  - If each node has space that can be used to "mark" it, then just mark all nodes seen during a traversal. Cycle iff we see a previously marked node.
  - Create a hash table (or set) of all the node addresses encountered, looking for duplicates while adding.
- With just O(1) extra space?
  - In $\Theta(n^2)$ time: At each node, check if its next node is reached first, when starting from the head, through another node: Extra time = 1 + 2 + … + n
- In O(1) extra space and O(n) time?

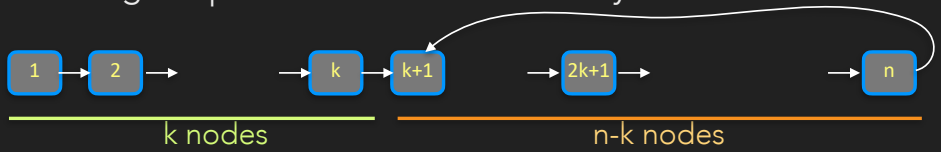


Floyd's Cycle Detection

- ```
  node* slow = head; node* fast = head;
  while(fast && fast->next) {
       slow=slow->next; fast=fast->next->next;
       if(slow == fast) return true; // cycle!
  }
  return false; // no cycle
  ```
  - In a straight list: `slow` will never catch up with `fast`, before `fast` (or `fast->next`) reaches the end of the list and we will return false
  - If there is a cycle: at some point both slow and fast will be in the cycle (because once you enter, you never leave). After that `fast` is chasing and catching up with `slow`, reducing the gap by one at each step. The two will meet!
  ```

Floyd's Cycle Detection

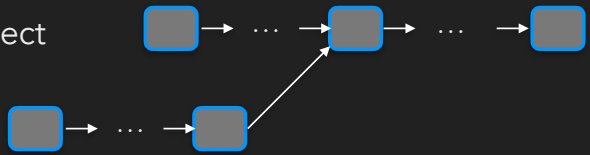
- We can even get a pointer to the start of the cycle!



- For simplicity, let us pretend n >> k.
- In k steps: slow reaches node k+1, fast reaches node 2k+1 (within the cycle)
- In the cycle, slow is (n-2k) steps ahead of fast (slow is at "position n+1")
  - In n-2k more steps, they meet at position n-k+1 (or 2(n-k)+1)
  - Now move one pointer to head, and move both forward in sync till they meet: In k more steps both reach position k+1

Intersection Detection

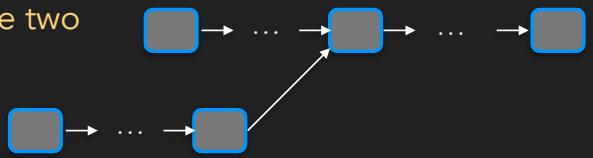
- Suppose two (singly-linked) lists intersect
- To find the first node that belongs to both lists



- Easy using O(n) extra space: Store all the addresses of the nodes of one list in a data structure (set or hash table), and go through each element of the second list checking if it is stored already
- Easy using $O(n^2)$ time and O(1) extra space: For each node in the second list, scan the entire first list to see if it is present there
- Can we do it in O(n) time with O(1) extra space?

Intersection Detection

- Suppose we were guaranteed that the two lists have the same length
- Then, there is some index i such that both lists have their i^{th} node identical



  - Traverse the two lists in synch, checking if both are at the same node
- What if we're not guaranteed same length?
- We can find the lengths of both the lists in O(n) time
- If the lengths are not the same, we can skip the first nodes in the longer list, so that it is left with exactly as many nodes as the shorter one. Now as above.

O(1) Reversal

Challenge: Design it so that it supports O(1) reversal of segments

Hint: For each node, allow either pointer to be next; no need to store this bit, but it can be inferred at traversal time (how?)

How will you redefine O(1) insert?

- Consider designing a doubly-linked list
  - Suppose we want to design it so that it can be quickly reversed
- Idea: store a bit in the list indicating which is the forward direction:

```
struct node { int val; node *A, *B; };
struct list { node S; bool A_is_next; /*..*/};
// if A_is_next, then A is head, else S.B is head
```

  - To reverse the list, simply toggle this bit:

```
void list::reverse() { A_is_next = !A_is_next; }
```

  - In other functions, instead of `p->next`, `p->prev`, use `next(p)`, `prev(p)`:

```
node*& list::next(node* p) { return A_is_next? p->A : p->B; }
node*& list::prev(node* p) { return A_is_next? p->B : p->A; }
```

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 6

Stack

Stack

- What is a stack?
  - Objects (say books) are kept one on top of the other
  - An item can be added only to the top of the stack
  - Can remove only the top-most item (the most recently added)
  - Last-In First-Out (LIFO)
- Why are stacks useful in programs?
  - To support "undo", a stack can used to store actions so far
  - Algorithms for parsing a string according to syntax rules ("grammar") use a stack
  - Systematic searches often use a stack too
  - Memory allocated for function calls naturally use a stack (last function invoked is the first to return)
  - …



A Quick Stack Implementation

```
struct charStack {
    vector<char> stk;
    bool pop(char& x) { // if stack is empty, return false
        if (stk.empty()) return false;
        x = stk.back(); stk.pop_back(); // back is top
        return true;
    }
    void push(char x) {
        stk.push_back(x); // back is top
    }
    void clear() { stk.clear(); }
    bool empty() { return stk.empty(); }
};
```

Example: Palindrome using a Stack

- Check if an input string is a palindrome

```
int main() {
  charStack st;
  char x, y;
  int n; std::cin >> n; // read the length of the string first
  for (int i=0; i<n/2; i++) { std::cin >> x; st.push(x); }
  if (n%2) std::cin >> x; // if n odd, ignore the middle character
  for (int i=0; i<n/2; i++) { // compare stored characters with rest
    std::cin >> x;
    st.pop(y); // guaranteed to succeed since we pushed n/2 items
    if(x != y) { std::cout << "Not a palindrome!" << std::endl; return 1; }
  }
  std::cout << "Palindrome!" << std::endl;
}
```

assuming no input error

Example: Undo Stack

```
bool docommand(char c, charStack& history); // push command into history
bool undo(charStack& history); // pop one command from history & reverse it
int main() {
  turtleSim();  // our commands are for the turtle (from CS101)
  cout << "Enter f/r/l for forward/right/left, u to undo, q to quit: ";
  charStack history; history.clear();
  char input;
  for(cin >> input; !cin.fail(); cin >> input) {
    if(input == 'q')
      break;
    else if (input == 'u')
      if(!undo(history)) // try undoing
        cerr << "Undo stack empty!\n";
    else if (!docommand(input,history)) // try doing
      cerr << "Ignoring unrecognized command" << endl;
  }
}
```

Example: Undo Stack

Demo

```
bool docommand(char c, charStack& history) {
  if (c == 'f' || c == 'r' || c == 'l') {
    history.push(c);
    if (c == 'f') forward(10);
    else if (c == 'r') right(90);
    else /* (c == 'l') */ left(90);
    return true;
  }
  return false;
}
```

```
bool undo(charStack& history) {
  char c;
  if(!history.pop(c)) {
    return false;
  } else {
    if (c == 'f') forward(-10);
    else if (c == 'r') right(-90);
    else /* (c == 'l') */ left(-90);
    return true;
  }
}
```

Vector Problems



- A vector provides more features than a stack
  - In particular, random access
  - But comes at a cost!
    - To keep data contiguous, reallocates memory and moves data, if vector grows beyond the reserved size.
      - Moving data from one vector to another costs O(n) time in the worst-case
- A stack implementation doesn't need to suffer this cost!

Lists for Stacks

- If we had an a priori bound on the stack size, an array/vector is an excellent fit for implementing a stack
  - It supports O(1) adding and deleting items at the end (stack's top), with concrete speeds much faster than a linked list
- The only problem is that the stack may outgrow the array
- Idea: If a fixed-size stack fills up, add a second fixed-size stack on top of it
  - The first stack should be accessible from the one on top (if the top one is fully popped): top stack should point to the one below it
  - A linked list of fixed-size stacks!

A Fixed-Capacity Stack

```
struct stackBlock {
   const int capacity = 512; // fixed capacity
   char stk[capacity]; // will use an array instead of std::vector
   int top = -1; // keep track of last element's index. -1 for empty.
   bool pop(char& x) { // if stack is empty, return false
        if (top < 0) return false;
        x = stk[top--]; // decrement top after retrieving into x
        return true;
    }
    bool try_push(char x) { // if stack already full, return false
        if(top == capacity-1) return false;
        stk[++top] = x; return true; // increment top before storing x
    }
    void clear() { top = -1; }
    bool empty() { return top==-1; }
};
```

A List of Bounded Stacks

```
struct stackNode { stackBlock block; stackNode* below = nullptr; }
struct stackList { // a list of stackNodes
  stackNode* head = nullptr; // top most stackNode
  bool empty() { return head==nullptr; } // stack empty iff list empty
  void push(char x) {
    if(head==nullptr) head = new stackNode; // if adding to empty list
    if(!head->block.try_push(x)) { // try pushing to top stack
      stackNode* tmp = new stackNode; tmp->below = head; head = tmp;
      head->block.try_push(x); // push again after adding a new node
    }
  }
  bool pop(char& x) {
    if (empty()) return false;
    head->block.pop(x); // if not empty, pop from top stack
    if(head->block.empty()) { // if it is empty after popping, delete it
      stackNode* tmp = head->below; delete head; head = tmp;
    }
    return true;
  }
};
```

Exercise: An Issue to Fix

- In this stackList, a series of strictly alternating pushes and pops at the beginning (or at the beginning of a new stackBlock), each push/pop will result in adding/deleting a stockBlock
  - Still O(1) worst-case, but larger constants
- An idea: Add a new block on top when the top block is full (like before); but don't delete the top block when it turns empty -- delete only when the block below becomes empty
  - Allow for the possibility that head is not where the top is
- Exercise

Forgetful Stack

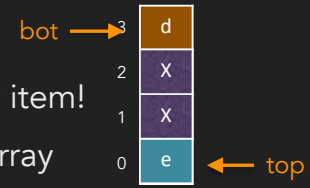
- An unlimited stack may not be suitable for many applications
- E.g., undo stack in an image editor
  - The information required to undo an action could be large (e.g., a snapshot of the image before a lossy blur operation)
  - Storing the undo information for all the commands in an editing session can grow to Giga Bytes of memory.
  - Better to have an a priori bound on the size of the undo stack
- Idea: When pushing a new item into a full stack, we can just forget the oldest item

Forgetful Stack

- How can you efficiently implement a forgetful stack
- Since fixed size, an array suffices: No need for a linked list or std::vector
- But what should we do once the array fills up?
  - Naively: Move all the items in the array to the left (letting the left-most item disappear) and make room for the new item at the right end
    - Problem: Push becomes $\Theta(n)$
  - We shouldn't move the items already in the stack, except possibly the one at the bottom (which is getting forgotten)
  - So we must overwrite the bottom element with the new item!

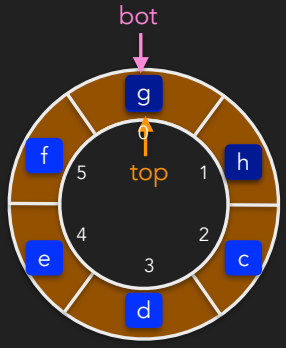
Forgetful Stack

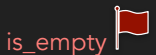
- How can you efficiently implement a forgetful stack
- Since fixed size, an array suffices: No need for a linked list or std::vector
- But what should we do once the array fills up?
  - We must overwrite the bottom element with the new item!
- Top position moves around, modulo N, the size of the array
- Bottom position also moves mod N (only when pushed up by top)
- Stack size can be any of N+1 values (0 through N), but the gap between top and bot can be only one of N values (0 through N-1)
  - Will use a flag to distinguish between two cases (zero or one item)



Forgetful Stack

```
struct stackRing {
  const int N = 512; // capacity
  char array[N];
  int top = 0, bot = 0;
  bool is_empty = true; // for us top==bot can mean 0 or 1 item
  void inc(int& i) { i = (i==N-1 ? 0:i+1); } // increment mod N
  void dec(int& i) { i = (i==0 ? N-1:i-1); } // decrement mod N
  void push(char x) {
    if(is_empty) is_empty = false; // top and bot stay put
    else { inc(top); if(top==bot) inc(bot); }
    array[top] = x;
  }
  bool pop(char& x) {
    if(is_empty) return false;
    x = array[top];
    if(top==bot) is_empty = true; // had one item: don't move top
    else dec(top);
    return true;
  }
};
```





Summary

- A stack provides only push and pop operations: sufficient for many interesting applications
- No random access and hence no need to maintain items contiguously
  - Implementation using a vector is not a good fit
- We saw three implementations with O(1) worst-case push/pop
  - stackBlock: A stack with a pre-determined capacity, using an array
  - stackList: An (uncapped) stack using a list of stackBlocks
  - stackRing: A stack with a pre-determined capacity, that forgets oldest items, using an array

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 7

Queue

Queue

- What is a queue?
  - An item can be added only to the back of the queue
  - Can remove only the front-most item
  - First-In First-Out (FIFO)
- Why are queues useful in programs?
  - Handling events (key presses, mouse clicks, network packets arriving at a router, …) in the order in which they happened
  - Maintaining a window of recent items: Add new item, remove old item
    - A forgetful stack (without any popping) will work too!
  - Systematic searches (We said that about stacks too!)
  - …



Queue

- Can implement using a linked list, and have O(1) enqueue and dequeue operations
- A quick implementation using a vector?
  - Say a fixed-capacity queue?
  - Not immediate! Vector offers O(1) operations at only one end
- Will come back to this soon. For now, let's assume we have a fixed-capacity queue
- Can we implement a full-fledged queue given a fixed-capacity queue?
  - Yes! Similar to what we did for stacks!

A Queue as a List of Bounded Queues

```
struct queueBlock; // TODO
struct queueNode { queueBlock block; queueNode* behind = nullptr; }
struct queueList { // a list of queueNodes
  queueNode *head = nullptr, *tail = nullptr;
  bool empty() { return head==nullptr; } // queue empty iff list empty
  void enqueue(char x) { // will enqueue into tail node
    if(tail==nullptr) head = tail = new queueNode; // if empty list
    if(!tail->block.try_enqueue(x)) { // may fail if tail is full
      queueNode* tmp = new queueNode; tail->behind = tmp; tail = tmp;
      tail->block.try_enqueue(x); // try again after adding a new node
    }
  }
  bool dequeue(char& x) {
    if (empty()) return false;
    head->block.dequeue(x); // if not empty, dequeue from front block
    if(head->block.empty()) { // if it is empty after dequeue, delete it
      queueNode* tmp = head->behind; delete head; head = tmp;
      if(head==nullptr) tail = nullptr;
    }
    return true;
  }
};
```

```
bool front(char& x) {
  if(empty()) return false;
  return head->block.front(x);
}
```

An Application

- A simple moving average is used in many situations to monitor real-time performance of systems
  - E.g., "telemetry data" measure system load at any point. If the load has been high for sometime, need to allocate more resources
  - Don't want to allocate resources just when the telemetry data spikes (the spike may pass)
  - Given telemetry data x_i, use $a_t = \frac{s_t}{N}$ where $s_t = x_t + x_{t-1} + \ldots x_{t-N+1}$
  - Can update from s_{t-1} to s_t in O(1) time: $s_t = s_{t-1} + x_t - x_{t-N}$
    - But need to remember x_{t-N} from time $t - N$ till time t

Simple Moving Average

```
#include <queue>

struct Averager {
  const size_t N = 50; // window (size_t, as returned by std::queue::size())
  std::queue<double> data;
  double sum = 0;
  void ingest(float x) {
    data.push(x); sum += x;
    if(data.size() > N) { sum -= data.front(); data.pop(); }
  }
  bool getAverage(double& a) {
    if(data.size() < N) return false;
    a = sum/N; return true;
  }
};
```

std::queue uses push(), pop() instead of enqueue(), dequeue()

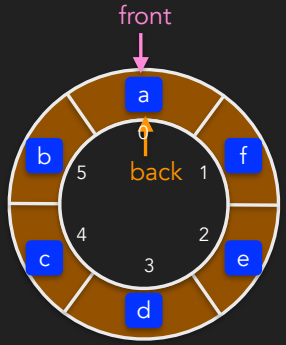
pop() doesn't return the dequeued value; use front() to read it before popping.

Fixed-Capacity Queue

- Our problem: Vector offers O(1) operations at only one end, but queue needs operations at both ends
- We already solved a very similar problem
  - Recall Forgetful Stack
    - Had to push/pop items at one end
    - But also had to forget from the other end!
  - We implemented it using a circular array (array indexed modulo the size of the array)
  - Same idea works for fixed-capacity queues!

A Fixed-Capacity Queue

```
struct queueBlock {
  const int N = 512; char Q[N]; // fixed capacity Q
  int front = 0, back = 0; // Can be any index with front==back
  bool is_empty = true; // front==back can mean 0 or 1 item
  void dec(int& i) { i = (i==0 ? N-1:i-1); } // decrement mod N
  bool full() {return (back==0 && front==N-1)||(back==front+1);}
  bool try_enqueue(char x) {
    if(full()) return false;
    if(is_empty) is_empty = false; else dec(back);
    Q[back] = x; return true;
  }
  bool dequeue(char& x) {
    if(is_empty) return false;
    x = Q[front];
    if(front==back) is_empty = true; // had one item, now empty
    else dec(front); // if not emptying, decrement front
    return true;
  }
  bool front(char& x) { x = Q[front]; return !is_empty; }
};
```



is_empty

Summary: Implementing a Queue

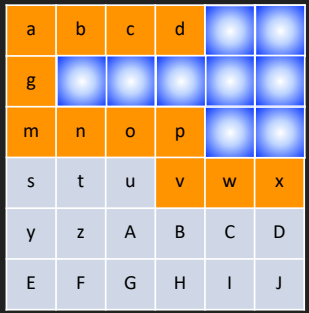
- Fixed-Capacity queue from circular arrays
- Full-fledged queue from a linked list of fixed-capacity queues
  - O(1) inserts and deletes
- The constant hidden in O(1) can be large if every insertion/deletion needs to add/delete a node in the list
  - Recall: Same issue in our stackList implementation last time
  - Mitigated by removing nodes slightly lazily, so that adding/removing nodes will be required only once every B operations (or more), B being queueBlock capacity
    - Same fix applies here too

Another Application: Systematic Exploration

- In the game of minesweeper, some cells have mines under them. Other cells either have a neighbouring cell that is mined (dangerous cells) or not (safe cells).
- When you click on a safe cell, it opens up an “island” of safe cells surrounded by a sea of dangerous cells
- How do you find the cells belonging to this island?
- High-level idea: Systematically explore starting from the clicked cells and moving to its safe neighbours
  - Concretely: Maintain a queue of safe cells encountered so far, and checkout the neighbours of each cell in the queue

Another Application: Systematic Exploration

- *If the clicked cell is safe, start by enqueuing it*
- *Repeat while queue not empty:*
  - *Dequeue a cell*
  - *Enqueue all its neighbours that are safe but were not already enqueued*
- Claim: Cells enqueued are exactly those in the island (i.e., connected to the starting node by a "path" of safe cells)



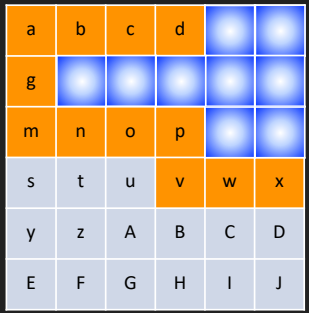
Can you *prove* this?
Hint: Use induction on the length of the path

Later

[f] ➡ [e k l] ➡ [q r e k] ➡ [j q r e] ➡ [j q r] ➡ [j q] ➡ [j] ➡ [i] ➡ [h] ➡ []

Another Application: Systematic Exploration

- What if we had used a stack or any other container?
- *If the clicked cell is safe, insert it into the container*
- *Repeat while container not empty:*
  - *Remove a cell from the container*
  - *Insert all its neighbours that are safe but were not already inserted*
- Still works! But the order of discovery can be different
  - With queue, cells closer to the start are discovered before the further off ones



[f] ➡ [l k e] ➡ [l k j] ➡ [l k q i] ➡ [l k q h] ➡ [l k q] ➡ [l k r] ➡ [l k] ➡ [l] ➡ []

Summary

- A queue provides only enqueue (push) and dequeue (pop) operations
- No random access and hence no need to maintain items contiguously
- Needs O(1) operations at both ends: Can't use std::vector
- We saw two implementations with O(1) worst-case push/pop
  - queueBlock: A queue with a pre-determined capacity, using a circular array
  - queueList: An (uncapped) queue using a list of queueBlocks
- Applications discussed: Processing items in the order they arrived, calculating Simple Moving Averages, systematic exploration.

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 8

Double-Ended Queues

And Amortisation

Deque

A feature of std::deque, which makes it more versatile

- What is a deque?
  - Can add items to either end of a deque (`push_back`, `push_front`)
  - Can remove from either end too (`pop_back`, `pop_front`)
  - Can read from either end (`front()`, `back()`)
  - Random access: Can read the ith item (0th being front) (using `[i]` or `at(i)`)
- Why are deques useful in programs?
  - Used in C++ STL to implement stacks and queues! A deque has all the methods needed by either (and also by vectors)
  - Deque can implement a forgetful stack. Or more custom versions where forgetting depends on the element (not based on the capacity)
  - Systematic searches using a container to hold items to explore, allowing more flexibility, still with O(1) time insertions/deletions

Deque

- Can implement using a linked list, and have O(1) {push/pop}_{front/back}
  - Can we get better constants?
- As a list of fixed-capacity deques (dequeBlock)?
  - A circular array can give a dequeBlock (easy extension of our queueBlock from the last time)
- Almost works, but doesn't support random access!
- First idea (we will improve on this): Reallocate the dequeBlock with double the capacity, each time we outgrow it
  - O(N) worst-case but O(1) *amortised* cost

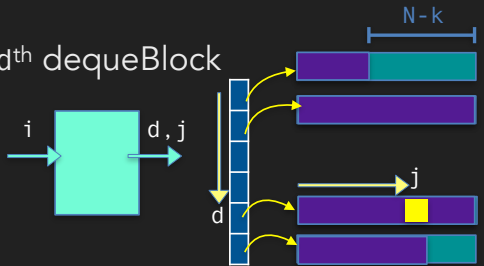
Amortised Cost

Important that we increase the size by a multiplicative factor

- Let us consider the reallocation cost of std::vector (same holds for our dequeBlock)
- Reallocation strategy: On reaching size M, we will reallocate the vector to have size 2M, and copy the M data items to the new location
  - We paid a reallocation cost of M after at least M/2 inserts since the last reallocation
  - Account this cost as spread across those M/2 inserts: each insert is charged an additional cost of 2 = O(1) moves
    - Important: An insert is charged only once — for the next reallocation
  - On average, the cost is still O(1)!

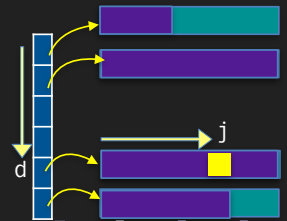
Deque

- For real-time systems, amortised guarantees are often not good enough
- Can we mitigate the O(N) worst-case cost? Maybe reduce the constant?
- A list of dequeBlocks almost worked, but didn't support random access
- What do we need for random access?
  - Translate an index `i` to `(d,j)` such that the item is at index `j` in the dth dequeBlock
- Use a "directory" to get a pointer to the dth dequeBlock
- If the first dequeBlock has `k` elements, `d = (i+N-k)/N`, and `j = (i+N-k)%N` (unless `i<k`, in which case `j=i`)



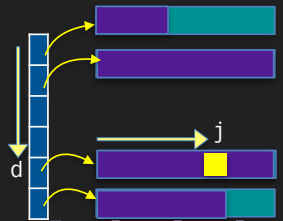
Deque

- directory[d] should have the pointer to the dth dequeBlock
- Should allow adding/removing dequeBlock entries to either end of the directory
  - We need a deque for the directory! But we are trying to build one…
  - Use a dequeBlock? But it will fill up…
    - We will reallocate when that happens! Not very expensive now!
    - Recall from our std::vector analysis: We spend O(M) time for every M entries in the directory
      - That is once for N = M×B items in the deque
    - O(1/B) amortised, O(N/B) worst-case cost



Deque: Exercise

- std::deque (just like std::vector) allows inserting and deleting from anywhere, in O(N) time
- We can actually guarantee O(B) + O(N/B) time
  - Which is technically still O(N) if you consider B a constant. But it is a large constant in practice, and can be modelled as a separate growing parameter.
- Idea: To insert/erase within a dequeBlock, we take time O(B). Then in a sequence of pop/push operations, update the O(N/B) dequeBlocks to one side of the index that is erased/inserted
- Exercise: do it!



An Application: Max in Window

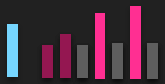
- Finding the max in a window in O(1) time
- Given a streaming sequence of numbers x_i, at any time t, find

  $m_t = \max\{x_t, x_{t-1}, \ldots, x_{t-N+1}\}$

  - Need to store many items from the stream: an x_i that is not the maximum in the current window can later become the maximum in a future window (after the current maximum drops out)
- Naive solution: Maintain the N-long window as a queue, and each time scan it to find the maximum. Total time after T inputs: O(NT)
  - Can we do it in time O(T) instead?

An Application: Max in Window

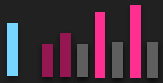
- Idea: Maintain the set of elements within the window that are eligible to be selected as max in the future



  - When a new element x_t arrives, all the elements in the set which are $\leq x_t$ can be removed
    - Because they will never get a chance to be the max again as this element will be in the window longer than they are
    - Note: Some elements would have been removed by previous items
- Refined idea: Maintain it as a stack of monotonically decreasing elements
  - When an item arrives, the elements less than it are at the top of the stack; pop them and then push the new item

An Application: Max in Window

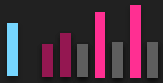
- Max element at any point is at the bottom of the stack
  - Will use a *deque* to access it
- How about items which get older than the window?
  - The queue is also sorted by arrival time
  - Remove items outside the window from the other end
    - Supported in a deque



An Application: Max in Window

```
#include <deque>

struct item {int val; unsigned int time; };
struct winMax {
  unsigned int now = 0; const int N=50;
  std::deque<item> window; // max candidates from last N items
  void ingest(int x) {
    // first remove at most one item that falls outside the window
    if(now>0 && now - window.front().time == N) window.pop_front();
    item xi{x,now++}; // prepare the new item, and increment time
    // remove all items superseded by the current item
    while(!window.empty() && window.back().val < x) window.pop_back();
    window.push_back(xi); // window always holds the newest item
  }
  bool getMax(item& m) { // window is sorted, max is always at the front
    if(now==0) return false; m = window.front(); return true;
  }
};
```



An Application: Max in Window

- How much time does this take?
- When an item is pushed into the stack, need to pop all smaller items, and this can take Θ(N) time in the worst-case
  - Since T such items, O(NT)
  - That is no better than our naive approach!
- But wait, we can count differently!
  - Each item is pushed in exactly once and popped out at most once (from either end of the deque). Θ(T) push/pop operations in all!
- Processing a single item can take Θ(N) time in the worst case, but that won't happen for every item. Total is O(T).

Amortised O(1) time per item

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 9

Stacks in Action

Course Logistics

- Quiz 1 (CS213 and CS293) dates announced
  - CS293: Tuesday Aug 25, during lab hours
  - CS213: Thursday Aug 27, lecture hour (rooms TBA)
- Partial-credits policy for Weekly-Lab Graded Submission announced
  - You have only two opportunities for claiming partial credits. Use wisely.
- CS213 Participation points: starting next week
  - Bring your device (laptop/phone)

Stack

Recall



- What is a stack?
  - Objects (say books) are kept one on top of the other
  - An item can be added only to the top of the stack
  - Can remove only the top-most item (the most recently added)
  - Last-In First-Out (LIFO)
- Why are stacks useful in programs?
  - To support "undo", a stack can used to store actions so far
  - Algorithms for parsing a string according to syntax rules ("grammar") use a stack
  - Systematic searches often use a stack too
  - Memory allocated for function calls naturally use a stack (last function invoked is the first to return)
  - …

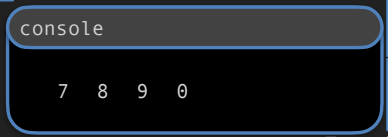
Today: Some More Examples

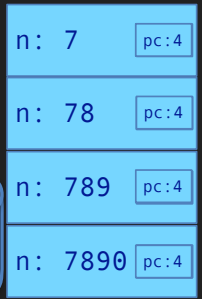
- Implicit stacks in recursive functions
- Balanced Brackets
- A Reverse Polish Notation (RPN) calculator
- An application of monotonic stack

Implicit Stack in Recursion

- When a function is called, memory for its local variables are allocated on a stack
- When the function returns, the stack is popped
- Consider calling the following function with input 7890

```
1. void printdigits(unsigned int n) {
2.  if(n>=10) { printdigits(n/10); }
3.  std::cout << " " << n%10;
4. }
```





Implicit Stack in Recursion

```
void printdigits_stack(unsigned int n) {
  std::stack<unsigned int> s; s.push(n);
  while(n>=10) { s.push(n/=10); }
  while (!s.empty()) { std::cout << " " << s.top()%10; s.pop(); }
}
```

```
1. void printdigits(unsigned int n) {
2.  if(n>=10) { printdigits(n/10); }
3.  std::cout << " " << n%10;
4. }
```

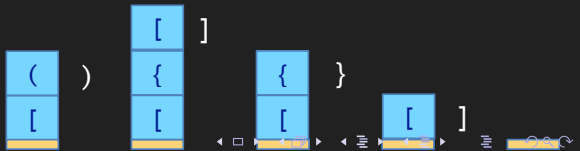
Balanced Brackets

- Consider a string with three kinds of brackets: `()`, `{}` and `[]`
- In a valid string, they should be properly nested: e.g., `[()]` is not OK
  - What is valid?
  - Has the form empty-string, `[`*valid*`]`, `(`*valid*`)`, `{`*valid*`}`, or *valid valid*
    - Either breaks into two shorter (non-empty) valid strings
    - Or the first character should be an opening bracket, and it should be closed/matched by the last one; after stripping off those two, the remaining should be a valid string (of shorter length)
  - How do we check if a string is valid: looking at the first/last characters can't tell which form it is: e.g., `[(){}]` vs. `[]()[]`

Balanced Brackets

- Check using a stack, scanning the string left-to-right once
- Each opening bracket is pushed into the stack
- When a closing bracket encountered, pop the stack to see if it matches
- While popping, if there is a mismatch or if the stack empty, unbalanced
- If the string ends while the stack is not empty, unbalanced
- Otherwise (the string ends with no mismatch, and the stack is empty at that point) balanced

[( ) { [ ] }]



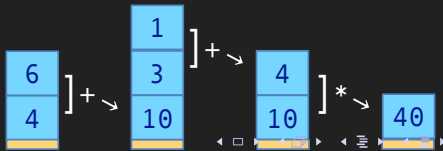
Balanced Brackets

- Claim: Exactly balanced bracket strings are accepted as balanced by this program
- Proof: By induction on the length of the string.
  - Base case: length 0 string. It is balanced, and our program accepts it as balanced ✅
  - Induction: For any k>0, suppose it holds for all strings of length < k. For a k-long string:
    - If valid, will accept: If of the form A B, where each is valid, non-empty, then by induction hypothesis, our stack will be empty after scanning A successfully, and then will be empty again after scanning B successfully. ✅ If of the form {A}, first we push { and then run the program on A, which by induction hypothesis, will return it to {, having never popped it; then on seeing }, will pop it. ✅
    - If accepts, then valid: If we accept, first character is one of [, (or {. Two cases: stack ever became empty before the end, or not. If it became empty after scanning a prefix A of a string AB, then it again became empty after scanning the suffix B; ie it accepts A and B each, and by induction hypothesis, both are valid. Then, by definition, AB is also valid. ✅ If it never became empty till the end, the first opening bracket, say {, stayed at the bottom of the stack throughout and was popped at the end by a matching closing bracket. So the string is of the form {A}; further, if run on A, the program will accept (all conditions met — check!). So A is valid by induction hypothesis. ✅

RPN Calculator

- Standard mathematical expressions need brackets
  - `(a+b)*(c+d)`
- In Reverse Polish Notation (RPN), no brackets needed
- In RPN, also known as postfix notation, the operator is placed after the operands (`arg1 arg2 op` instead of `arg1 op arg2`)
  - Instead of `(a+b)*(c+d)` we have `a b + c d + *`
- Makes it very easy to evaluate an expression in an input stream, using a stack of numbers

`4 6 + 3 1 + *`

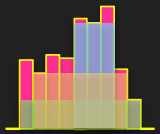


RPN Calculator

```
enum token_t { plus, minus, star, slash, num };
struct token { token_t ty;  double val; };
bool rpnCalc(const std::vector<token>& tokens, double& result) {
  std::stack<double> s;
  for(const token& t: tokens) { // iterate through all the tokens
    if (t.ty == num) { s.push(t.val); } // numbers go into the stack
    else { // apply operator on top two items on stack
      if (s.size() < 2) return false; // stack underflow!
      double b = s.top(); s.pop();
      double a = s.top(); s.pop();
      switch(t.ty) {
        case plus:  s.push(a + b); break;
        case minus: s.push(a - b); break;
        case star:  s.push(a * b); break;
        case slash: if (b == 0) return false; s.push(a / b); break; //divide by 0 is error
        default: return false; // won't get here (unless the code missed a token_t)
      }
    }
  }
  if (s.size() != 1) return false; // correct expressions leave a single value on stack
  result = s.top(); return true;
}
```

Largest Rectangle in a Patchwork of Strips

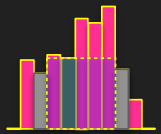
- Equal-width but varying-length strips of cloth are sewn together, with one end aligned, to form a large patchwork
  - What is the largest area of a rectangle aligned with the straight edge, that can be cut-out of this?



- Call a rectangle *maximal* if there is a "limiting strip" whose height (i.e., length) equals the rectangle's height, and the strips to the rectangle's immediate left and right are strictly shorter
- The largest rectangle is maximal, imagining sentinel strips of height 0 to either end
- For each strip we will find the area of the largest maximal rectangle for which that strip is a limiting strip
  - Then the max among those is the largest rectangle

Largest Rectangle in a Patchwork of Strips

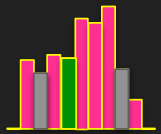
- For each strip we will find the area of the largest maximal rectangle for which that strip is a limiting strip
- We need to find the closest strip to the left and the closest strip to the right that are strictly shorter than this strip



- Naively: scan the strips to the left and to the right, starting from this strip. $\Theta(n)$ time.
  - To do this for all strips, we need $O(n^2)$ time. In fact, $\Theta(n^2)$ time (eg. if all the strips are the same length).
- Goal: $O(n)$ total time

Largest Rectangle in a Patchwork of Strips

- For each strip we will find the area of the largest maximal rectangle for which that strip is a limiting strip
- We need to find the closest strip to the left and the closest strip to the right that are strictly shorter than this strip



- Scanning left to right, push each strip into a stack such that heights are strictly monotonically increasing

Each item is pushed in once and pepped out once

  - When a strip enters the stack we pop entries taller than it. So its closest shorter strip to the left is just under it in the stack
  - When that strip is later popped, it is popped by its closest shorter strip to the right (the ones that didn't pop it are taller)
    - At that point, we know both the neighbours we were looking for



Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 10

Searching

Storing and Retrieving

- So far, Linked Lists, LIFO, FIFO, Deque
- O(1) insertion, deletion from either end
  - Or from anywhere, for linked lists
- For deque, O(1) random access
- What is missing?
  - Search! Given a "search key" retrieve an element from the data structure with the same key (or report there is no such element)
- Search key often supports a total ordering (==, <, >, <=, >= operators)
  - Numerical, lexicographical, ...
  - Full entry may be key + payload (e.g., key=roll no., payload=name)

Searching in a Vector

- Given a vector or array of (say) integers, check for the absence/ presence of various numbers in the array (return an index, or -1)

```
int search(std::vector<float> A, float query);
```

- Simplest idea, in Θ(n):

```
for (int i=0; i<A.size(); i++)
  if (A[i]==query) return i;
return -1;
```

- Can we do better? Yes, if the vector is sorted, *much better*!

Binary Search

- Consider searching for a number, say 3.14 in an array of sorted numbers
- Let us look at the middle first!



- We can eliminate half of the array from consideration now!
- To search in the rest of the array, we *recurse*!
  - Look at the middle of the remaining array
  - Eliminate half again
- Until a match found (a middle element equals the target) or all of the array eliminated

Binary Search

```
int srch (const std::vector<float>& A, float query, int left, int right) {
    if (right < left)
      return -1;                          // empty array: query not found
    int mid = (left+right)/2;
    if (query < A[mid])
      return srch(A,query,left,mid-1);    // recurse to the left of mid
    else if (A[mid] < query)
      return srch(A,query,mid+1,right);   // recurse to the right of mid
    else // query == A[mid]
      return mid;                         // found!
}
```

Binary Search: Non-Recursive Version

- How did we use the implicit stack in the recursion?
- We didn't! Can rewrite as a loop with constant extra space!

```
int srch_loop(vector<float>& A, float query) {
  int left = 0, right = A.size()-1, mid = (left+right)/2;
  while (right >= left && query != A[mid]) {
    if(query < A[mid]) right = mid-1;
    else left = mid+1;
    mid = (left+right)/2;
  }
  return (right < left) ? -1 : mid;
}
```

- Aside: Such rewriting possible whenever it is a "tail recursion"

Tail Recursion

- In many cases of recursion, the return value is the same as what the recursive call returns
- Such a recursion can be rewritten as a loop
- Avoids overheads of function calls and returns
  - Like pushing/popping a frame onto/from the function-call stack, copying arguments and return values
- Compilers may often do such rewriting automatically

```
resultType f(inputType x) {

  if (baseCase(x))
    return handleBaseCase(x);

  x = smallerProblem(x);

  return f(x);
}
```

```
resultType f(inputType x) {
  while (!baseCase(x)) {
    x = smallerProblem(x);
  }
  return handleBaseCase(x);
}
```

Tail Recursion

```
!( right < left
   || query == A[mid] )
```

```
int srch (std::vector<float>& A, float query, int left, int right) {
  if (right < left)
    return -1;
  int mid = (left+right)/2;
  if (query == A[mid])
      return mid;

  if (query < A[mid])
    right = mid-1;
  else
    left = mid+1;

  return srch(A,query,left,right);
}
```

```
int srch_loop(vector<float>& A, float query) {
  int left = 0, right = A.size()-1, mid;
  while (right >= left
         && query != A[mid=(left+right)/2]) {
    if(query < A[mid]) right = mid-1;
    else left = mid+1;
  }
  return (right < left) ? -1 : mid;
}
```

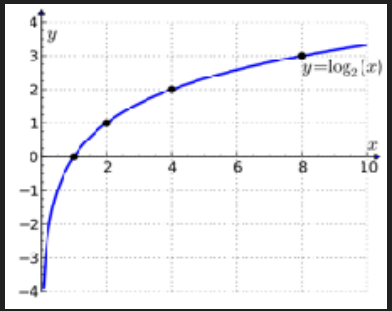
```
resultType f(inputType x) {
  while (!baseCase(x)) {
    x = smallerProblem(x);
  }
  return handleBaseCase(x);
}
```

How Many Steps?

- How many comparisons in a binary search over n elements?
- Important point: in each level of recursion, n reduces by at least half
  - Verify! (Check even n, odd n, left half, right half.)
- Suppose $n \leq 2^k$
  - Input after t recursive calls is at most 2^{k-t} long. At most k+1 calls before empty input.
  - So $O(\log_2 n)$ recursive calls
- $\log_2 n = (\log_2 10) \log_{10} n = (3.32..) \log_{10} n$
  - For O( ) notation, the base of the logarithm doesn't matter
- Binary search on a sorted array of n elements takes O(log n) steps

O(log n)

- Log is a very slow growing function
- When n doubles, $\log_2 n$ increases by just 1
- When n is multiplied by a factor 1000, $\log_2 n$ increases by less than 10
  - $\log_2$ (million) ≤ 20
  - $\log_2$ (billion) ≤ 30
  - $\log_2$ (trillion) ≤ 40
- The curve gets flatter (closer to horizontal) as n increases



Example: (Integer) Square Root

- Find $\lfloor \sqrt{x} \rfloor$ using binary search!
- Search in the implicit "sorted array" $\{ 0, 1, \cdots, x \}$

```
int isqrt(int x, int left=0, int right=0) {
    if(right==0) right = x; // set range [0..x] if called without a range
    if (right < left)       // we just passed the square-root
        return right;       // right has the smaller number
    int mid = (left+right)/2;
    int sqr = mid*mid;      // to check √x <=> mid, will check x <=> sqr
    if (x < sqr)
        return isqrt(x,left,mid-1);
    else if (sqr < x)
        return isqrt(x,mid+1,right);
    else // sqr == x
        return mid;
}
```

Example: Square Root

- Find $\sqrt{x}$ using binary search
- Search in the implicit "sorted array" of real numbers!

Adds one bit of precision in each iteration. Faster methods exist.

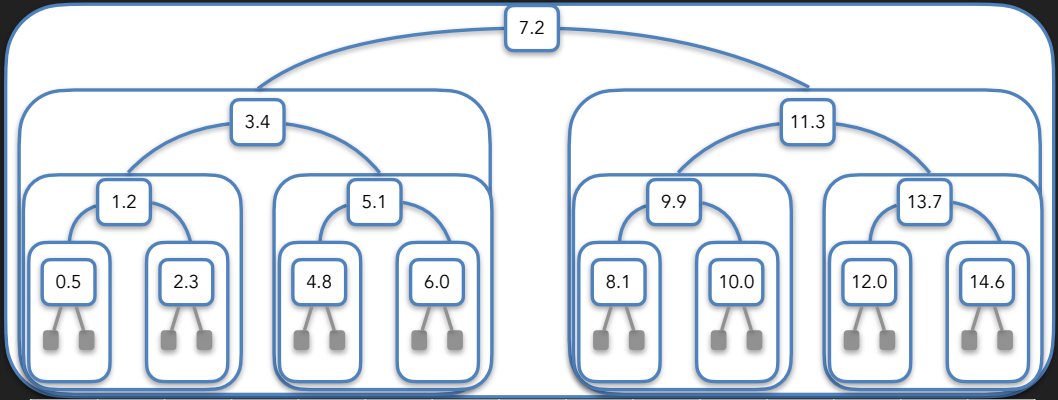
```
double sqrt(double x, double left=0, double right=0) { // assumes x >= 0
    if(right==0) right = std::max(1.0,x); //search [0,x], or if x<1, [0,1]
    double mid = (left+right)/2;
    if (mid==left || mid==right) //interval too small for double to bisect
        return mid;
    double sqr = mid*mid;
    if (x < sqr)
        return sqrt(x,left,mid);
    else if (x > sqr)
        return sqrt(x,mid,right);
    else // sqr == x
        return mid;
}
```

Binary Search Beyond Arrays

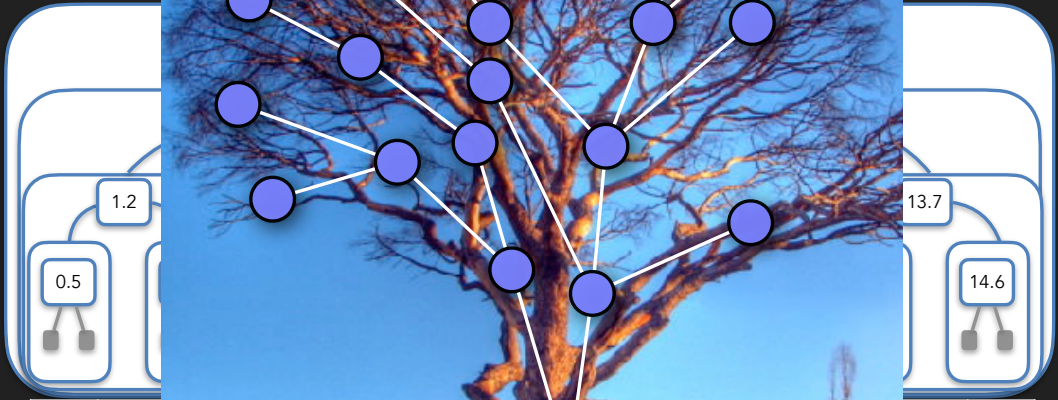
- Remember, an array or vector doesn't support quick insertions in the middle
  - Expensive ($O(n^2)$ time) to maintain a sorted array, if items keep arriving
- Can we keep data arranged so that we have the benefits of a sorted array plus quick insertions?
- That is,
  - O(log n) search
  - O(n) time to output in sorted order
  - Plus, O(log n) insertion time (Later: can't hope for better)

Binary Search Beyond Arrays

- Mimicking binary search: keep A[mid] directly accessible. Keep the left and right halves in the same recursive structure



- Mimicking ep the left and right h

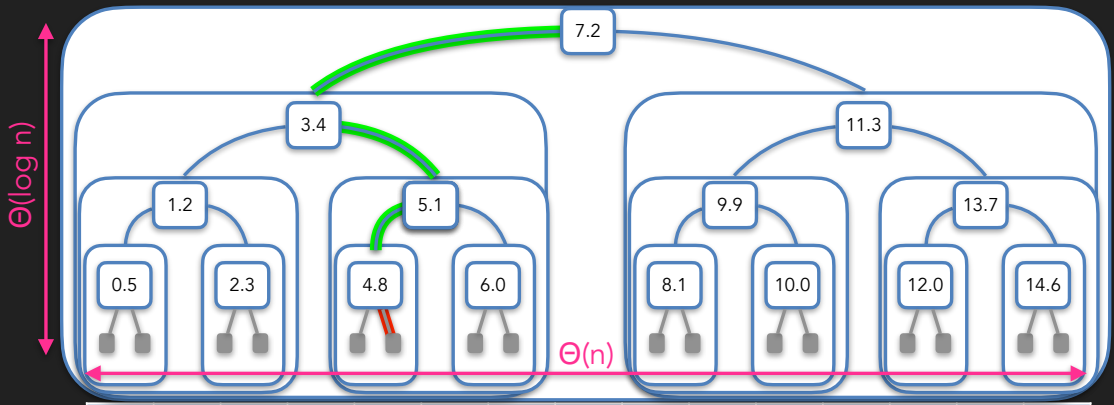


0.5	1.2					13.7	14.6

Binary Search Trees

- Searching: Follow the binary-search path "down" the tree

query = 5.0



0.5	1.2	2.3	3.4	4.8	5.1	6.0	7.2	8.1	9.9	10.0	11.3	12.0	13.7	14.6

Binary Search Trees

- Coming up:
  - Anatomy of a Binary Tree
  - Mechanics of a Binary Search Tree:
    - Already saw: Searching in O(log n) time
    - Next: Inserting, deleting a few elements in O(log n) time
      - Later: Inserting, deleting any number of elements

Data-Structures & Algorithms

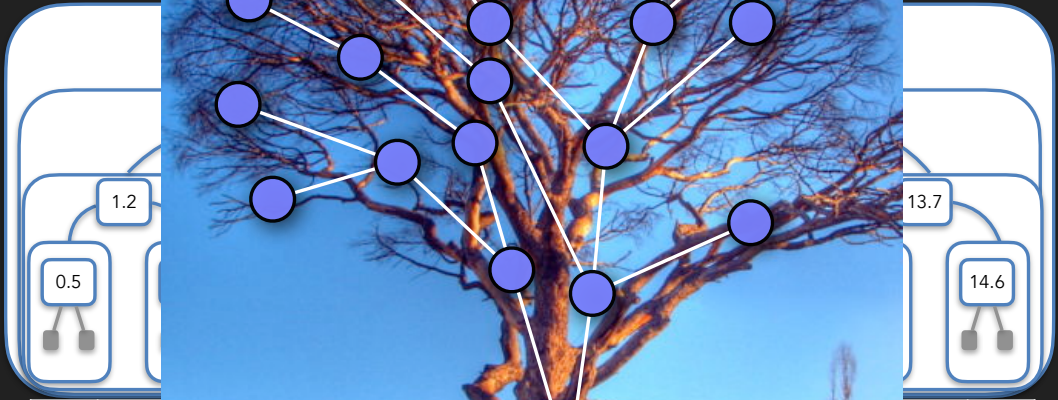
with

Manoj Prabhakaran

Lecture 11

Binary Trees

- Mimicking ep the left and right h



0.5	1.2					13.7	14.6

Anatomy of a Binary Tree

- A tree is a collection of "nodes", each of which can have links to other nodes ("child" nodes)
- A tree has no cycles
- A unique node designated as the "root" node
- Every node has a path from the root
  - Path: a sequence of nodes $v_1,\ldots,v_t$ where v_i has a link to v_{i+1}
- Every node other than the root has a unique parent node. Root has none.
- Every node has a unique path from the root
  - Starting form the given node, trace back through the parent at each node, until no parent (can't loop). We must be at the root.

Anatomy of a Binary Tree

- A node could be an internal node (carrying data), or could be an external or leaf node (without any data, and no link going out)
  - For us the leaves will be "missing" (pointer to a leaf is nullptr)
  - The root can be a leaf node (if the tree is empty)
- Each internal node has exactly two child nodes
  - Such a tree is called a *full* binary tree (we consider only full binary trees)
  - Note: any number of children (0, 1 or 2) can be leaves

Anatomy of a Binary Tree

- Depth of a node: How far is it from the root along this unique path (depth of root = 0)
  - Height of the tree = max depth over all nodes (i.e., over all leaves)
- Number of nodes, n = $O(2^{height})$
- But can't say height = O(log n)
  - In fact, in the worst case, height = $\Theta(n)$
    - The tree can look like a linked list!

Anatomy of a Binary Tree

- A complete binary tree is a full binary tree that has all leaves at the same depth
- A complete binary tree of height h has exactly $2^{h+1} - 1$ nodes, of which 2^h are leaves and 2^h-1 are internal nodes
- More generally, a full binary tree with n internal nodes has n+1 leaves

- Base case: n=0 (one leaf)
- Induction step: Consider k≥1. Suppose holds for n ≤ k-1. If n=k, consider a node of maximum depth; both its children are leaves (else it won't be max depth). Consider removing its two children, so that it becomes a leaf. The smaller tree is still full, and has n'=k-1 internal nodes and by induction, n'+1=k leaves. The original tree has one extra leaf (lose one leaf, gain two leaves): so k+1 leaves.

The Node Structure

- A simple implementation (actual STL implementations are more optimised)
- `struct node {int key; node *parent, *left, *right;}`
  - Parent pointer is optional, akin to `prev` in a (doubly) linked list
- A tree object can be a `node*` pointer to its root
  - Could legitimately be `nullptr` (when the tree is empty)
  - Alternately, a sentinel node could be used to point to the root node (to enable cleaner code without branching for insertion and deletion)

Traversing a Binary Tree

- Nodes in a tree can be visited systematically (and printed out as a sequence), using a recursive logic, starting from the root

```
void explore(node* tree) {
  if(tree==nullptr) return;
  std::cout << tree->key << " ";
  explore(tree->left);
  explore(tree->right);
}
```

pre-order

```
void explore(node* tree) {
  if(tree==nullptr) return;
  explore(tree->left);
  explore(tree->right);
  std::cout << tree->key << " ";
}
```

post-order

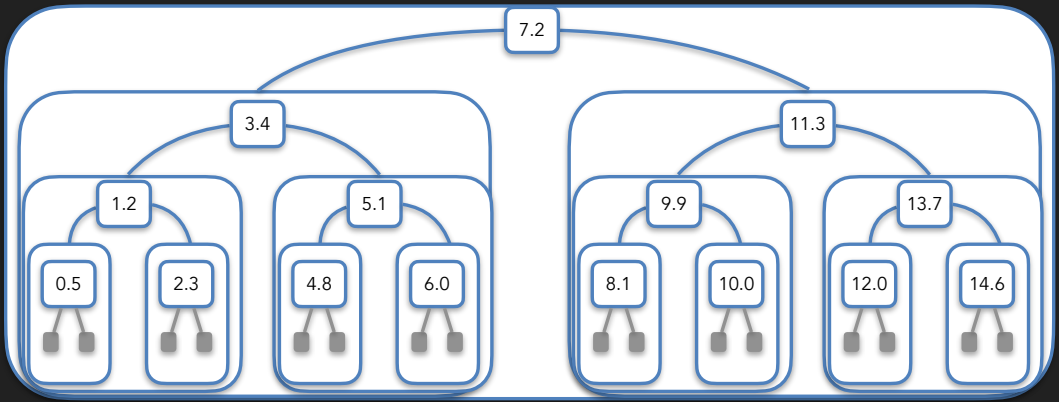
On a tree with n nodes and height h, these traversals take O(n) time, and O(h) extra memory (stack depth)

```
void explore(node* tree) {
  if(tree==nullptr) return;
  explore(tree->left);
  std::cout << tree->key << " ";
  explore(tree->right);
}
```

in-order

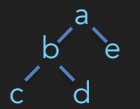
Same as reversed pre-order on mirrored tree

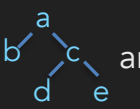
Traversing a Binary Tree

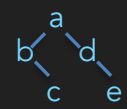


pre-order:	7.2	3.4	1.2	0.5	2.3	5.1	4.8	6.0	11.3	9.9	8.1	10.0	13.7	12.0	14.6
post-order:	0.5	2.3	1.2	4.8	6.0	5.1	3.4	8.1	10.0	9.9	12.0	14.6	13.7	11.3	7.2
in-order:	0.5	1.2	2.3	3.4	4.8	5.1	6.0	7.2	8.1	9.9	10.0	11.3	12.0	13.7	14.6

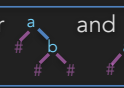
Traversing a Binary Tree

- Traversal sequence loses information about the actual structure of the tree
- E.g., pre-order sequence a b c d e for 

and 

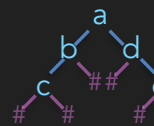
and 

- Can disambiguate using brackets: print "(value *left right*)"
- More compactly: enough to include leaf markers (for pre/post-order)

Not enough for in-order: # a # b # for 


```
void explore(node* tree) {
  if(tree==nullptr) { cout << "# "; return; }
  cout << tree->key << " ";
  explore(tree->left);
  explore(tree->right);
}
```

a b c # # # d # e # #



Data-Structures & Algorithms

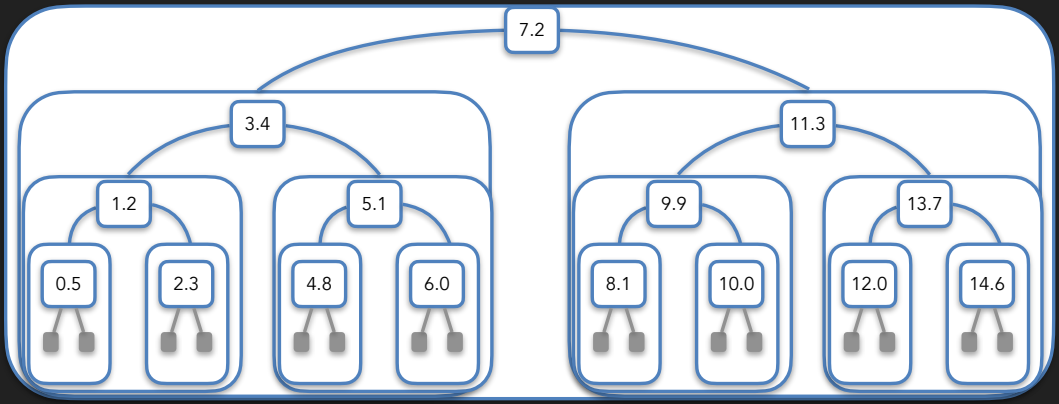
with

Manoj Prabhakaran

Lecture 12

Binary Search Trees

Binary Search Trees

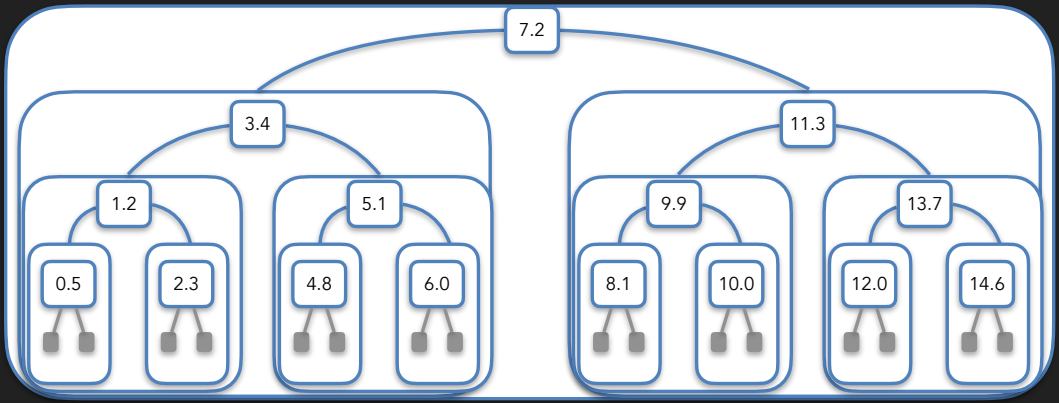


Defining a BST: A binary tree such that for any node with key K,

all nodes in its left subtree have keys < K, and

all nodes in its right subtree have keys ≥ K.

Binary Search Trees



in-order: 0.5 1.2 2.3 3.4 4.8 5.1 6.0 7.2 8.1 9.9 10.0 11.3 12.0 13.7 14.6

In a BST, the in-order sequence is sorted

Inserting into a BST

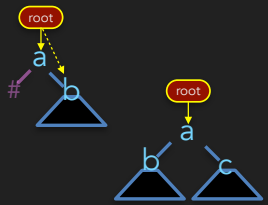
- When searching for a key not present in the tree, the search terminates at a leaf where the key could have been
- To insert a key:
  - First search for the key
    - Ignore if found! Pretend the key we are searching for is slightly larger (because equal keys go to the right sub-tree)
  - Grab the parent of the leaf where the search terminates
  - Create a new node with the key and insert it under this parent

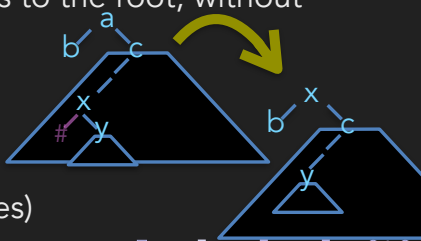
Inserting into a BST

```
struct node{int key; node *parent, *left=nullptr, *right=nullptr};
struct tree {
  node* root = nullptr;
  void insert(int v) {
   // search for the parent whose child the new node will be
    node* parent = nullptr;
    for(node* curr = root; curr!=nullptr; ) {
      parent = curr; // update parent before updating current
      if(v < curr->key) curr = curr->left;
      else /*if(v>=curr->key)*/ curr = curr->right;
    }
    // create and attach the new node
    node* newnode = new node{v,parent};
    if(parent == nullptr) root = newnode; // was an empty tree
    else if (v < parent->val) parent->left = newnode;
    else parent->right = newnode;
  }
};
```

Deleting from a BST

- Consider deleting the root from a (non-empty) tree
  - Has only leaves as its children? Simply make it a leaf (empty the tree)
  - Has one leaf and one internal node as children?
    - Promote the child internal node to be the root
  - If both children are internal nodes?
    - Plan: Bring a node from one of the sub-trees to the root, without moving any other node!
    - Minimum node on the right sub-tree
      - Detach that node from the sub-tree that it is root of (using one of the first two rules)





Deleting from a BST

```
struct node{int key; node *parent, *left=nullptr, *right=nullptr};
struct tree {
  node* root = nullptr;
  node*& where(node* p) { // which pointer in the tree is p
    if(p==root) return root;
    return (p->parent->left==p)? p->parent->left : p->parent->right;
  }
  node* minimum(node* p) { // min in subtree under p (p!=nullptr)
    while(p->left) {p = p->left;} // go as left as possible
    return p;
  }
  void detach(node* p); // detach but don't delete p (p!=nullptr)
  void erase(node* p) {if(!p) return; detach(p); delete p;}
};
```

Deleting from a BST

```
void tree::detach(node* p) { // detach but don't delete p (p!=nullptr)
  node*& pp = where(p);      // pp = where p is recorded in the tree
  if(!p->left || !p->right){ // if at least one leaf child
    pp = p->left ? p->left : p->right; // promote the other child
    if(pp) pp->parent = p->parent;
    return;
  }
  node* q = minimum(p->right);       // if no leaf child, find successor q
  detach(q);    // since q->left==nullptr, this will use the first case
  // replace p with q. update pointers to p
  pp = q; p->left->parent = q;       // p->left!=nullptr
  if(p->right) p->right->parent = q; // detach(q) can set p->right=nullptr
  // update pointers from q
  q->left = p->left; q->right = p->right; q->parent = p->parent;
  p->left = p->right = p->parent = nullptr; // optional: clean up p
}
```

Attendance Break!

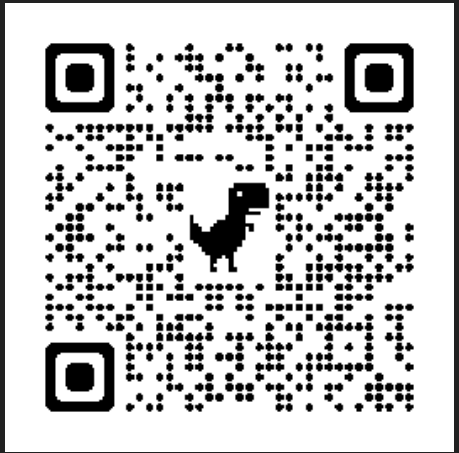
Login using CSE LDAP id at
https://www.cse.iitb.ac.in/course_vm/cs433

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DDAC!



Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 13

Traversing a Binary Tree

Today

- Traversal (pre/in/post-order) without recursion
  - Or a stack. With O(1) additional memory.
- Traversal starting at any node
- A closer look at in-order, pre-order and post-order traversal
  - And again, traversal starting at any node

Traversing a Binary Tree

- This is recursive. Can we write it as a loop with O(1) extra space?
- Not tail recursion (two calls)
- What is on the stack?
  - Below any node, is its parent node
  - But we can find out the parent node from the tree itself!
  - Also on the stack: the program counter (what is the next action)

```
void explore(node* tree) {
  if(tree==nullptr) return;
  explore(tree->left);
  std::cout << tree->key << " ";
  explore(tree->right);
}
```

in-order

```
void explore(node* tree) {
  if(tree==nullptr) return;
  std::cout << tree->key << " ";
  explore(tree->left);
  explore(tree->right);
}
```

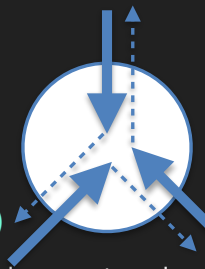
pre-order

```
void explore(node* tree) {
  if(tree==nullptr) return;
  explore(tree->left);
  explore(tree->right);
  std::cout << tree->key << " ";
}
```

post-order

Traversing a Binary Tree: Non-Recursive

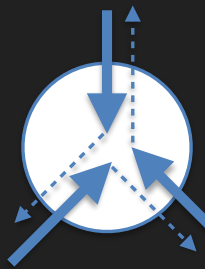
- When traversing the tree, we "enter" a node at 3 points:
  - Enter from the parent (and exit to the left child)
  - Re-enter when the left child returns (and exit to the right)
  - Re-enter when the right child returns (and exit to the parent)



- Current state of traversal (what gets stored in the stack) is fully determined by the current node, *and* which of the three ways we entered
- When re-entering a parent node, the point of entry (from left or from right) is determined by the current node
- So, when popping a state in the stack, we already know what is under it!
  - We don't need the stack! (But we do need parent pointers)

Traversing a Binary Tree: Non-Recursive

```
enum from_t { PARENT, LEFT, RIGHT };
struct state {
  node* p; from_t from; // p != nullptr except at the end
  void step() { // assumes p != nullptr
    switch(from) {
    case PARENT:                // go left
      if(p->left) { p=p->left; from = PARENT; return; }
      from = LEFT; return;      // visit+return from leaf
    case LEFT:                  // go right
      if(p->right) {p=p->right; from = PARENT; return; }
      from = RIGHT; return;     // visit+return from leaf
    case RIGHT:                 // return to parent
      from = (p->parent && p == p->parent->left) ?
                LEFT:RIGHT;     // p->parent can be nullptr
      p = p->parent;
    }
  }
};
```

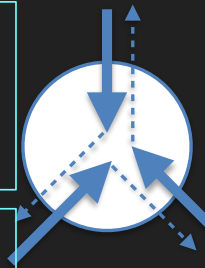


Traversing a Binary Tree: Non-Recursive

```
void explore_inorder(node* root) {
  for(state st={root,PARENT}; st.p; st.step()) {
    if(st.from == LEFT) std::cout << st.p->key << " ";
  }
}
```

```
void explore_preorder(node* root) {
  for(state st={root,PARENT}; st.p; st.step()) {
    if(st.from == PARENT) std::cout << st.p->key << " ";
  }
}
```

```
void explore_postorder(node* root) {
  for(state st={root,PARENT}; st.p; st.step()) {
    if(st.from == RIGHT) std::cout << st.p->key << " ";
  }
}
```



Attendance Break!

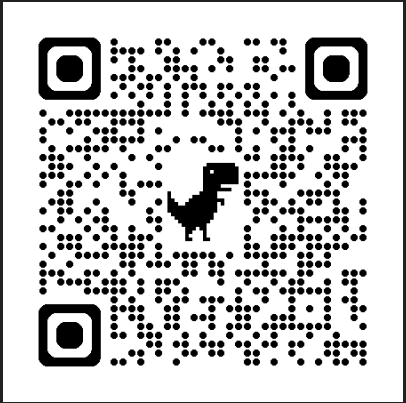
Login using CSE LDAP id at
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

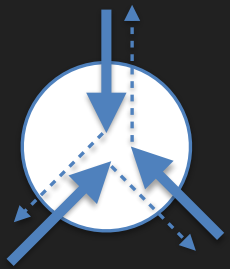
If you are not in the classroom, a report will be sent to DDAC!



Traversing a Binary Tree: Next

- Finding the next node in the traversal sequence (in the order printed)

```
node* next_inorder(node* p) {
  if(!p) return nullptr;
  state st{p, LEFT}; // pretend p just printed
  do st.step(); while(st.p && st.from != LEFT);
  return st.p;
}
```



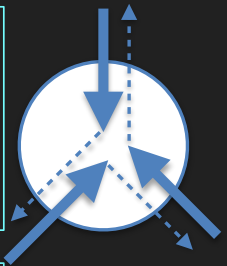
- Traversing starting at a given node

```
void explore_in(node* begin, node* end=nullptr) { // excluding end
  for(node* curr=begin; curr!=end; curr=next_inorder(curr))
    std::cout << curr->key << " ";
}
```

Traversing a Binary Tree: Next

- Finding the next node in the traversal sequence (in the order printed)

```
node* next_preorder(node* p) {
  if(!p) return nullptr;
  state st{p, PARENT}; // pretend p just printed
  do st.step(); while(st.p && st.from != PARENT);
  return st.p;
}
```



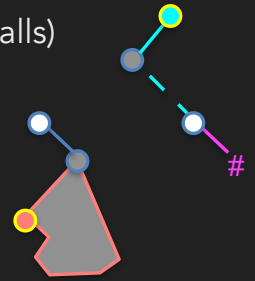
```
node* next_postorder(node* p) {
  if(!p) return nullptr;
  state st{p, RIGHT}; // pretend p just printed
  do st.step(); while(st.p && st.from != RIGHT);
  return st.p;
}
```

Traversing a Binary Tree: Next, Optimised

- A more direct version (avoiding multiple step function calls)

```
node* next_inorder(node* p) {
  if(!p) return nullptr;
  if(p->right) return first_inorder(p->right);
  else return next_up(p);
}
```

```
node* first_inorder(node* p) { // p not nullptr
  while(p->left) {p = p->left;}
  return p;
}
```



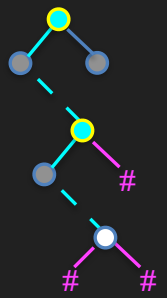
In a BST, this function computes the minimum in the subtree

```
node* next_up(node* p) { // p not nullptr
  while(p->parent && p->parent->right==p) {p = p->parent;}
  return p->parent;
}
```

Traversing a Binary Tree: Next, Optimised

- A more direct version (avoiding multiple step function calls)

```
node* next_preorder(node* p) {
  if(!p) return nullptr;
  if(p->left) return p->left;
  if(p->right) return p->right;
  do { p = next_up(p) } while (p && !p->right);
  return p ? p->right: nullptr;
}
```

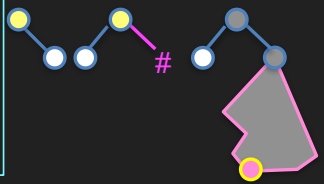



```
node* next_up(node* p) { // p not nullptr
  while(p->parent && p->parent->right==p) {p = p->parent;}
  return p->parent;
}
```

Traversing a Binary Tree: Next, Optimised

- A more direct version (avoiding multiple step function calls)

```
node* next_postorder(node* p) {
  if(!p || !p->parent) return nullptr;
  if(p==p->parent->right || !p->parent->right)
    return p->parent;
  return first_postorder(p->parent->right);
}
```




```
node* first_postorder(node* p) { // p not nullptr
  while(p->left || p->right) {p = p->left ? p->left : p->right;}
  return p;
}
```

Summary

- Using parent pointers, can traverse a binary tree with O(1) extra space
  - in-order, pre-order or post-order
- Can start from any node and continue traversal
  - `next()` function
- We saw two approaches to the next function
  - A clean, uniform approach using `state` and `step`
  - An optimised version handling different traversal orders differently

Data-Structures & Algorithms

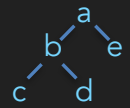
with

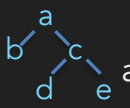
Manoj Prabhakaran

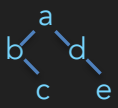
Lecture 14

More on BST

Binary Search Tree (BST)

- Recall: For a general binary tree, pre-order sequence is not sufficient for reconstructing the tree (unless leaves are printed)
- E.g., pre-order sequence a b c d e for 

and 

and 

- But a BST can be uniquely reconstructed from its pre-order sequence
  - Pre-order sequence uniquely determines a BST
  - There is O(n) time algorithm to generate this BST from the sequence

BST Fixed by Pre-order Sequence

- Why does the pre-order sequence uniquely determine a BST?
- Let's try an example:

7.2 3.4 1.2 0.5 2.3 5.1 4.8 6.0 11.3 9.9 8.1 10.0 13.7 12.0 14.6

- Pre-order sequence has the form:

root *pre-ord. seq. of left subtree* *pre-ord. seq. of right subtree*

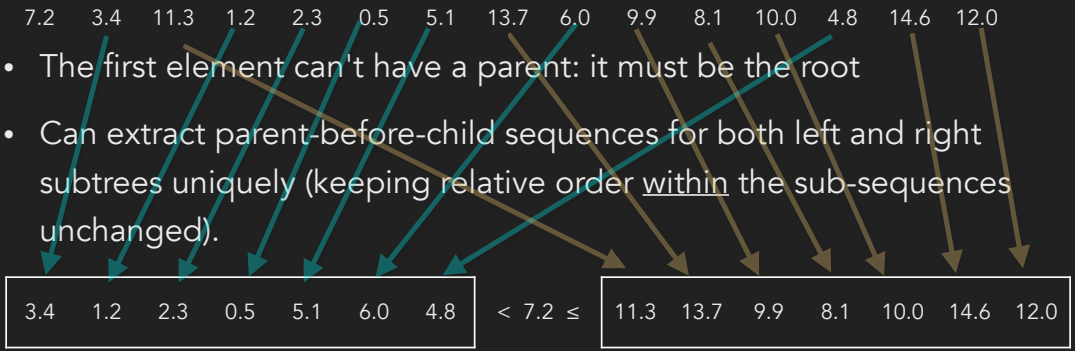
- So 7.2 is the root. Which ones are to its left?
  - They are exactly the ones < 7.2

7.2	3.4 1.2 0.5 2.3 5.1 4.8 6.0	11.3 9.9 8.1 10.0 13.7 12.0 14.6

- Inductively each of these two pre-order sub-sequences have a unique BST
- So only one BST possible for a given pre-order sequence

BST Fixed by Pre-order Sequence

- Only one BST possible for a given pre-order sequence.
- In fact, any parent-before-child sequence uniquely determines the BST



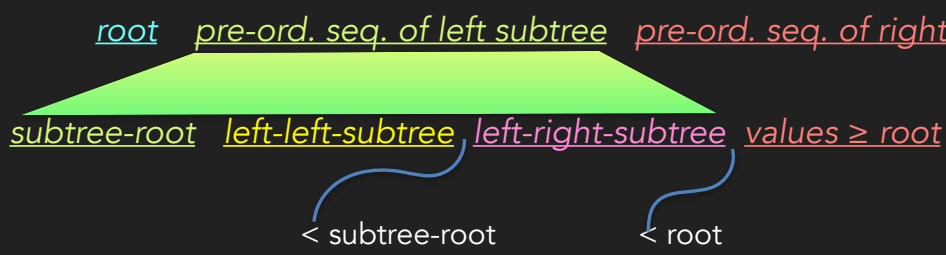
- The first element can't have a parent: it must be the root
- Can extract parent-before-child sequences for both left and right subtrees uniquely (keeping relative order within the sub-sequences unchanged).
- Inductively, unique BST for these two subtrees

BST From Pre-order Sequence

- How easily can we reconstruct the tree from a pre-order sequence?
- Just insert the elements into the tree in that order!
  - We only rely on the fact that it is a parent-before-child sequence
  - For a BST constructed by inserting elements x_1, x_2, ..., x_n in order (using the insertion algorithm from Lecture 12), that sequence is a parent-before-child sequence
    - Why? Once inserted, a node's parent doesn't change. The parent was already in the tree means it preceded this child.
  - And this is the unique BST with that sequence as *a* parent-before-child sequence
- Time is O(nh), where h is the height of the tree

BST From Pre-order Sequence

- Can we reconstruct the BST in O(n) time?
  - Not from general parent-before-child sequences!
    (Turns out the best possible is Θ(n log n) time)
  - But we can do it for pre-order sequences in O(n) time!
- Idea: Recall we can determine the boundary of a sub-tree's sub-sequence



BST From Pre-order Sequence

```
#include <queue>
#include <limits> // for MAX
const int MAX =  std::numeric_limits<int>::max(); // beyond values allowed
struct node{int key; node *parent, *left=nullptr, *right=nullptr};
// Takes reference to a queue which is popped by recursive calls
node* makeBST(std::queue<int>& preseq, node* parent=nullptr, int bound=MAX) {
  if(preseq.empty() || preseq.front() >= bound) return nullptr;
  int key = preseq.front(); preseq.pop();
  node* root = new node{key, parent};
  root->left = makeBST(preseq,root,key); // root's key is bound for left child
  root->right = makeBST(preseq,root,bound); // send given bound to right child
  return root;
}
```

Left subtree keys already popped by the previous call, since the queue is passed by reference

Attendance Break!

Login using CSE LDAP id at

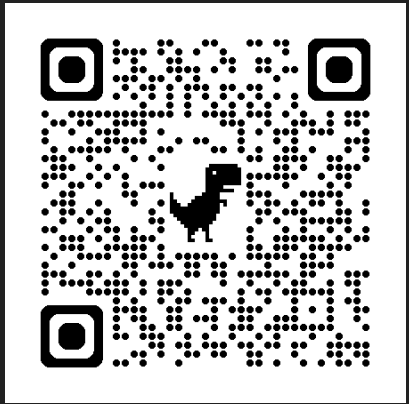
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DDAC!



BST in C++ STL

- Many containers in C++ STL internally use a BST
- `std::set`
  - Keys can be inserted, searched for, retrieved. Also can iterate through the elements in the set.
  - Allows only one copy of a key (inserting again has no effect)
- `std::map`
  - Holds "key-value pairs." Search by key, retrieve key and value.
    - Implemented just like `std::set`, but nodes have "payloads"
  - Requires unique keys (can replace a value by inserting again)
- `std::multiset` and `std::multimap`
  - Like std::set and std::map, but allows multiple entries with the same key

BST in C++ STL

- These containers can hold any data type which has `operatror<`
  - Other comparison operations can be defined in terms of <
    - Instead of `a >= b`, use `!(b<a)`
    - Instead of `a == b`, use `!(a<b || b<a)`
- The containers expose in-order traversal
  - `next_inorder()` is implemented as `operator++` for an iterator for the class
  - Also supports decrementing, `operator--`
- The internal state of the tree is not revealed by the interface
  - In particular, no pre-order or post-order traversal
- These containers offer O(log n)-time search, insert and delete
  - Keeps the height of the tree O(log n) by "*balancing*" [later]

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 15

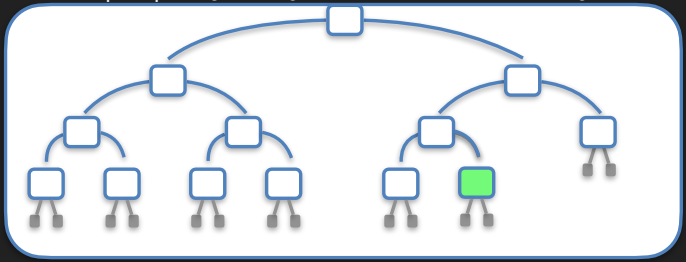
Heaps

BST

- Time for searching, inserting and deleting from a BST is all O(h), where h is the height of the BST
- Recall: the number of internal nodes a binary tree of height h can have is $n \leq n_{max} = 2^0 + 2^1 + \ldots + 2^h = 2^{h+1} - 1$
  - So $h \geq \log_2(n+1) - 1$. Ideally, we should keep h = O(log n), where n is the number of internal nodes
- But without additional balancing mechanisms, h can grow well beyond Θ(log n)
  - Demo

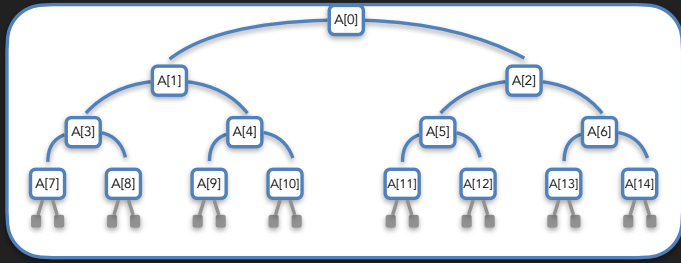
Nearly Complete Binary Tree

- In a nearly complete binary tree, only last level can be incomplete, and there all the internal nodes are to the left of all the leaves (if any)
  - $2^h \leq n < 2^{h+1}$ (recall convention: h=0 for n=1)
- If we didn't need to maintain the BST property, easy to maintain a nearly complete binary tree
  - Always insert and delete at the "bottom-most, right-most" position
    - To delete another node, first copy the bottom-right element to that node, and then remove the bottom-right node



Nearly Complete Binary Tree in an Array

- Can maintain a nearly complete binary tree without using any pointers, in a vector!
- Idea: Store the nodes (just the data) in a vector in "breadth-first order"
  - Contrast with "depth-first" order of pre/in/post-order traversal, where the left-subtree is finished before the right subtree is started



- `A[0]` is root. Children of `A[i]` are `A[2i+1]` and `A[2i+2]` (right/both possibly leaves)
  - Why? Verify base case of i=0. Now, if `A[i-1]` has children `A[j]`, `A[j+1]`, then `A[i]` has `A[j+2]`, `A[j+3]`. Inductively, `j=2i-1`.
- Supports insertion/deletion at the back of the vector

Attendance Break!

Login using CSE LDAP id at

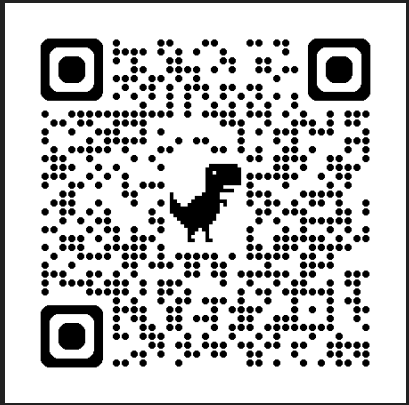
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Containers in C++ STL

- So far:
  - `std::vector` : Dynamically resized array
  - `std::list` : Doubly linked list (`std::forward_list`: singly-linked list)
  - `std::queue, std::stack, std::deque` : Deque has random access
  - `std::set, std::map, std::multiset, std::multimap`
    - Balanced Binary Search Tree (balancing covered later)
- Coming up later
  - `std::unordered_{set,map,multiset,multimap}`: Hash table
- Today:
  - `std::priority_queue` : Heap

Priority Queue

- Stack implemented LIFO, Queue implemented FIFO
  - Retrieval order irrespective of the actual content of the items
- But often we would like to assign items priorities as they are entered into a queue, and retrieve the highest priority item first
  - E.g., when urgent tasks should be processed first, not depending on how long they have been in the queue
  - Or, as a subroutine in many algorithms, where the "best" option so far should be explored first

Maintaining the Minimum

- Given a sequence of numbers, finding the minimum is easy
  - In O(n) time, with O(1) extra space
- How about finding the minimum, removing it, and finding the next minimum and so forth?
  - Equivalent to sorting! (Either task enables the other with little overhead)
  - Sorting n numbers takes Θ(n log n) time [later]. So best possible worst-case (or even amortised) bound is O(log n)
- Can we *repeatedly* find and remove the minimum in O(log n) *worst case* time?
  - Yes! Using a binary heap.

A Binary Heap

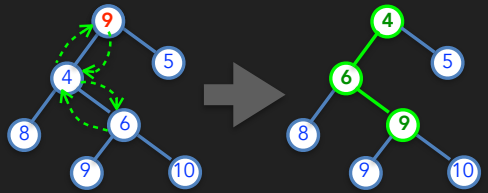
- A nearly-complete binary tree with the heap property:
  - A child is at least as big as its parent
- In every subtree, root is the minimum
- How do you insert/delete a node in O(log n) time?
  - Remember: to maintain the nearly-complete binary tree structure, we should add/remove at the bottom-right
  - To insert, add a node at the bottom-right and then update its value maintaining the heap property
  - To delete, remove the bottom-right entry and update the value of the node to be deleted to be the removed entry
- Subroutine needed: Update the value in a node of the heap while maintaining the heap property, in O(log n) time



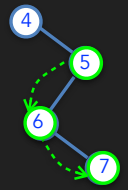
Changing a Single Node's Key

- Can bubble down or bubble up the changed node to fix the heap property
  - If update decreases the value of the node, bubble up. Else bubble down.
- Bubbling up algorithm:
  - Compare new key with parent; if not in heap order, swap; repeat until heap property fixed.
- Bubbling down algorithm:
  - Compare new key with both children; if not in heap order, swap with the smaller child; repeat until heap property fixed.
- Why does this work?





Changing a Single Node's Key



- Equivalent version: Instead of swapping, copy from next active node to previously active node. Insert new value only after finding its final position.
- Invariant: After each iteration, we have a valid heap
  - Initially true
  - If bubbling up: Active node's parent ≤ active node's children
  - If bubbling down: child copied up is the smallest among the active node and its children.
- On stopping, new key can be placed at the active node keeping heap valid
- Will stop in at most height many steps

Changing a Single Node's Key

```
void update(node* actv, int newkey) {
  if(actv->key > newkey) bubbleUp(actv,newkey); else bubbleDown(actv,newkey);
}
void bubbleUp(node* actv, int newkey) {
  for(; actv->parent && actv->parent->key > newkey; actv = actv->parent;)
    actv->key = actv->parent->key;
  actv->key = newkey; //outside the loop: newkey copied into actv only now
}
void bubbleDown(node* actv, int newkey) {
  for(bool done=false; !done; ) {
    int smallest = newkey;
    if(actv->left && actv->left->key < smallest) smallest = actv->left->key;
    if(actv->right && actv->right->key < smallest) smallest = actv->right->key;
    if(smallest==newkey) done = true;
    else { // bubble the smaller child up (left child guaranteed to exist)
      actv->key = smallest;
      actv = (smallest==actv->left->key) ? actv->left : actv->right;
    }
  }
  actv->key = newkey; //outside the loop: newkey copied into actv only now
}
```

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 16

Heaps

And Heap Sort

Last time we saw...

- A nearly complete binary tree can be implemented using an array of values
  - ```
    int Left(int i, int N) { return (2*i+1)>N-1 ? -1:(2*i+1); }
    int Right(int i, int N) { return (2*i+2)>N-1 ? -1:(2*i+2); }
    int Parent(int i) { return (i==0) ? -1:(i-1)/2; }
    ```
- A heap is a nearly complete binary tree with the heap property
  - So a heap can be implemented using an array!
    ```

Heap in an Array

```
struct heap {
  std::vector<int> Keys; // the actual data
  int Left(int i) { return (2*i+1) < Keys.size() ? (2*i+1) : -1; }
  int Right(int i) { return (2*i+2) < Keys.size() ? (2*i+2) : -1; }
  int Parent(int i) { return (i>0) ? (i-1)/2 : -1; }
  void bubbleUp(int i, int newkey);
  void bubbleDown(int i, int newkey);
  void push(int x) {Keys.push_back(0); bubbleUp(Keys.size()-1,x);}
  void pop() { // assumes heap not empty
    int x = Keys.back(); Keys.pop_back(); if(!Keys.empty()) bubbleDown(0,x);
  }
  int top() { return Keys[0]; } // assumes heap not empty
 /* more functions */
};
```

Heap in an Array

```
void heap::bubbleUp(int actv, int newkey) {
  for(; Parent(actv)!=-1 && Keys[Parent(actv)] > newkey; actv = Parent(actv);)
    Keys[actv] = Keys[Parent(actv)];
  Keys[actv] = newkey; //outside the loop: newkey copied into actv only now
}
void heap::bubbleDown(int actv, int newkey) {
  for(bool done=false; !done; ) {
    int smallest = newkey;
    if(Left(actv)!=-1 && Keys[Left(actv)]<smallest) smallest = Keys[Left(actv)];
    if(Right(actv)!=-1 && Keys[Right(actv)]<smallest) smallest = Keys[Right(actv)];
    if(smallest==newkey) done = true;
    else { // bubble the smaller child up (left child guaranteed to exist)
      Keys[actv] = smallest;
      actv = (smallest==Keys[Left(actv)]) ? Left(actv) : Right(actv);
    }
  }
  Keys[actv] = newkey; //outside the loop: newkey copied into actv only now
}
```

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Heap vs. BST

- BST organises the data more: Essentially sorts the data (since sorted order can be retrieved in O(n) time by in-order traversal). O(log n) search in a balanced BST.
- Heap maintains enough information to only access/remove the minimum. Provides no search facility (other than O(n) scanning).
  - What do heaps offer in return?
    - Uses only O(1) extra space beyond the data: Data is held in its original container (provided it supports random access)
      - If we just need a priority queue, a BST is a waste of space
    - O(n) time construction from a vector (coming up)
      - If we are not planning to dequeue all the elements, constructing a BST (in time $\Theta(n \log n)$) is a waste of time

Make-Heap

- Can make a heap, in O(n) time

```
struct heap {
  std::vector<int> Keys; // the actual data
  /* ... */
  void bubbleDown(int i, int newkey);

  void makeHeap(int root = 0) { // a recursive version
    if(root==-1) return;
    makeHeap(Left(root)); makeHeap(Right(root));
    bubbleDown(root, Keys[root]);
  }

  heap(std::vector<int> v) : Keys(v) { if(Keys.size()>0) makeHeap(); }
};
```

Make-Heap

- Can make a heap, in O(n) time
- Why O(n)?
  - Every node is visited once (like in a post-order traversal): O(n) for this
  - From every node, bubbleDown is called
    - $O\left(\sum_{\text{node}} \text{height(node)}\right)$ for this
    - At *depth* d, 2^d nodes. i.e, at *height* i, 2^{h-i} nodes (h = height of root)
    - $\sum_{\text{node}} \text{height(node)} \leq \sum_{i=0}^{h} i2^{h-i} = 2^h \sum_{i=0}^{h} \frac{i}{2^i} = O(2^h).$
    - Again, O(n)

```
void makeHeap(int root = 0) {
  if(root==-1) return;
  makeHeap(Left(root));
  makeHeap(Right(root));
  bubbleDown(root, Keys[root]);
}
```

Make-Heap: Non-Recursive

- Can we eliminate recursion, and use only O(1) additional space?
- Just need that before bubbleDown is called on a node, its two children have been processed (and are roots of valid heaps)
- Any child-before-parent order suffices (post-order traversal being one such)

```
void makeHeap(int root = 0) {
  if(root==-1) return;
  makeHeap(Left(root));
  makeHeap(Right(root));
  bubbleDown(root, Keys[root]);
}
```

- A simple child-before-parent order is the reverse of the "breadth-first" order in which the data is maintained
  - For index ≤ n/2, no left child (at n/2, 2*(n/2)+1 = n or n+1) or right child, and no need to call `bubbleDown`

```
void makeHeap() { //assumes Keys not empty
  for(int i = Keys.size()/2-1; i >= 0; --i)
    bubbleDown(i, Keys[i]);
}
```

Heap Sort

- An *in-place* sorting algorithm running in O(n log n) time
- Two modifications to our heap:
  - max-heap (bigger elements at the top of the heap)
  - We need to be able to set the heap to be any prefix of the array (specifically, `Left()` and `Right()` used in `bubbleDown()` take heap-size as input)
- In-place sorting:
  - Given an array `A` of values, in O(n) time and using O(1) extra space, make a heap out of them (using the non-recursive version of `makeHeap()`)
  - Repeat n-1 times:
    - `A[0]` has the maximum element. Swap `A[0]` and `A[n-1]` and set the size of the heap to be n-1 (so that `A[n-1]` is outside the heap).
    - Call `bubbleDown()` on the root with the value in `A[0]` as the new key

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 17

Sorting

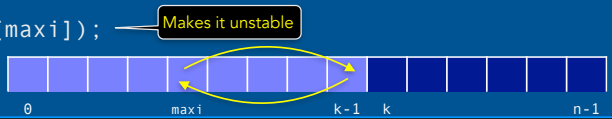
Sorting

- There are several sorting algorithms with asymptotic worst-case costs ranging from $O(n^2)$ to $O(n \log n)$
  - In-place or not
    - In-place: O(1) additional space. Input rearranged into sorted order
  - Stable or not
    - Stable: If there are equal valued keys, they appear in the output in the same order in which they are in the input
  - In practice, concerns beyond worst-case asymptotic behaviour
    - Average case costs. Performance on small inputs.

Selection Sort

- Recall the heap sort approach:
  - At each iteration identify the max, and move it to its final location
- Selection sort is the simplest implementation of this

```
void selection_sort(std::vector<int>& A) {
  bool sorted = false;   // optional flag to check for early exit
  for(int k = A.size(); k>1 && !sorted; --k) { // k items remain to be sorted
    sorted = true;
    for (int i=1, maxi=0; i<k; ++i) {//scan remaining items for max's index
      if(A[i] >= A[maxi]) maxi=i; else sorted = false;
    }
    if(maxi!=k-1) std::swap(A[k-1],A[maxi]);
  }
}
```



- $O(n^2)$ comparisons. Only O(n) assignments (swaps).
- Not a stable sort (equal items can change order when max item swapped out)
- Can exit early if the array becomes sorted sooner

Selection Sort → Heap Sort

- Instead of O(n) time scan to find max, use a *max-heap*
- O(log n) time to remove the max item; O(n log n) time overall

```
void heap_sort(std::vector<int>& A) {
  makeMaxHeap(A);  // a function that turns A into a max-heap in place
  for(int k = A.size(); k>1 ; --k) { // k items remain to be sorted
    int x = A[k-1]; A[k-1] = A[0];
    bubbleDown(A,k-1,0,x); // A[0] is bubbled down in heap A[0..k-1] with key x
  }
}
```

Sending A[k-1] to the root (and bubbling down) makes it unstable

specify how many items in A are part of the heap

location to bubbleDown and its new key value

- Exercise: In-place implementations of the functions makeMaxHeap and bubbleDown as required above
- In-place, and not stable (like selection sort)

Insertion Sort

- A slightly different idea yielding a stable sort
  - We will still maintain a suffix in sorted order
  - But the sorted items needn't be in their final locations
  - In each iteration absorb the last unsorted item (instead of max)
    - *Insert it into* the sorted part (sliding items less than it)

```
void insertion_sort(std::vector<int>& A) {
  for(int k = A.size(); k>0; --k) { // k items remain to be absorbed
    int i, x = A[k-1];              // will absorb x into the sorted part
    for (i=k; i<A.size() && A[i] < x; ++i) // scan for x's spot
      A[i-1]=A[i];                  // while sliding items less than x to the left
    A[i-1]=x; //outside the loop, i-1 is where x settles in.
  }
}
```

Important for stability: use <, not <=



- Stable, in-place sorting

Tree Sort

- Same as insertion sort, but instead of keeping the sorted part in (the suffix of) a vector, keep it in a balanced BST

```
void tree_sort(std::vector<int>& A) {
  balancedBST tree; // not in-place any more
  for(int k = 0; k < A.size(); ++k) { // k items remain to be absorbed
    tree.insert(A[k]);
  }
  tree.inorder(A); // overwrite the vector with tree's in-order sequence
}
```

For stability, requires that a duplicate key inserted later goes to the right of the existing key (and stays so while balancing)

- O(n log n) time, stable
- Needs O(n) additional space: not in-place

Attendance Break!

Login using CSE LDAP id at

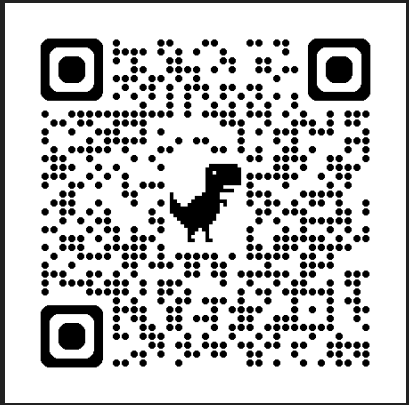
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Divide-and-Conquer

- Some *algorithms* use a Divide-and-Conquer strategy
  (a.k.a. Divide-Conquer-and-Combine)
  - A problem instance is divided into two or more smaller problems
  - The smaller instances are solved recursively
  - The results are then combined to get the result for the original instance
- Two examples: Merge Sort and Quicksort

```
resultType f(inputType x) {

  if (baseCase(x))
    return handleBaseCase(x);

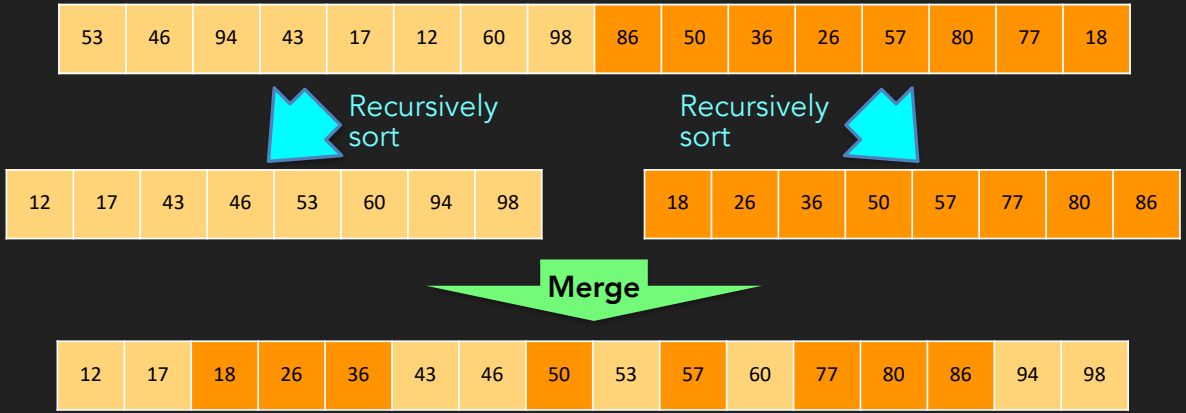
  inputType x1, x2;
  Divide(x,x1,x2);

  resultType y1, y2;
  y1 = f(x1); y2 = f(x2);

  return Combine(y1,y2);
}
```

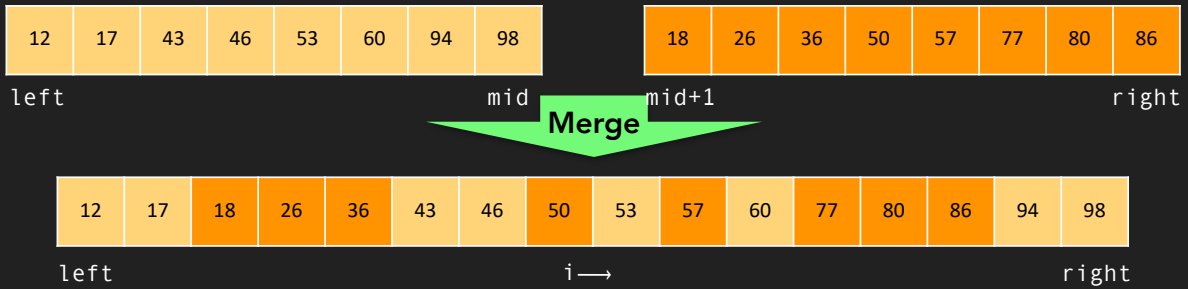
Merge Sort

- Split into two (almost) equal halves, and recursively sort each half
- Merge the two sorted arrays into a single sorted array



Merge Sort

```
// merge X[left..mid] and X[mid+1..right] into Y[left..right]
void merge(const vector<int>& X, int left, int mid, int right, vector<int>& Y) {
     int L = left, R = mid+1;    // L,R: next indices of left/right halves
     for(int i=left; i <= right; ++i) {
          if(L <= mid && (R > right || X[L] <= X[R])) Y[i] = X[L++];     // copy from left
          else Y[i] = X[R++];                                            // copy from right
     }
}
```



Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {

  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);  // outputs into aux
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out); // aux is input
}
```

Output will be in `out[left,...,right]`. A temporary vector `aux` is passed (to avoid allocating Θ(n) space in each recursive call)

Total extra space is O(n) + O(log n) = O(n)

```
resultType f(inputType x) {

  if (baseCase(x))
    return handleBaseCase(x);

  inputType x1, x2;
  Divide(x,x1,x2);

  resultType y1, y2;
  y1 = f(x1); y2 = f(x2);

  return Combine(y1,y2);
}
```

Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {

  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out);
}
```

- Time?
  - T(n) = 2T(n/2) + O(n)
  - O(n log n)

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 18

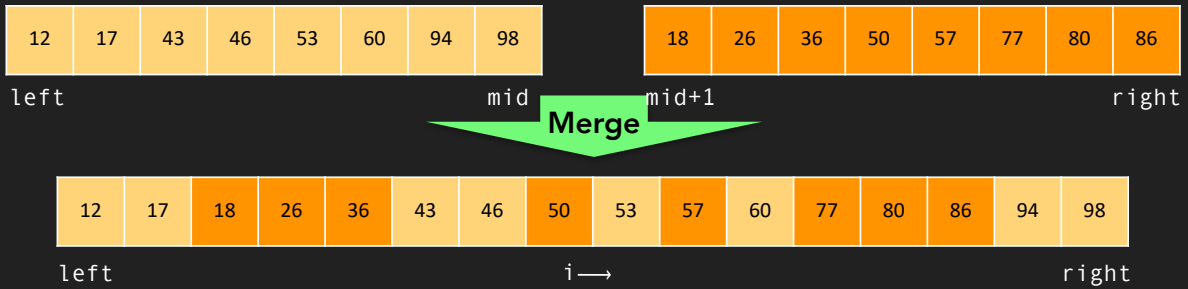
Sorting (ctd.)

Sorting

- So far:
  - Selection sort and Heapsort:
    - $O(n^2)$ vs $O(n \log n)$. Both in-place, not stable.
  - Insertion sort and Tree sort:
    - $O(n^2)$, in-place vs $O(n \log n)$, not in-place. Both stable.
  - Merge sort
    - $O(n \log n)$. Not in-place. Stable.
    - Let's recap

Merge Sort

```
// merge X[left..mid] and X[mid+1..right] into Y[left..right]
void merge(const vector<int>& X, int left, int mid, int right, vector<int>& Y) {
     int L = left, R = mid+1;    // L,R: next indices of left/right halves
     for(int i=left; i <= right; ++i) {
          if(L <= mid && (R > right || X[L] <= X[R])) Y[i] = X[L++];     // copy from left
          else Y[i] = X[R++];                                            // copy from right
     }
}
```



Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {

  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);  // outputs into aux
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out); // aux is input
}
```

Output will be in `out[left,...,right]`. A temporary vector `aux` is passed (to avoid allocating Θ(n) space in each recursive call)

Total extra space is O(n) + O(log n) = O(n)

- Time?
  - T(n) = 2T(n/2) + O(n)
  - O(n log n)

Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {

  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);  // outputs into aux
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out); // aux is input
}
```

- Non-recursive version?
  - Need to sort both child segments before sorting parent segment
  - Any child-before-parent order suffices
    - Recall: makeHeap
- Bottom-up ("reverse breadth-first" order) will work!

Merge Sort: Non-Recursive

```
void sort (std::vector<int>& in) {
  int N = in.size(); std::vector<int> aux(N);
  auto A = &in; auto B = &aux; // for conveniently switching between the two
  // imagine a nearly complete binary tree, where i-th node at height t
  // covers a block of T=2^t indices from i*T to (i+1)*T-1
  for(int t=0, T=1; T < N; T *= 2, ++t) { // merge pairs of T-long blocks
    for(int left=0; left < N; left += 2*T) {
      int mid = std::min(N-1,left+T-1), right = std::min(N-1,left+2*T-1);
      merge(*A,*B,left,mid,right); // if empty right half, just copies A to B
    }
   std::swap(A,B); // merge's output (B) should become its input (A) next time
  }
  // sorted array is in A
  if(A != &in) in = aux; // if necessary, copy aux to in
}
```

O(1) option available:
`in = std::move(aux);`

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Quicksort

- Another divide-and-conquer sorting algorithm
  - Almost in-place. Not stable.
- Divide into two lists, S_1 = [items < pivot] and S_2 = [items > pivot]
- Combine: [quicksort(S_1) [items == pivot] quicksort(S_2)]
- How is pivot picked? Various approaches
  - Pick the first element: Worst case $O(n^2)$
  - Pick a random element: Worst case O(n log n) with high probability
  - Pick the "median of medians": Worst case O(n log n)
  - Heuristic choices: Worst case $O(n^2)$, but often better than that
- Hybrid models: switch to another sorting if taking too long

E.g., when the data is sorted or reverse-sorted

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 19

Sorting (ctd.)

Sorting

- So far:
  - Selection sort and Heapsort:
    - $O(n^2)$ vs $O(n \log n)$. Both in-place, not stable.
  - Insertion sort and Tree sort:
    - $O(n^2)$, in-place vs $O(n \log n)$, not in-place. Both stable.
  - Merge sort
    - $O(n \log n)$. Not in-place. Stable.
- Today: Quicksort

Quicksort

- Another divide-and-conquer sorting algorithm
  - Almost in-place. Not stable.
- Divide into two lists, S_1 = [items < pivot] and S_2 = [items > pivot]
- Combine: [quicksort(S_1) [items == pivot] quicksort(S_2)]
- How is pivot picked? Various approaches
  - Pick the first element: Worst case $O(n^2)$
  - Pick a random element: Worst case O(n log n) with high probability
  - Pick the "median of medians": Worst case O(n log n)
  - Heuristic choices: Worst case $O(n^2)$, but often better than that
- Hybrid models: switch to another sorting if taking too long

E.g., when the data is sorted or reverse-sorted

With high probability, close to the ideal situation where T(n) = 2T(n/2) + O(n)

Deterministically, close enough to the ideal

Quicksort: Using Aux Space

```
void qsort (std::vector<int>& A) {
  int N = A.size();
  if(N<=1) return;
  int pivot = A[0]; // may use alternate logic
  std::vector<int> L, E, G; // Less, Equal and Greater
  for(int i=0; i<N; ++i) { // scan and populate the 3 arrays
    if (A[i] < pivot) L.push_back(A[i]);
    else if (A[i] > pivot) G.push_back(A[i]);
    else E.push_back(A[i]);
  }
  qsort(L); qsort(G);
  // copy L, E, G to A
  std::copy(L.begin(), L.end(), A.begin());
  std::copy(E.begin(), E.end(), A.begin()+L.size());
  std::copy(G.begin(), G.end(), A.end()-G.size());
}
```

Quicksort: In-Place Pivoting

```
void qsort (std::vector<int>& A, int left, int right) {
  if(left>=right) return;
  int pivot = A[left]; // may use alternate (in-place) logic
  int L=left, G=right;
  for(int i=left; i<=G; ) {
    if (A[i] < pivot) std::swap(A[L++],A[i++]);
    else if (A[i] > pivot) std::swap(A[G--],A[i]);
    else ++i;
  }
  qsort(A,left,L-1); qsort(A,G+1,right);
}
```



- Almost in-place: The data itself stays in the original container
  - Recursion uses a stack to store left and right (can't predict them)
  - Recursion depth can be made O(log n) by the choice of pivot

Merge Sort and Quicksort

- Both are practically important algorithms: Quicksort used in `std::sort` and Merge Sort in `std::stable_sort`
- In practice, combined with several optimisations
  - The base-case is not one element, but a small enough array . Then it switches to, say, insertion sort (in-place and stable).
  - `std::sort`, uses a more complex (deterministic) choice of pivot, and if the recursion depth goes well beyond log n, switches to heapsort
  - `std::stable_sort`, if not enough memory for standard merge sort, falls back to a more complex, but in-place, $O(n \log^2 n)$-time merge algorithm
  - Heuristics to exploit pre-sorted subsequences and patterns

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



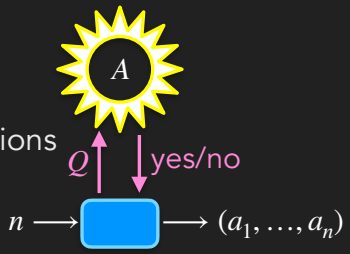
Lower Bound: $\Omega(n \log n)$ Comparisons

- General purpose sorting algorithms are "comparison-based"
  - e.g., `std::sort` and `std::stable_sort` work, by default, on any data-type with `operator<` (and supports moving)
- **Claim:** Any comparison-based sorting algorithm will need to make, in the worst-case, at least $cn \log n$ comparisons, where c is a constant

  $(\exists n_0, c > 0 \; \forall n > n_0 \;\; N_{\text{comp}}(n) \geq cn \log n)$
  - $\Omega(n \log n)$ comparisons
- There are some non-comparison-based sorting algorithms, exploiting additional guarantees/structure on the data
  - E.g., bucket sort and radix sort (can be faster for sorting numbers in certain ranges)

Lower Bound: $\Omega(n \log n)$ Comparisons

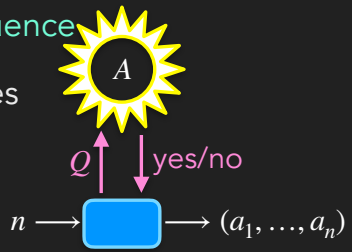
- Formalising a comparison-based sorting algorithm:
  - Given n and an "oracle" that answers boolean questions about an n-item vector A (e.g. $A[i] < A[j]$?)
  - To output indices $(a_1, \ldots, a_n)$ s.t. $A[a_1] \leq \ldots \leq A[a_n]$



- Fix n and a correct sorting algorithm. How many queries should it ask the oracle?
- Key observation: The final output is completely determined by the response sequence
  - Can reproduce the output by running the algorithm and feeding it responses from this sequence (without needing to know A)

Lower Bound: $\Omega(n \log n)$ Comparisons

- Output completely determined by the response sequence
- Suppose the algorithm makes at most k oracle queries
  - W.l.o.g, assume it always makes exactly k queries
    - Because the algorithm can be modified so that it makes additional dummy queries at the end if necessary
  - Number of possible response sequences from the oracle is at most 2^k
  - So number of distinct outputs is $\leq 2^k$
- But number of distinct outputs by a fully correct algorithm should be $n!$
- $n! \leq 2^k \implies k \geq \log_2(n!) \geq \log_2((n/2)^{n/2})$



Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 20

Sorting Networks

Sorting

- So far:
  - Selection sort and Heap Sort: $O(n^2)$ vs $O(n \log n)$. Both in-place, not stable. Heap Sort is a fall back for `std::sort`
  - Insertion sort and Tree Sort: $O(n^2)$, in-place vs $O(n \log n)$, not in-place. Both stable. Insertion sort used by `std::sort` and `std::stable_sort` to handle the base-case.
  - Merge Sort: $O(n \log n)$. Not in-place. Stable. Main component of `std::stable_sort`
  - Quicksort: $O(n \log n)$ on average, or using certain pivot selection schemes (not). Main component of `std::sort`

Bubble Sort

- Another simple $O(n^2)$-time in-place sorting algorithm (cf. selection sort and insertion sort)
  - Moves largest items to their final location, one by one (like selection sort)
  - Relative order of two items flipped only if required by sorting order, making it stable (unlike selection sort)

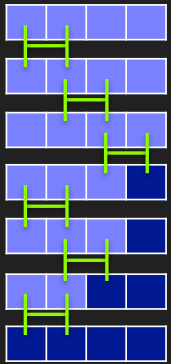
```
void sortpair(int& a, int& b) {
  if (a>b) std::swap(a,b)
}
void bubblesort (std::vector<int>& A) {
  for(int k=A.size(); k > 1; --k)
    for(int i=0; i<k-1; ++i)
      sortpair(A[i],A[i+1]);
}
```



Exercise:
1. Exit early, when already sorted.
2. Replace calls to sortpair with shifting to save on number of assignments.

Bubble Sort

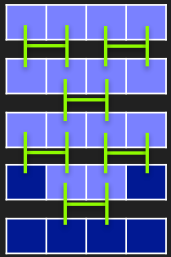
- The order of comparisons/moves doesn't depend on the data!
- Can build a "circuit" using "sortpair gates" (comparators) for sorting
  - Such a circuit is called a sorting network
- Complexity measures: Number of comparators (hardware cost), number of levels (time taken)



```
void sortpair(int& a, int& b) {
  if (a>b) std::swap(a,b)
}
void bubblesort (std::vector<int>& A) {
  for(int k=A.size(); k > 0; --k)
    for(int i=0; i<k-1; ++i)
      sortpair(A[i],A[i+1]);
}
```

Sorting Networks

- Can improve on the sorting network from bubble sort
  - Instead of $O(n^2)$ time, $O(n)$ levels enough
    - With $O(n)$ comparators at each level
    - Odd-Even Transposition Network
- How do we know this works?
  - Two-step argument:
    - **0-1 Principle:** Enough to analyse inputs in $\{0,1\}^n$
    - No error after n steps in a $\{0,1\}^n$ input



Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



0-1 Principle

- If a sorting network correctly sorts all 0-1 sequences, then it correctly sorts all inputs
- Contrapositive: If a sorting network is wrong on some input, then it must be wrong on some 0-1 sequence
- Proof: Suppose on input $(x_1,\ldots,x_n)$ it is wrong. Then some inputs a and b such that $a < b$ that ended up in the wrong order (b before a)
  - Consider input z such that $z_i = T(x_i)$ where $T(x) = \begin{cases} 0 \text{ if } x \leq a \\ 1 \text{ otherwise.} \end{cases}$
  - Inductively, at any level, results while sorting x and z, are related by $z^{(t)} = T(x^{(t)})$: because $\min(T(u),T(v)) = T(\min(u,v))$ and $\max(T(u),T(v))=T(\max(u,v))$.
  - At the final level, T(b)=1 before T(a)=0

n Steps for Odd-Even Transposition Network

- Two-step argument to prove a sorting network is correct:
  - **0-1 Principle:** For any sorting network, enough to analyse inputs in $\{0,1\}^n$
  - The given network correctly sorts every input in $\{0,1\}^n$
- Claim: The odd-even transposition network on n inputs, with n levels, correctly sorts every input in $\{0,1\}^n$
  - The rightmost 1 may or may not move in the first step (depending on if it is compared to the left or to the right); after that it will start *flowing*: i.e., move to the right at every step until it reaches its final position, n-1
  - Inductively: The kth-from-right 1 (whose final position is n-k) will start flowing after at most k steps, because it will not be blocked by a 1 to its right that is already flowing (but could stall once before starting to chase it)
  - All 1s reach final positions in at most n steps: $\leq$ k steps before flowing, and $\leq$ n-k steps in the flow

Sorting Networks

- Odd-even transposition network takes O(n) time using O(n) processors in parallel
  - As a circuit, needs $O(n^2)$ comparators
  - Can do much better! (cf. $\Omega(n \log n)$ lower bound)
- Batcher's Bitonic sorting & Batcher's Odd-Even Mergesort
  - $O(\log^2 n)$ time, using O(n) processors in parallel (total $O(n \log^2 n)$ comparators).
  - Practical (small constants hidden by O). Used in GPUs and circuits, for moderately large n.
- In principle, O(log n) time using O(n) processors enough!
  - But the hidden constants possible for the best known algorithms are in 1000s (where as $\log_2 n$ is *well under* 100 for all practical n

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 21

Non-Comparison-Based Sorting

Sorting

- So far:
  - Several comparison-based sorting algorithms
    - Merge sort, quicksort, heap sort, tree sort : $O(n \log n)$ time
    - Insertion sort, selection sort, bubble sort : $O(n^2)$ time
    - Sorting networks
      - Odd-even transposition: $O(n)$ depth with $O(n^2)$ comparators
      - $O(\log n)$ depth possible, $O(\log^2 n)$ depth practical
  - Lower Bound:
    - Any comparison-based sorting requires $\Omega(n \log n)$ comparisons
- Today: Non-comparison-based sorting: Counting sort, bucket sort, radix sort

(Basic) Counting Sort

- When the domain of keys to sort is small -- say, [0,ulim)

```
void basic_counting_sort (std::vector<unsigned int>& A, unsigned int ulim) {
  std::vector<int> counts(ulim,0); // one index per key, initialised to 0
  for(auto x : A)                  // compute count of each key in O(n) time
    ++counts[x];
  for(unsigned int x=0, index=0; x < ulim; x++) // for each key x in domain
    for(; counts[x] > 0; counts[x]--) // write counts[x] copies of x into A
      A[index++] = x;                 // keeping track of next index to use
}
```

- Extends to values in a small range `[llim,ulim)` (Exercise)
- Let `k = ulim - llim` be the size of the domain of the keys
- O(n+k) time. For small k, beats the Ω(n log n) lower bound for comparison sorts!
- O(k) auxiliary space

Collect Instead of Count

- When the items are structured, but with integer keys from a small range
  - Instead of just keeping the count of each key, we need to collect all the items with that key

```
struct item { unsigned int key; /* more data */ };
void collection_sort (std::vector<item>& A, unsigned int ulim) {
  std::vector<std::queue<item>> collection(ulim);  //per key a queue
  for(auto& x : A)                 //for each item in A (without making a copy)
    collection[x.key].push(std::move(x));          //move it into collection
  for(unsigned int x=0, index=0; x < ulim; x++)    //for each key x
    for(; !collection[x].empty(); collection[x].pop()) //for each item for x
      A[index++] = std::move(collection[x].front());   //move it into A
}
```

  - Stable (by collecting in a queue for each key)

Counting Sort : General Version

- Use counts to directly find the index in the output (no queues)

```
struct item { unsigned int key; /* more data */ };
void counting_sort (std::vector<item>& A, unsigned int ulim) {
  std::vector<unsigned int> offset(ulim,0);     // used for counting
  std::vector<item> output(A.size());
  for(auto& x : A) ++offset[x.key]; // after this, offset has counts
  for(unsigned int i=0, sum = 0; i < ulim; i++) {
    unsigned int tmp = offset[i];
    offset[i] = sum; sum += tmp;     // offset[i] has sum of previous counts
  }
  for(auto& x : A) // output[ offset[i] ] is where next item with key i goes
    output [ offset[x.key]++ ] = std::move(x); // move items from A to output
  A = std::move(output);             // finally move output vector to A
}
```

- O(n+k) time and auxiliary space. Stable.

Bucket Sort

- Can we generalise counting sort to large key domains (e.g., all of `int`, or `float`)?
  - By *bucketing*! `bucketof(key)` sends key to a small domain of "buckets"
  - Sort items into their respective buckets
    - Using counting sort: with `bucketof(key)` as the key
  - Within each bucket, sort (using any sorting algorithm)

2.31	9.82	7.39	4.31	8.25	9.01	6.77	8.10	4.99	0.22	9.32	6.38	1.13	4.00	3.75	1.67	8.27	7.08

counting sort with `bucketof(key)`

0.22	1.13 1.67	2.31	3.75	4.31 4.99 4.00		6.77 6.38	7.39 7.08	8.25 8.10 8.27	9.82 9.01 9.32

sort each bucket

0.22	1.13 1.67	2.31	3.75	4.00 4.31 4.99		6.38 6.77	7.08 7.39	8.10 8.25 8.27	9.01 9.32 9.82

0.22	1.13	1.67	2.31	3.75	4.00	4.31	4.99	6.38	6.77	7.08	7.39	8.10	8.25	8.27	9.01	9.32	9.82

Bucket Sort

- If we use k buckets, takes O(n+k) auxiliary space
- Total time for sorting the buckets (using a comparison sort)?
  - If n = O(k) and each bucket has O(1) items, total time is O(n)
    - E.g., when the keys are uniformly spread throughout its domain
  - But can be as bad as $\Omega(n \log n)$ if one bucket has $\Theta(n)$ items
- Parallelisable
  - Can collect items into buckets in parallel
  - Can sort each bucket in parallel
  - Can copy back from buckets into a vector in parallel (after offsets computed)

Bucket Sort

- What if Θ(n) items fall into one bucket?
  - If using a comparison based sorting algorithm for sorting within buckets, takes Θ(n log n)
- Can we still beat comparison based sorting?
  - Can use application-specific heuristics for defining buckets to avoid crowded buckets
  - Or, recursively, use bucket sort?
    - When a bucket has "too many" items, split it into smaller buckets
    - If the keys are "granular" (e.g., integers), not too many levels of recursion
    - Leads to another sorting algorithm! MSD Radix Sort

Most-Significant-Digit Radix Sort

- Consider d-digit numbers
  - First collect by MSD



  - Each bucket with more than one item is sorted using the remaining digits

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 22

Radix Sort: MSD vs. LSD

Sorting

- Last time:
  - Counting sort: When the key-space is small, calculate the offsets for items with each key, and move them to the correct position
  - Bucket sort: Counting sort, applied to bucketed keys, followed by sorting each bucket
  - MSD Radix Sort: Bucket sort, with large buckets sorted recursively
- Today:
  - MSD Radix Sort (continued)
  - Least-Significant Digit (LSD) Radix Sort
  - MSD vs LSD Radix sort

Most-Significant-Digit Radix Sort

- Consider D-digit numbers
  - First collect by MSD



  - Each bucket with more than one item is sorted recursively using the remaining digits

Most-Significant-Digit Radix Sort

- What MSD Radix sort implements is *lexicographic sorting* (dictionary order)
  - E.g., `cab < car < dad < dog`
  - Strings needn't be the same length: `dog < dogma` (imagine an end-of-string character lesser than all other characters)
- For integers, to ensure lexicographic ordering is the same as numerical ordering, need 0-padding so that all numbers have the same number of digits
  - E.g., 22 < 113 numerically but not lexicographically; after 0-padding, lexicographically also, 022 < 113
  - "Digit" can be in any convenient base, e.g., base 256 allows each byte to be a single digit

Most-Significant-Digit Radix Sort

- There are B^D D-digit numbers (or D-long strings) in base B
  - This space is exponentially large in D, making counting sort impractical even for modest D
  - Also, comparison sort is expensive for large D, since a single comparison of two keys requires inspecting up to D digits, one by one
- MSD Radix sort costs:
  - Worst-case time and space: $O(D(n+B))$
    - Exercise: Prove by induction on D. Base case, D=1, is counting sort.

LSD Radix Sort

- What if we first sort the numbers by their LSD?

4012
1091
30
785
7703
4388
9452
92
1179
6905
3498

1000	100	10	1
4	0	1	2
1	0	9	1
0	0	3	0
0	7	8	5
7	7	0	3
4	3	8	8
9	4	5	2
0	0	9	2
1	1	7	9
6	9	0	5
3	4	9	8

1000	100	10	1
0	0	3	0
1	0	9	1
4	0	1	2
9	4	5	2
0	0	9	2
7	7	0	3
0	7	8	5
6	9	0	5
4	3	8	8
3	4	9	8
1	1	7	9

1000	100	10	1
7	7	0	3
6	9	0	5
4	0	1	2
0	0	3	0
9	4	5	2
1	1	7	9
0	7	8	5
4	3	8	8
1	0	9	1
0	0	9	2
3	4	9	8

1000	100	10	1
4	0	1	2
0	0	3	0
1	0	9	1
0	0	9	2
1	1	7	9
4	3	8	8
9	4	5	2
3	4	9	8
7	7	0	3
0	7	8	5
6	9	0	5

1000	100	10	1
0	0	3	0
0	0	9	2
0	7	8	5
1	0	9	1
1	1	7	9
3	4	9	8
4	0	1	2
4	3	8	8
6	9	0	5
7	7	0	3
9	4	5	2

30
92
785
1091
1179
3498
4012
4388
6905
7703
9452

LSD Radix Sort

- LSD Radix sort: Sorting all the numbers by their digits using counting sort, one digit at a time, starting from LSD
- Why does LSD work? Proof by induction:
  - Base case is the 1 digit case.
  - Enough to argue that any two numbers are correctly sorted relative to each other
    - If they differ in MSD, the last pass sorts them correctly
    - If they have the same MSD, then they were correctly sorted before the last pass (by induction) *and* *the last pass does not change their relative order* (we crucially need each pass to be a stable sort)

LSD Radix Sort

- Simply call counting sort in a loop, one digit at a time as the key
  - Here each item is a D-sized vector of B-ary digits

```
void LSD_Radix(std::vector<std::vector<int>>& A, int D, int B) {
  std::vector<std::vector<int>> aux(A.size());
  for(int d=0; d<D; ++d) { // counting sort using d-th digit (d=0 is LSD)
    std::vector<int> offset(B,0);
    for(auto& x : A) ++offset[x[d]]; // after this, offset has counts
    for(unsigned int i=0, sum = 0; i < B; i++) {
      unsigned int tmp = offset[i];
      offset[i] = sum; sum += tmp;     // offset[i] has sum of previous counts
    }
    for(auto& x : A) // aux[ offset[i] ] is where next item with key i goes
      aux [ offset[x[d]]++ ] = std::move(x); // move items from A to aux
    A.swap(aux); // finally move aux vector to A, keeping aux's size same
  }
}
```

MSD vs. LSD Radix Sort

- MSD version can eliminate short strings (or strings that fall into small buckets) early on. LSD version goes over every string in all D iterations.
  - MSD radix sort better suited for sorting variable length strings, or long strings that are lexicographically "far apart"
- MSD version requires more book-keeping, as sorting is applied to each bucket separately. LSD version is simpler and more cache-friendly.
  - LSD radix sort better suited for sorting fixed-length "strings" (e.g., 32-bit integers with exactly D = 4 digits, in base B = 2^8 = 256)
- Both are highly parallelizable, but the LSD version allows all processors working equally and in-sync
  - LSD radix sort (with optimisations) used for sorting integers in GPUs

Summary

- Counting sort
  - Useful when the key domain size k is small. O(n+k) time and auxiliary space
- Bucket sort
  - If data is "well-spread" over a few buckets into which keys can be partitioned
  - On well-spread data, using k buckets, O(n+k) time and auxiliary space. But worst-case time is O(n log n + k) (assuming O(1)-time comparisons).
- MSD Radix Sort
  - Using bucket sort recursively, for lexicographically sorting strings. Worst-case time and auxiliary space: O(D(n+B)) for D-long strings over an alphabet of size B. (Faster, if common prefix of every pair is short.)
- LSD Radix Sort
  - Using counting sort iteratively. O(D(n+B)) time and auxiliary space.
  - Parallel optimised version used for sorting numbers in modern GPUs

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 23

Radix Trees

A Data Structure for Storing Strings

- How can we store a set of strings and search in it efficiently?
  - Option we already have: Use std::set<std::string>
    - Comparing a pair of strings takes time proportional to the length of the (shorter) string
    - Searching, inserting, deleting in a set of n strings of length D takes O(D log n) time
  - Option : Use a radix tree
    - Searching, inserting, deleting in a set of n strings of length D takes O(D) time!

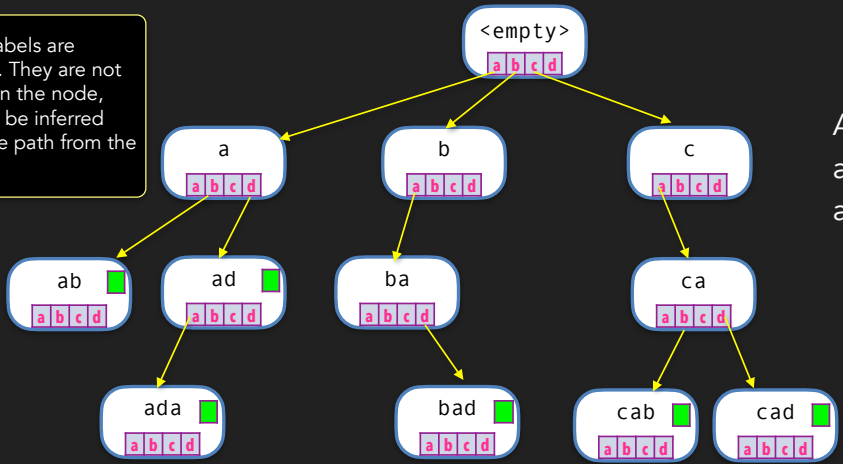
Trie

- A Radix Tree is an optimisation of a simpler data structure called a Trie
- For storing strings over an alphabet of size B:
  - A full B-ary tree: Each internal node has B children, each child labelled by a distinct character from the alphabet
  - The labels on the path from the root to a node is a string that is associated with that node
  - For each prefix of a string that is stored, there is a unique internal node. Leaves correspond to minimal strings which are not such prefixes.
  - Each node has a boolean flag `full_entry` indicating whether the corresponding string is stored, or if it is merely a prefix (alternately/ equivalently, include a special end of string character in the alphabet)

Trie

- Strings: { ab, ad ada, bad, cab, cad }. Alphabet: (a, b, c, d)

Node labels are implicit. They are not stored in the node, but can be inferred from the path from the root.

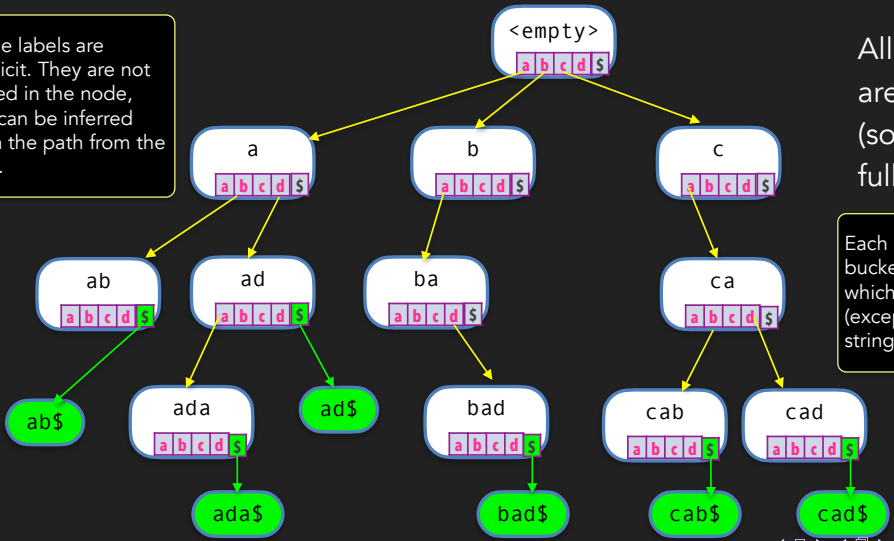


All internal nodes are prefixes. Some are also full entries.

Trie

- Alternate view: If `full_entry` set, has a leaf child that is a full entry

Node labels are implicit. They are not stored in the node, but can be inferred from the path from the root.



All internal nodes are prefixes. Only (some) leaves are full entries.

Each node corresponds to a bucket in MSD Radix Sort which is recursively sorted (except nodes with only one string left)

Trie

- Searching in a trie: Simply follow the unique path determined by the characters of the string ("MSD" first)
  - If the path exists, and further when the string ends, the full_entry flag is set (i.e., child for $ also exists), then the string is present
- Time: O(D) for searching D-long strings
  - Independent of how many strings are stored in the trie!
  - cf. Using std::set<std::string> requires Θ(D log n) time
- Can also search for all strings with a given prefix query, or the strings that share the Longest Common Prefix with a given query string
  - Important in many applications: Internet Packet routing tables, autocomplete from a dictionary (e.g., in IDEs, browser URLs, tab completion in shells, ...)

Trie

- Trie also supports easy insertion and deletion
  - Insertion: Search to find the longest prefix, and insert a chain of nodes from there
  - Deletion: Find the longest prefix which has at least two children, and delete all the nodes in the chain from there till the entry being deleted
  - Handle the full_entry flag correctly when inserting/deleting the leaf in these chains
    - Inserting the leaf node: just set the full_entry flag in the parent
    - Deleting the leaf node: just reset the full_entry flag in the parent

Attendance Break!

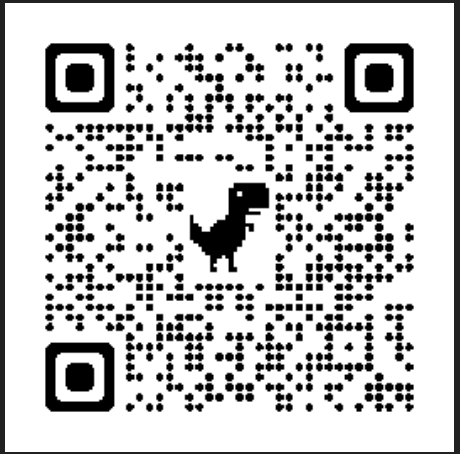
Login using CSE LDAP id at
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

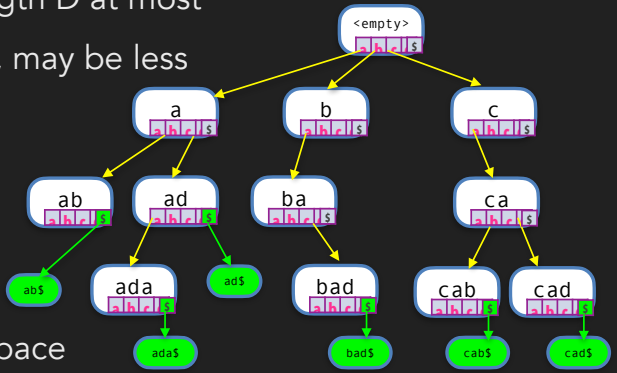
The selected students should come to the front

If you are not in the classroom, a report will be sent to DDAC!



Space in a Trie

- Each internal node takes O(B) space for the B child pointers
- O(nD) nodes for n strings of length D at most
  - By sharing common prefixes, may be less
- Total space: O(nDB)
- Compare to O(nD) for std::set<std::string>
- Can we do better?
  - Yes, two avenues of saving space
    - Don't use Θ(B) space for all nodes
    - Reduce the total number of nodes

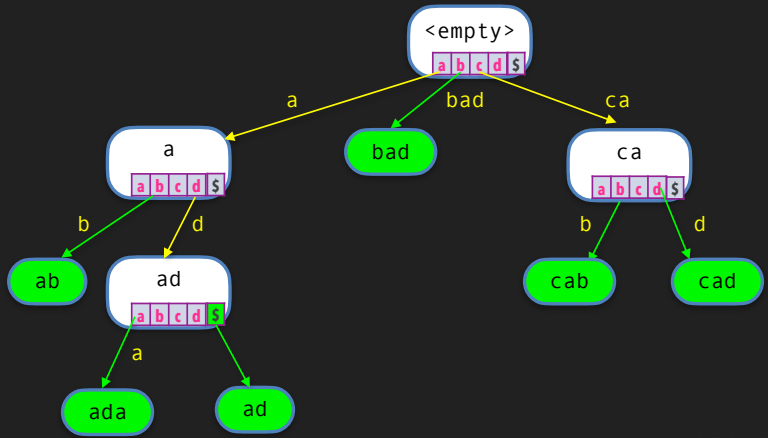


Reducing Space per Node

- Instead of O(B) space, we can use just O(#children) space
  - Use a map (e.g., BST as in std::map or Hash Tables as in std::unordered_map) to store pointer to each child, keyed using the alphabet of size B.
    - std::map (as opposed to std::unordered_map) changes lookup time from O(1) to O(log B), but retains ability to search for smallest string with a prefix, etc.
    - Can adaptively switch the representation, as the number of children changes: when the number of children is small, just store a sorted vector
- How many edges?
  - O(nD) nodes with O(B) children. O(nDB). No gain?
  - Count by the other end of the edge: O(nD) edges!

Reducing the Number of Nodes: Radix Tree

- Path compression: Each edge has a string of symbols, and there is no internal node with a single child (except possibly the root, when n=1)



Reducing the Number of Nodes: Radix Tree

- Path compression: Each edge has a string of symbols, and there is no internal node with a single child (except possibly the root, when n=1)
- How many nodes?
  - Recall: if every internal node has exactly 2 children, n leaves ⇒ n-1 internal nodes (proof by induction)
    - Trimming two leaves with the same parent reduces the number of leaves by 1 and number of internal nodes by 1
  - If every internal node has ≥ 2 children, n leaves ⇒ ≤ n-1 internal nodes
    - Trimming k ≥ 2 leaves reduces the number of leaves by k-1 and number of internal nodes by 1

Reducing the Number of Nodes: Radix Tree

- Path compression: Each edge has a string of symbols, and there is no internal node with a single child (except possibly the root, when n=1)
- How many nodes?
  - n leaves $\Rightarrow \leq$ n-1 internal nodes (except if n=1)
- Total number of nodes is now independent of the string length
- Edge-labels (stored in a map at the parent-node) take a total of O(nD) space
- O(n) space for tree, O(nD) space for edge-labels $\Rightarrow$ O(nD) total space
- O(D) or O(D log B) search time (depending on how the edge map is implemented)

Radix Tree

- Searching in a radix tree requires checking if edge label matches the remainder of the query string
- Inserting a string into a radix tree could require splitting an edge into two (where the mismatch with the inserted string occurs), and inserting a node in between (which then gets a second child corresponding to the inserted string)
- Deleting a string may cause a (non-root) node to have only one child left. Then the node is eliminated and its incoming edge and outgoing edge are merged.

Substring-Matching Using a Suffix Tree

- Goal: To search for query substrings in a long text
- A Radix Tree allows searching for prefixes of a set of strings
- Any substring of T is a prefix of some suffix of T!
- If we store all the suffixes of the text in a Radix Tree, we can query any substring!
  - Suffix Tree of T: A Radix Tree of all the suffixes of T
- There is an O(D) time algorithm ("Ukkonen's algorithm") for constructing a suffix tree for a D-long text
  - Subsequently, searching for a substring of length d takes O(d) time

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 24

Hash Tables

Motivation

- Suppose we are storing n distinct keys from a domain of size N
- Store them in an array A of size N
  - Store item with key i at A[i] (initially A[i] is marked "empty")
  - O(1) time for search/insert/delete!
- This is called Direct Addressing
- But $\Theta(N)$ space: realistic only when N is "small." Not feasible when the keys are say, integers ($N=2^{64}$ typically), or byte-strings of length 5 ($N=2^{40}$)
- Question: Can we match the speed of direct addressing using a reasonable amount of space?

Motivation

- Now, suppose we are storing n keys from a very large domain K
- Without direct addressing, we just need an array A of size $m \geq n$ to hold them
  - But searching would be slow, as a key could be sitting anywhere in the array
  - What if we could *calculate* its index as H(key) directly, using a "hash function" $H: K \rightarrow \{0,\ldots,m-1\}$?
  - When searching for key, we just compute H(key) and check if A[H(key)] has key stored there.
- Too good to be true! If $|K| >> m$, any function $H: K \rightarrow \{0,..,m-1\}$ will have to map many keys to the same index (collisions)
  - Hope: If we keep m slightly larger than n, then among the n items we get, there will only be a few collisions in each index. And, we can handle a few collisions.

Hash Tables

- Ingredients to make hash tables work
  - Good hash functions: how to ensure that there will only be a few collisions in each index (with high probability)
  - Handling increasing load: Grow the hash table as more keys inserted
  - Handling collisions: Two different paradigms
- Coming up today:
  - What are good hash functions

Hash Functions

- Central idea: If the function H is "random", and the load factor n/m is bounded by a constant (say < 1), then with high probability, there will be at most a few collisions in almost all indices
- What does with high probability mean?
  - There is a "universe of functions" $\mathcal{H}$ such that for any set of n keys chosen to be stored, *most* $H \in \mathcal{H}$ (say, more than 1-ε fraction) lead to low collisions
  - So if H is picked randomly from this universe, the probability that we pick a bad H (i.e., not with low collisions) is at most ε
    - However, for each H, there exist many key-sets with high collisions (in fact, if $|K| > mn$, there will be a key-set that is entirely mapped to a single index): No single H is universally good for all key-sets

Hash Functions

- What is a good universe of hash functions?
  - One that enforces "uniform hashing": each of the n keys is mapped to a uniformly random index, independent of the others
    - Then the expected number of items in any index is n/m, and further the probability that it is very far from expectation is very low
  - E.g., Universe of all functions from the domain of keys K to {0,...,m-1}
    - Picking a random H from this universe is exactly the same as uniform hashing (without limiting to n keys)
    - But no succinct description of a random function in this universe
  - We can "mimic" the universe of all functions using a smaller "pseudorandom" universe with efficiently representable functions

Hash Functions

- Pseudorandom: Has a similar effect as truly random.
  - Empirical (or Heuristic): Designed to work like random on a benchmark of inputs. No guarantees against adversarial inputs. Very fast.
  - Combinatorial: Provably as good as random against one-shot adversarial inputs. No guarantees against adaptive adversarial inputs.
  - Cryptographic: Strong guarantees. Only slightly slower, if relying on "lightweight-cryptography". Default in Python/Rust.
- Applications (e.g., those running on a web-server) which store untrusted inputs in hash tables are vulnerable to Denial of Service attacks, unless cryptographic hashes used

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 25

Hash Tables (ctd.)

Hash Tables

- Recall Hash Table template:
- Initialise an array A of size m, and a hash function H drawn from a family $\mathscr{H}$
- To search for the location of a key, compute an index H(key) $\in \{0,\ldots,m-1\}$
  - H(key) should appear to be uniformly random, independent of the other keys
  - Load factor n/m determines the expected number of collisions at each index. If it is too large, rehash: increase m, pick H from a corresponding $\mathscr{H}$, and reinsert all elements into a new table of size m
  - Need to handle collisions
- Today:
  - Rehashing, and handling collisions

Two-Stage Hash Functions

- Recall that to limit collisions, even using an ideal hash function, the load factor n/m should be upper bounded by a small constant (say 1)
- So, as n grows, m will have to grow too
- When m changes, the hash function will have to change too!
- This motivates a *two-stage hash function:* $H(\text{key}) = g_m(h(\text{key}))$, where $h: K \to R$ for a fixed range R (usually 64-bit integers) and $g_m: R \to \{0,..,m-1\}$
  - h is drawn from a good hash function family that doesn't depend on m
  - g_m is a (fixed, fast) *regular* function: i.e., sends a uniformly random input to a uniformly random output. Compresses $r \in R$ to an index.
- In some cases (e.g., C++ STL defaults) h is not good (i.e., not approximating uniform), but only collision resistant. Then g_m should ideally be drawn from a good hash function family (not so in STL!)

Two-Stage Hash Functions

- In std::unordered_set and std::unordered_map, while instantiating the template, one can/must provide a hash function h
  - Defaults are available for basic types, pointers, std::string etc.
  - But not for vectors or structs/classes
- The function g_m is fixed by the library for each m (e.g., modulo m)
  - In some implementations (with weak default h), m is chosen to be a prime number, to prevent g_m from creating collisions on regularly spaced inputs
- When m changes, the hash function $H = g_m \circ h$ needs to be redefined, and recomputed for every item in the hash table
  - h(key) will be *cached* along with the item, so that rehashing can avoid recomputing h, and only compute g_m

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Handling Collisions

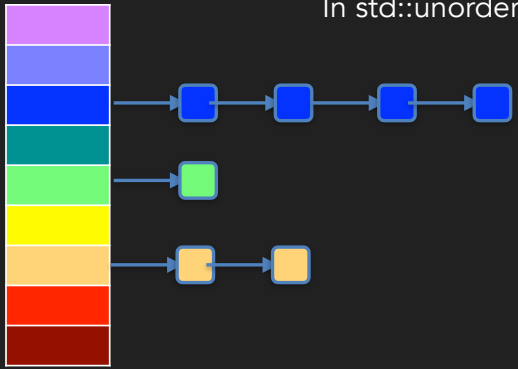
- What about the few collisions that do occur, even when we are lucky?
  - Simplest paradigm: At each index, allow multiple items to be held
  - Alternate paradigm: When collision, find another spot (next lecture)
- In std::unordered_map, items falling into the same index are held in a linked-list
  - Other containers would work too
    - E.g., in Java, switch to balanced BST when large number of items in an index (possibly because adversarial inputs broke uniform hashing assumption)
  - If the number of items in this container is O(1), looking through them will cost only O(1) time, and so total look up time is still O(1)
    - Assuming uniform hashing and bounded load factor, indeed O(1) expected list size, and further, with overwhelming probability, only at a few indices the number of items could be slightly larger: in $O(n^{\delta})$ indices O(log n/log log n) items

Handling Collisions

- A secondary goal in a Hash Table: O(n) time enumeration of all the items stored in the table (in arbitrary order)
  - So each bucket should be enumerable
  - Should be able to move to next bucket from the end of one bucket
- Idea: A single global list of all items, formed by lists from all the (non-empty) indices linked together
  - At each non-empty index, head and tail of its segment in the global list can be stored
  - When an index is populated for the first time, its list can be inserted at the head/tail of the global list

Chaining

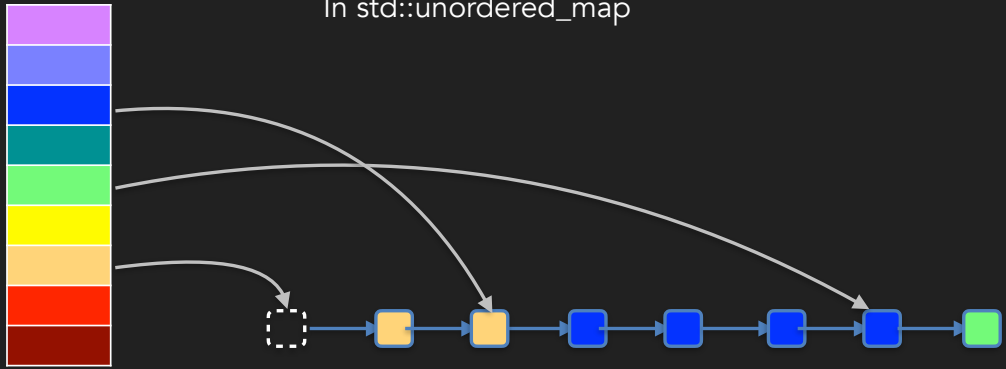
In std::unordered_map



Each index holds zero or more items, in a chain

Chaining

In std::unordered_map

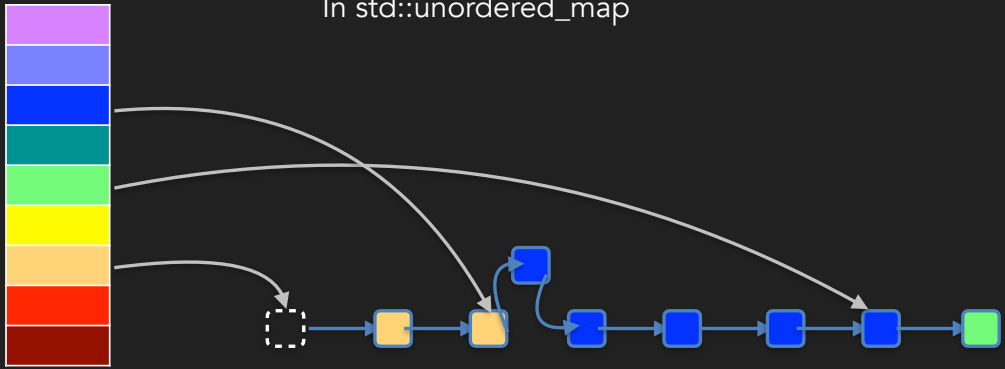


All items held in a single global list to enable O(n) time enumeration

While searching in a single sub-list, should be able to tell where it ends

Chaining

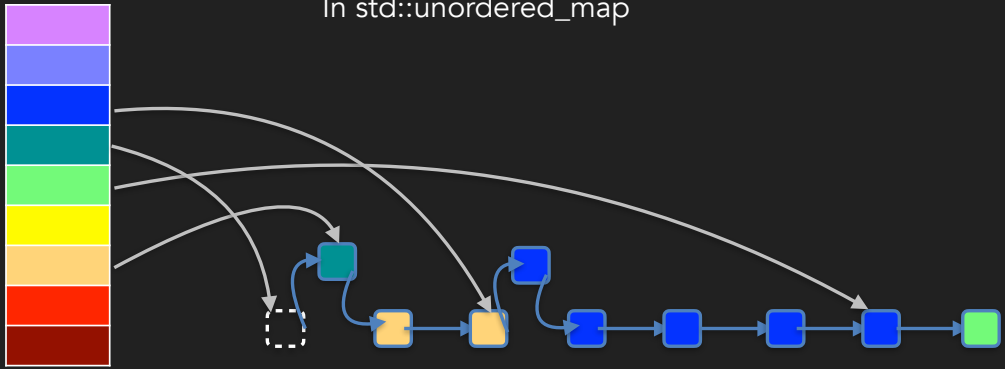
In std::unordered_map



New items at each index are added to the head of their sub-lists

Chaining

In std::unordered_map



New sub-lists are created at the head of the list

Optimisations

- In each item, store cached_tag = h(key), where $H = g_m \circ h$
- While searching in a list:
  - Compare cached_tags (integers) before comparing keys (which may be more complex)
  - Stop when g_m(cached_tag) changes -- it is the next bucket!
    - In each index store only a pointer to the start of its segment in the global list (or a nullptr): no need for storing the tail.
- When rehashing
  - Copy all items from the current global list into a new hash table, but while recomputing H, only need to compute g_m(cached_tag) now

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 26

Hash Tables

Open Addressing

Hash Tables

- Recall two paradigms for handling collisions:
  - At each index, allow multiple items to be held
    - The bucket holding the colliding items can grow arbitrarily, unless the hash functions are (cryptographically) strong
    - Items are scattered around in memory (linked together in a global linked list), negating cache benefits
  - When collision, find another spot in the hash table
    - Benefits from caching of the hash table
    - While inserting, may need to try multiple times before finding a spot. And while searching, will need to look at all those potential spots.

Closed Addressing

Open Addressing

Probing

- (Greedy) Probing template:

```
initialise an (implicit) sequence H₀(key), H₁(key), ...
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if inserting: // Assumes key not already present
                 if A[t] free: insert at A[t] and return
   if searching: if A[t] matches key:  return A[t]
                 else if A[t] never used: return not found
return no_space  // no space for key in the table
```

- Inserts at the first free location in the sequence (hence greedy)
- Later, when searching, guaranteed to look till the inserted location (will return early only if a slot that is never used is encountered, which insertion couldn't have skipped)
- If hash sequence is periodic mod m, N=m. If the hash sequence guaranteed to visit all m locations, N = m-#never used slots suffices: after N+1 probes guaranteed to see a never used slot.

Probing

- (Greedy) Probing template:

```
initialise an (implicit) sequence H₀(key), H₁(key), ...
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if inserting: // Assumes key not already present
                 if A[t] free: insert at A[t] and return
   if searching: if A[t] matches key:  return A[t]
                 else if A[t] never used: return not found
return no_space  // no space for key in the table
```



- To handle deletions, we mark a slot as never used, previously used (deleted), or currently used. Free slots are those which are not currently used.
  - Suppose key_1 is inserted at index t_1, followed by key_2 at index t_2. Also, suppose while inserting key_2, we encounter t_1 before reaching t_2.
  - Now suppose key_1 is deleted, and then we search for key_2. While looking for key_2 we should not stop at the free slot t_1.

Probing

- (Greedy) Probing template:

```
initialise an (implicit) sequence H₀(key), H₁(key), ...
for i = 0,...,N  // N can be set to m
   let t = Hᵢ(key)
   if inserting: // Assumes key not already present
                 if A[t] free: insert at A[t] and return
   if searching: if A[t] matches key:  return A[t]
                 else if A[t] never used: return not found
return no_space  // no space for key in the table
```

- To insert new elements, need the load factor to be < 1 (cf. closed addressing)
- Probing doesn't affect entries already in the hash table (later: probing with reordering)
- Efficiency determined by Probe-Sequence Length: number of iterations of the loop
- Above, inserting *assumed* key is not already present. If key present, should update instead of inserting a new item. Should do this without scanning the array twice.

Probing

- A more precise template. Initially, every slot in A has `(never_used,-)`.

```
Probe(key): // returns (FOUND,index) or (NOTFOUND,first free slot)
initialise an (implicit) sequence H0(key), H1(key), ...
first_free = -1
for i = 0,...,N  // N can be set to m
   let t = Hi(key)
   if A[t] matches key:  return (FOUND,t)
   if A[t] deleted or never_used: if first_free == -1: first_free = t
   if A[t] never_used: break // end the probe
return (NOTFOUND, first_free)
```

```
Search(key):
(status, slot) = Probe(key)
if status == FOUND: return A[slot]
else return not_found
```

```
Delete(key):
(status, slot) = Probe(key)
if status == FOUND: A[slot] = (deleted,-)
```

Tombstones can slow down probe

```
Insert(item):
(status, slot) = Probe(item.key)
if slot != -1: if A[slot] never_used, ++N; A[slot] = (used,item); return SUCCESS
else return FAILED
```

Uniform Probing

- Uniform probing: An "ideal" that we shall approximate
  - For each (key,i), H_i(key) is a fresh random index
- Expected probe-sequence length:
  - Suppose load factor is α = n/m. Each probe has a probability α of failing (i.e., landing in an occupied slot) and probability (1-α) of succeeding.
  - Expected probe-sequence length $= \sum_{i=1}^{\infty} i \times \Pr[\text{took } i \text{ tries}]$

$$= \sum_{i=1}^{\infty} i \times \alpha^{i-1}(1-\alpha) = \frac{1}{1-\alpha}$$

$\Theta\left(\frac{1}{1-\alpha}\right)$ is the best possible for any greedy, non-reordering probing strategy

  - Note: When α approaches 1, expected number of probes spike

Attendance Break!

Login using CSE LDAP id at

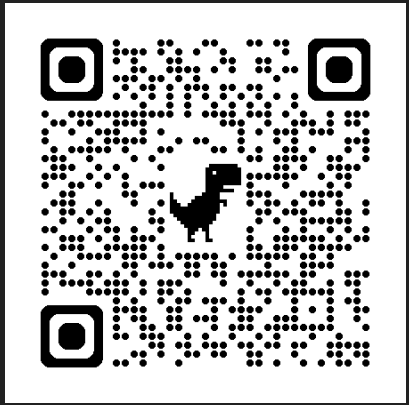
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Linear Probing

- Linear probing: $H_i(\text{key}) = H_0(\text{key}) + i \pmod m$ where H_0 is supposed to be a uniform hash
  - Simple: Can compute $H_i(\text{key})$ by simply incrementing (mod m)
  - Cache-friendly: Inspected locations will mostly be in the cache
  - But if $H_i(\text{key}_2)$ runs into $H_0(\text{key}_1)$, then it will have to traverse the segment already traversed by key_1
  - Expected probe-sequence length = $\Theta\left(\left(\frac{1}{1-\alpha}\right)^2\right)$
    - For α close to 1, much larger than in uniform probing

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 27

Hash Tables

Open Addressing (Ctd.)

Open Addressing

- Open addressing: Probe the hash table at a sequence of indices $H_0(key)$, $H_1(key)$, ...
  - Until reaching a location that has never been used
  - For inserting, first free location along the way is used (greedy)
  - In practice, probe sequence designed so that it reaches all indices in m steps
- Uniform Probing: For each (key,i), $H_i(key)$ is independent random (unrealistic)
  - Expected Probe-sequence length: $\Theta\left(\frac{1}{1-\alpha}\right)$
  - Optimal for greedy, non-reordering probing strategy
- Linear Probing: $H_i(key) = H_0(key) + i \pmod m$
  - Expected Probe-sequence length: $\Theta\left(\frac{1}{(1-\alpha)^2}\right)$

Quadratic Probing

- Quadratic probing: $H_i(key) = H_0(key) + i^2 \pmod m$
  - Still simple: increment H_i by 2i+1 (increment the increment by 2)
  - Somewhat cache-friendly: Initially, for small i, inspected locations will mostly be in the cache
  - Expected probe-sequence length close to uniform probing for small α
- The above basic quadratic probe sequence can visit at most half of the indices (which happens when m is prime). Alternatives visit all indices:
  - When $m=2^k$, $H_i(key) = H_0(key) + i + 2\,i^2 \pmod m$ covers all indices. So does $H_i(key) = H_0(key) + i(i+1)/2 \pmod m$.
  - When m is a prime s.t. m-1 is not a multiple of 4, $H_0(key) \pm i^2 \pmod m$

Double-Hash Probing

- Double-Hash probing: $H_i(key) = H_0(key) + i \times H_\Delta(key) \pmod m$
  - Requires computing two hashes instead of one
  - Not cache friendly: Steps of size $H_\Delta(key)$ while probing
  - Probe-sequence length behaviour very close to that in uniform probing
- If $H_\Delta(key)$ is co-prime with m, this sequence reaches all m indices
  - E.g., $m=2^k$ and $H_\Delta(key) = 2H_\delta(key)+1$
  - Because 0, a, 2a, ..., (m-1)a (mod m) is a permutation of 0,1,...,m-1 whenever gcd(m,a)=1
  - Why? If ai=aj (mod m), a(i-j) is a multiple of m. Since a has no factors of m, i-j has to be a multiple of m. Since |i-j| < m, we need i-j=0

Hybrid Probing

- We can consider a hybrid version that combines linear probing with another probing strategy h_0, h_1, ...
- Probe sequence: $T \cdot h_0(key)$, ... , $T \cdot h_0(key) + T\text{-}1$, $T \cdot h_1(key)$, ... , $T \cdot h_1(key) + T\text{-}1$, ...
- In modern hardware, can carry out a match on a small set of memory locations (T here) in one step
  - Match a one-byte *fingerprint* of the key (just another hash, typically implemented as a few reserved bits of h_0) against the fingerprints of the T locations. Look for a key match iff fingerprint matches.
  - Similarly, in one step can find a free slot out of the T slots
- Also, cache locality benefits of linear probing, along with probe-sequence length of the other probing strategy

Attendance Break!

Login using CSE LDAP id at

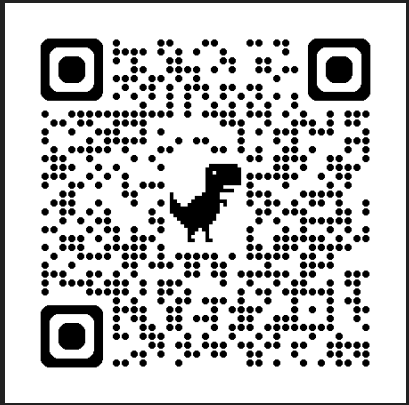
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Beating Uniform Probe

- For greedy probing, the best probe-sequence lengths were by uniform probe (or double-hash probe)
  - Expectation $\frac{1}{1-\alpha}$, and about 5% probes are over 3x the expectation
- Can we do better?
- Yes, by allowing reordering while probing!
- We'll see two schemes:
  - Robin Hood hashing: Take from the rich and give to the poor. Doesn't change the average, but virtually eliminates unlucky probes.
  - Cuckoo Hashing: Push out and occupy. Insert takes longer, but search is O(1), irrespective of the load.

Recent discovery: Also, possible without reordering, by not being greedy

Robin Hood Hashing

- Modified linear probing. Every item stores how far they are from their "home": `PSL(i) = i - h0(A[i].key) (mod m)`. The "rich" are close to their home.
- Inserting: Start linearly probing from the home index i_0, with `myPSL(i0)=0`.
  - As we are probing, the item we are holding gets poorer (`myPSL(i+1)=myPSL(i)+1`)
  - At an index i, if the item sitting there is richer than the item we are holding, (`PSL(i) < myPSL(i)`) then we swap the two. The richer item that is evicted continues down the linear probe path (getting poorer as it goes).
  - Continue till we find a free slot for the item we are holding.
- Searching: Stop when we encounter items richer than us.
- Deleting: Left shift each item to the right of the deleted item by one, till we encounter an empty slot or an item at its home. No tombstones!

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 28

Hash Tables

Open Addressing (Ctd.)

Open Addressing

- Open addressing:
  - Probe the hash table at a sequence of indices H_0(key), H_1(key), ...
- Probing without reordering: Uniform, linear, quadratic, double-hash
  - Hybrid: quadratic or double-hash probing combined with "parallel linear" probing
- Probing with reordering: Designed to "beat" Uniform Probing
  - Robin Hood hashing: Modified linear probing. Expected time for (successful or unsuccessful) search is $O(\frac{1}{1-\alpha})$, for insertion $O(\frac{1}{(1-\alpha)^2})$. Successful searches stay within an additive constant of expectation, and unsuccessful ones within O(log n) w.h.p.
  - Cuckoo hashing: O(1) worst-case search time. Insertion O(1) for small enough α.

Cuckoo Hashing

- We use d hash functions $H_1,\ldots,H_d$ where d is small (think of d=4)
- If key is stored, it will be at index $\in \{ H_1(\text{key}),\ldots,H_d(\text{key}) \}$: O(1) lookup time!
- Insertion algorithm, cuckoo_insert(item, $H_1(\text{key}),\ldots,H_d(\text{key})$):
  - Look for an empty slot in the given set. If found, insert item there.
  - If no empty slot:
    - insert into one of the given slots (at random), "evicting" the item already there
    - recursively, cuckoo_insert the evicted item into one of its other d-1 slots
- Key question: (how fast) will cuckoo_insert terminate?

Cuckoo Hashing

- Only with α^d probability do we need to start on the eviction path
  - E.g., at α=0.8, d=4, about 0.4 probability
- Once we do start on the eviction path:
  - For a given d, there is a critical load factor α_{max}
    - For d=4, $\alpha_{max} \approx 0.97$. For d=2, $\alpha_{max} \approx 0.5$.
  - If $\alpha < \alpha_{max}$ probability of needing an additional step $\approx \alpha_{effective} := \frac{\alpha}{\alpha_{max}}$
    - Expected number of steps $\approx O(\frac{1}{1 - \alpha_{effective}})$
  - If $\alpha \geq \alpha_{max}$, almost surely goes into an infinite loop!

Cuckoo Hashing

- α_{max} is a "phase-boundary"
  - For $\alpha < \alpha_{max}$ we terminate fairly quickly
    - If $\alpha < \alpha_{max} - \varepsilon$ (where $\varepsilon > 0$ a constant independent of n) then expected number of steps during insertion = $O(1/\varepsilon) = O(1)$, and highly unlikely we go, say, 5x above the expectation
  - For $\alpha \geq \alpha_{max}$, if we start on the eviction path, we almost surely go into an infinite loop!
- If insertion takes too many steps, then we we're very likely to be in the infinite loop phase (assuming a cryptographically strong hash)
  - Fix: Reduce α. i.e., grow the hash table (double it) and rehash
- Alternately, before α gets too close to α_{max}, grow and rehash (and use a cryptographically strong hash)

Attendance Break!

Login using CSE LDAP id at

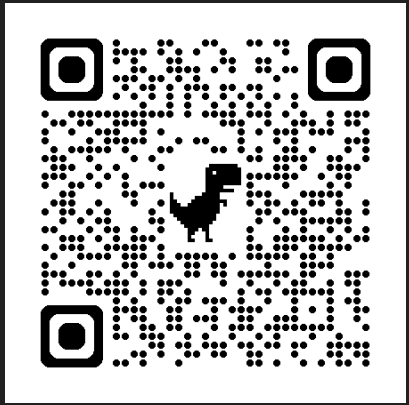
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Cuckoo Hashing with Parallel Checks

- Recall "parallel linear" probing:
  - Hash function yields an index to a "bucket" of B items
  - Fingerprints of all B items are checked in parallel (supported by hardware) against the current key's fingerprint. Full key checked only when fingerprint matches
- Extends to cuckoo hashing too: d hashes yield indices for d B-sized buckets. All Bd slots can be checked in one cycle.
- By increasing B, we can decrease d, without affecting α_{max}
  - E.g., d=2, B=4 has almost the same effect as d=4, B=1
  - Lower d is better (fewer hash functions to evaluate, fewer random locations to fetch), and a slightly large B is for free (still only one cycle)

Handling Adversarial Inputs

- Efficiency guarantees break down if the hash function doesn't behave like a uniformly random function (possibly under adversarial inputs)
- Can be prevented by using (light-weight) cryptographic hash functions
  - Against non-adaptive adversaries, enough to use a combinatorial hash function
- Alternately, can use a fallback mechanism on encountering a long eviction path/ probe path when load factor is small
  - Migration: Migrate the hash table to an alternate data structure (e.g., balanced BST, or a closed-addressing hash table with BST for each bucket)
  - Spillover stash: Add the input in hand to a stash implemented using an alternate data structure. When searching, first search in hash table, and then in the stash.

Hash Table Summary

- Main feature: practically O(1) lookup
  - In cuckoo hashing, guaranteed worst-case O(1)
  - In other open addressing schemes, expected O(1) time, assuming uniform hashing and $\alpha < 1 - \varepsilon$
    - In Robin Hood hashing, stays very close to $O(1/\varepsilon)$ for successful searches w.h.p. (and O(log n) for unsuccessful searches)
  - In closed addressing, assuming uniform hashing and $\alpha = O(1)$, all buckets remain practically O(1) size (technically $\Theta(\log n / \log \log n)$ w.h.p.)
- Also O(1) amortised insertion time
  - Insertion takes at least as much as an unsuccessful search, plus O(n) time when rehashing

Hash Table Practical Considerations

- Cache benefits
  - Looking up main memory several times adds a significant delay if the items are not cached. Using open addressing with a "hybrid" scheme avoids this delay.
    - Hybrid schemes for double-hash/quadratic probing and for cuckoo hash
    - Robin Hood hash is based on linear probing and avoids this delay.
    - In fact, closed addressing could also use chains of B-long vectors
- Parallelisation: Can check fingerprints of many slots in one step
- Adversarial inputs
  - Guarantees break down if the hash function used is not from a uniform hash family, and the inputs are crafted to exploit that
  - Can be avoided using (light-weight) cryptographically strong hash functions
  - Otherwise, in closed addressing, can fall back to BST for large buckets; in open addressing, migrate to BST/closed addressing table, or use a spillover stash

Data-Structures & Algorithms

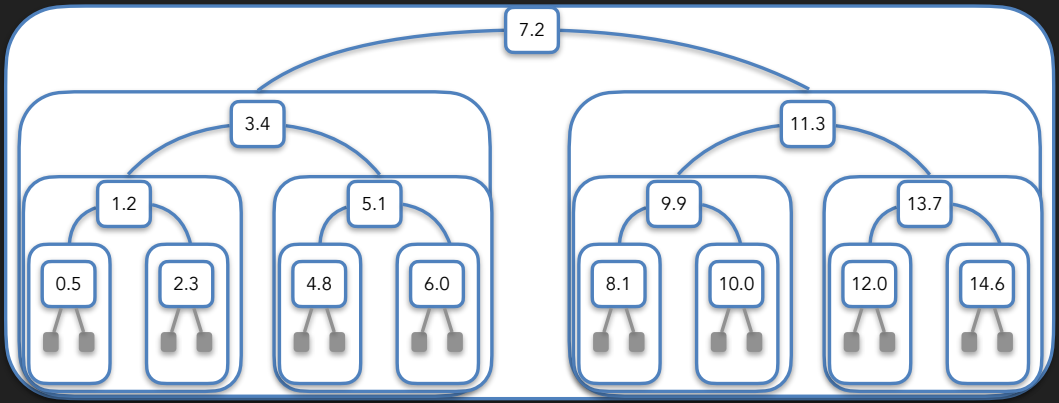
with

Manoj Prabhakaran

Lecture 29

Balanced BST

Recall: Binary Search Trees



A binary tree such that for any node with key K,

all nodes in its left subtree have keys $< K$, and

all nodes in its right subtree have keys $\geq K$.

Recall: Binary Search Trees

- BSTs are a "workhorse" in data structures. E,g, in C++ STL:
  - `std::set`, `std::map`, `std::multiset` and `std::multimap`
  - Can hold any data type with `operator<`
- Recall: BST allows search, insertion, deletion of worst-case time $O(h)$
  - In particular, no need for $\Theta(n)$ time "rehashing" like in hash tables
- Also recall: $h = \Omega(\log n)$ (because $n = O(2^h)$)
- Goal: Maintain $h = \Theta(\log n)$
- In C++ STL (and in most other major implementations) this is done using a class of balanced binary trees called Red-Black Trees
  - There are other schemes that maintain slightly better balance and hence allow slightly faster lookup, but take (a const. factor) more time for insertion/deletion

Attendance Break!

Login using CSE LDAP id at

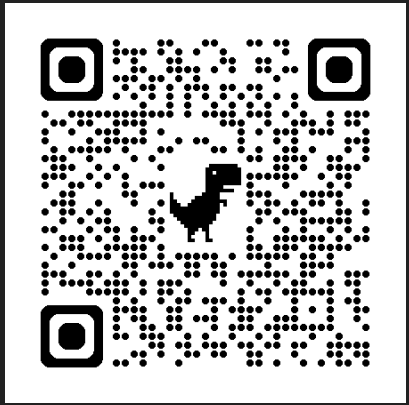
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Red-Black Trees

- Ideally, all root-to-leaf paths should be within ±1 of each other in length
  - E.g., in a nearly complete binary tree, in which case $h \leq \log_2 n$
- Maintaining a BST to be nearly complete is difficult/expensive
  - cf. Heap property can be maintained in O(log n) time per insertion/deletion
- We shall allow some slack:
  - All root-to-leaf paths are within a *factor 2* of each other in length
- Enough to guarantee that $h = O(\log n)$
  - If some leaf has depth h, every other leaf has depth ≥ h/2. So till depth h/2 a complete binary tree with $2^{h/2} - 1$ internal nodes. Overall, $n \geq 2^{h/2}$ and hence $\log n \geq h/2$ or $h \leq 2 \log n$

Red-Black Trees

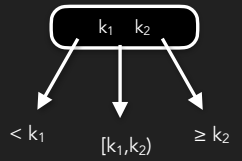
- In a Red-Black tree, every node is coloured black or red
- Main invariants:
  - Every red node has a black parent
  - Every node has a unique *black-height*
    - The number of black nodes in the path from that node to any leaf under it is the same
- If the invariants hold, then:
  - Depth of any leaf ≥ black-height(root)
  - Depth of any leaf ≤ 2 x black-height(root), because every red node in the path has a black parent that is counted in black-height(root)
  - So, all root-to-leaf paths are within a factor 2 of each other in length!

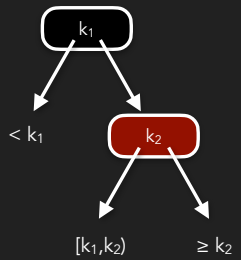
2-3-4 Trees

- Red-Black Trees can be viewed as an optimised representation of a perfectly balanced non-binary search tree, with 2, 3 or 4 children per node
- Non-Binary Search Tree
  - A node with d children carries d-1 keys $k_1 \leq \ldots \leq k_{d-1}$ and for i=0 to d-1, the ith child's subtree has keys in the range $[k_i, k_{i+1})$ (using $k_0=-\infty$ and $k_d=+\infty$)
    - E.g., binary nodes (d=2) have one key k_1 with child 0 holding keys $< k_1$ and child 1 holding keys $\geq k_1$
- Perfectly balanced: All leaves (empty nodes) appear at the same level
  - #leaves between 2^h and 4^h. #keys = #leaves-1

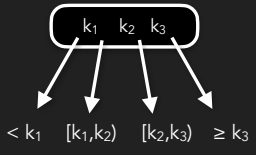
2-3-4 Trees and Red-Black Trees

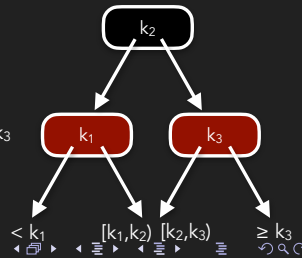
- A 2-3-4 tree can be "expanded" into a red-black tree
- Idea: A node with 3 or 4 children can be viewed as a combination of 2 or 3 binary nodes





- In the expanded binary tree, the original nodes are black and newly inserted nodes are red
- Black-height of any (black) node is the same as its height in the 2-3-4 tree
- Every red node has a black parent
- Root and leaves stay black





- How do you keep a 2-3-4 tree perfectly balanced?

Data-Structures & Algorithms

with

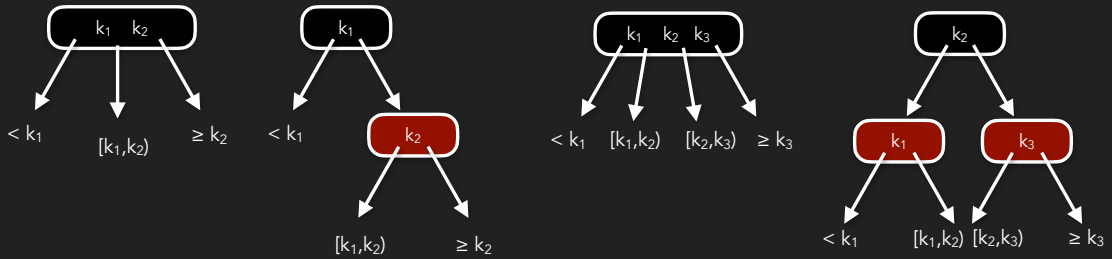
Manoj Prabhakaran

Lecture 30

Balanced BST (ctd.)

Recall

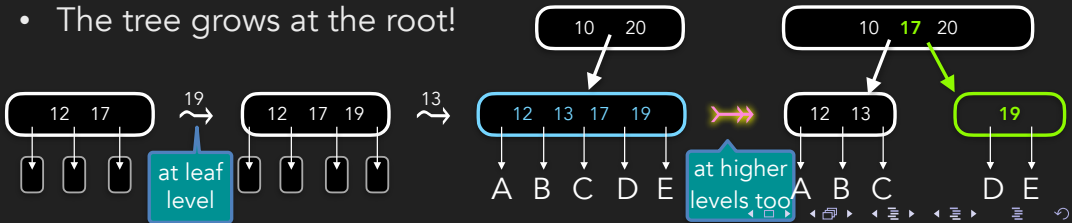
- A Red-Black tree is a BST with every node coloured black or red, and:
  - Every red node has a black parent
  - Every node has a unique *black-height*
- A 2-3-4 tree is a perfectly balanced search tree, with d-ary nodes, $d \in \{2,3,4\}$
- A 2-3-4 tree can be "translated" into a red-black tree



- How do you keep a 2-3-4 tree perfectly balanced?

Keeping 2-3-4 Trees Balanced: Insertion

- ASearch for the smallest interval where the key would go (at the bottom-most level), and insert the key into that node, increasing the node's number of children (2-ary → 3-ary, or 3-ary → 4-ary)
  - If it was already 4-ary (had 3 keys), split into two and push one of its keys up to its parent (recursively).
  - If root splits, form a new root
    - The tree grows at the root!



Attendance Break!

Login using CSE LDAP id at

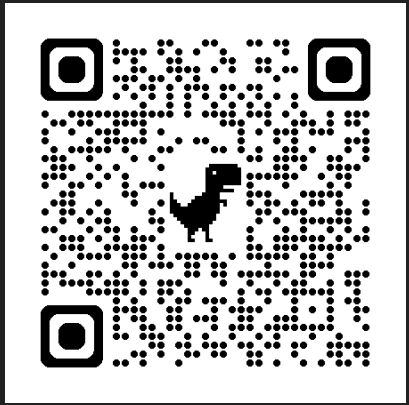
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!

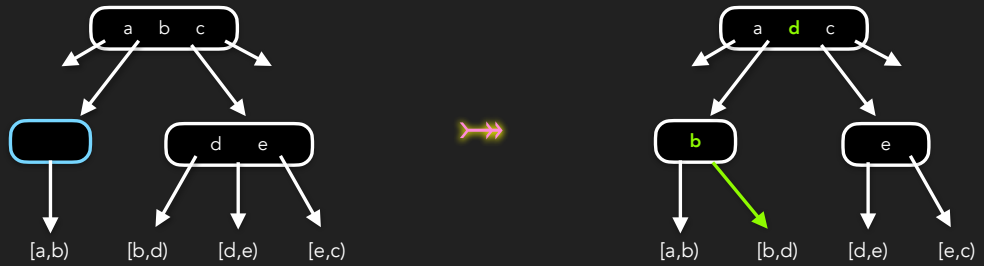


Keeping 2-3-4 Trees Balanced: Deletion

- To delete a key not at the bottom-most level, replace it with its in-order successor and delete that key (which is at the bottom-most level)
- To delete a key from the bottom-most level, first remove it from its node
  - If that node has no keys left (only one child), fix it by either:
    - *Transferring* a child from a 3-ary or 4-ary adjacent sibling; or,
    - *Merging* with a 2-ary adjacent sibling, by pulling the separating key down from the parent, to form a 3-ary node
      - If the parent becomes empty, recursively repopulate it, unless it is the root, in which case make this node the root.

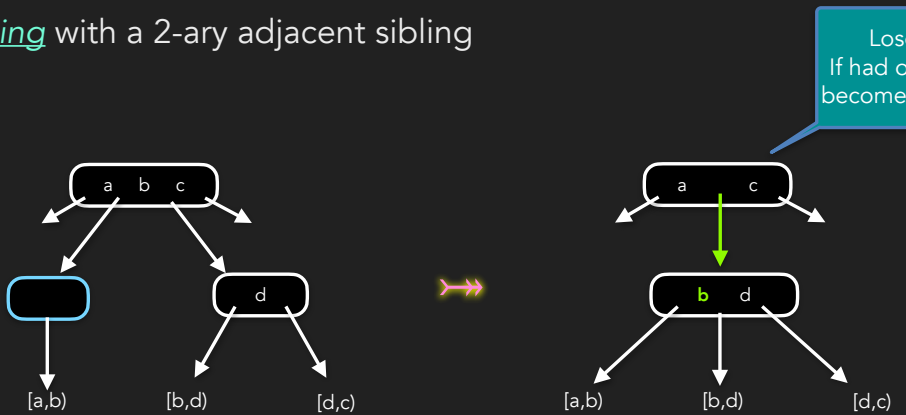
Keeping 2-3-4 Trees Balanced: Deletion

- Eliminating an empty node (with no keys and a single child)
- *Transferring* a child from a 3-ary or 4-ary adjacent sibling



Keeping 2-3-4 Trees Balanced: Deletion

- Eliminating an empty node (with no keys and a single child)
- *Merging* with a 2-ary adjacent sibling

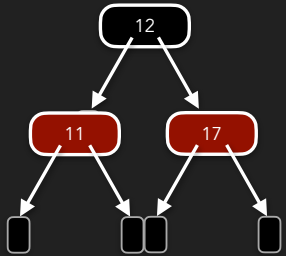


Translating to Red-Black Tree Operations

Insertion

- All the operations above can be translated to the binary red-black tree representation
- We will see a few examples (many similar cases omitted)
- Recall insertion: Find the bottom-most interval containing the new key and insert into that interval
  - In the binary tree, inserting into an interval translates to creating a child node
  - So insert normally, and colour it red

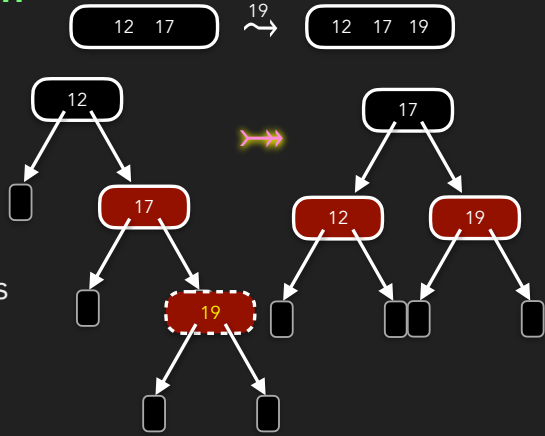




Translating to Red-Black Tree Operations

Insertion

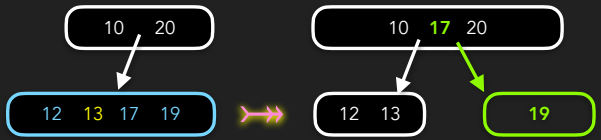
- But we *may* need a bit of adjusting
- If parent is red and parent-sibling is black, we have inserted into a 3-ary node expanded the "wrong way"
  - Switching the expansion corresponds to a "rotation" in the binary tree
- Not an issue when we are inserting into a 2-ary node (in 2-3-4 tree it goes from 2-ary to 3-ary; in red-black tree, a red child added to a black node)



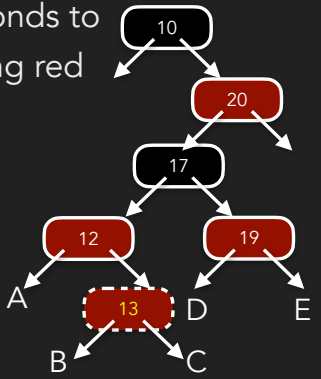
Translating to Red-Black Tree Operations

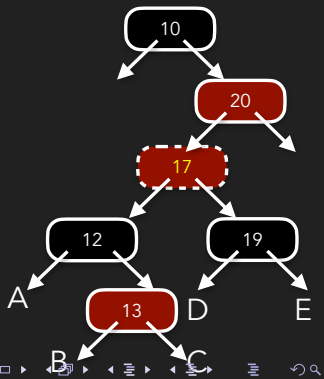
Insertion

- If inserting into a 4-ary node, we need to split (and recursively insert a key into the parent node)



- In red-black tree, this corresponds to parent and parent-sibling being red
  - Splitting is just recolouring

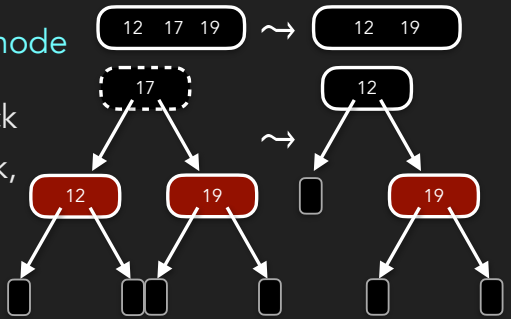




Translating to Red-Black Tree Operations

Deletion

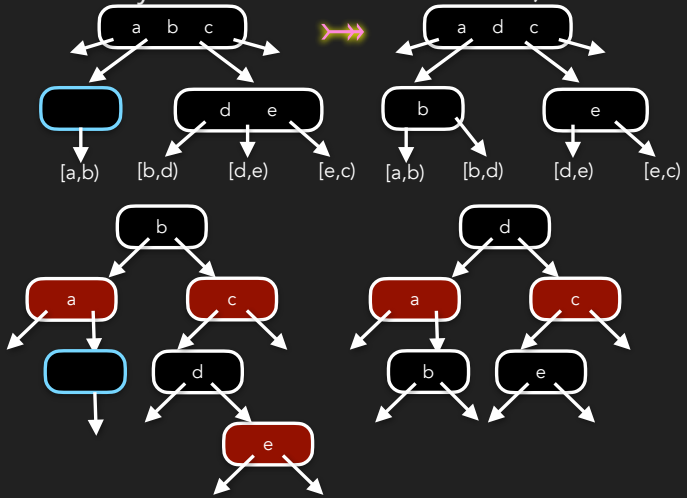
- Recall main step of deletion in 2-3-4 tree: delete from a bottom-most node
- When deleting a key from a 3-ary or 4-ary node
  - In red-black tree, if deleted node is black promote one of its red children as black, else simply remove it
- When deleting a key from a binary node
  - Need to repair an empty black node
  - Done recursively (as it can make the parent in 2-3-4 tree empty)



Translating to Red-Black Tree Operations

Deletion

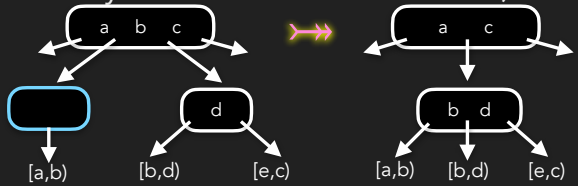
- Repairing an empty node (not necessarily at the bottom-most level)
  - If an adjacent sibling has ≥3 children, transfer a child
  - In red-black tree, a rotation involving an ancestor (possibly a grandparent)

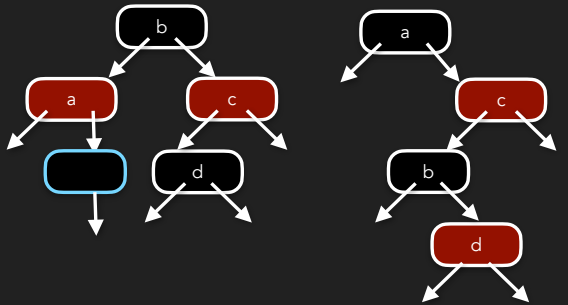


Translating to Red-Black Tree Operations

Deletion

- Repairing an empty node (not necessarily at the bottom-most level)
  - If no adjacent sibling with ≥3 children, then there must be an adjacent sibling with 2 children: merge with it

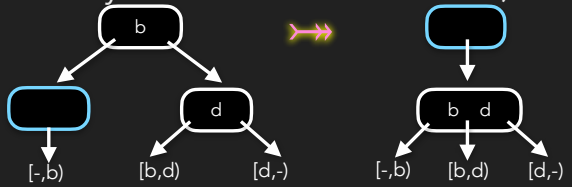




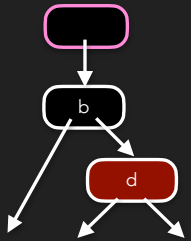
Translating to Red-Black Tree Operations

Deletion

- Repairing an empty node (not necessarily at the bottom-most level)
  - If no adjacent sibling with ≥3 children, then there must be an adjacent sibling with 2 children: merge with it



  - Merger can cause the parent in the 2-3-4 tree (a black ancestor in the red-black tree) to be empty
    - If it is the root, simply remove it
    - Else, recursively repair it



Data-Structures & Algorithms

with

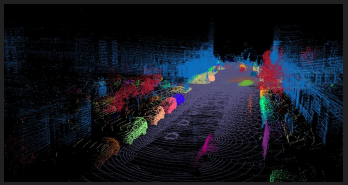
Manoj Prabhakaran

Lecture 31

Union-Find

a.k.a Disjoint Set Union

Partitioning Items into Sets



- A data-set from a LiDAR is a bunch of points in 3D space. Need to identify solid surfaces/objects
  - If two points "touch each other" (i.e., are in adjacent *voxels*) they are part of the same "object"
- Goal: Partition an incoming stream of points into objects
- Operations to support:
  make-set: create a new set with a single element
  find-set: given an element, find the set it is part of
  union: merge two sets into one set

When a point x arrives, make-set(x), and for each of its neighbours y, call union(x,y)

Attendance Break!

Login using CSE LDAP id at

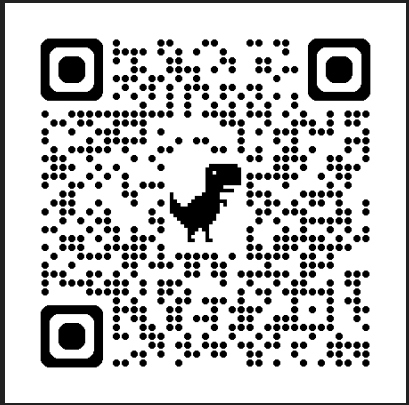
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Partitioning Items into Sets

- A first attempt
  - Keep elements in a standard data structure (say, a hash table). In each element, include a set ID (just an integer)
    - New make-set calls get a new ID: Can maintain a counter of last set ID used, or can use the key for the element itself.
  - make-set and find-set: O(1)
  - union: When taking the union of two sets, replace the set ID of all the elements in one of the sets with that of the other set
    - O(n) time!

A Forest

- Goal: (essentially) O(1) amortised time for make-set, find-set and union
- Challenge:
  - When two sets are combined, cannot afford to update individual elements in either set
- Idea: Each set is organised as a shallow tree (only parent pointers needed) and the item at the root of the tree serves as the set ID
  - A collection of trees is called a forest
  - union takes O(1) time: Merging two trees by simply attaching the root of one as the child of the other
  - make-set is O(1) too: just add a new tree with a single node to the forest
  - But find-set requires traversing parent pointers to the root

Working with Trees

- Each set is organised as a shallow tree (only parent pointers needed) and the item at the root of the tree serves as the set ID
- find-set (after locating the item's node, say via a hash table) requires traversing parent pointers to the root
  - The tree nodes are not limited in arity: so in the best case, all nodes are children of the root
  - In the worst case the tree is a chain of $\Theta(n)$ elements, and find-set takes time $\Theta(n)$
    - Each union operation can attach a large set to a set with one element and grow its height by 1
- Can we reduce find-set time while keeping union $O(1)$?

Union by Size

- Idea: In union, attach the smaller tree under the bigger tree's root
  - Intuition: Dissuades the bigger tree from getting taller
- What is the maximum height a tree with n items can have?
- For item x, let us track its depth, d(x), and the size of the set it is part of, S(x)
  - When x is created by make-set, d(x)=0, S(x)=1
  - In each union operation: If x's root stayed on as the root, then d(x) doesn't change, and S(x) doesn't decrease. Otherwise, d(x) increases by 1 and S(x) at least doubles (because the other set was not smaller)
    - Whenever d(x) increases, S(x) doubles (and S(x) never decreases)
    - So $S(x) \geq 2^{d(x)}$ or equivalently, $d(x) \leq \log_2 S(x)$

Better than O(log n)

- Idea: In union, attach the smaller tree under the bigger tree's root
  - Intuition: Dissuades the bigger tree from getting taller
- What is the maximum height a tree with n items can have?
  - O (log n)
- Can we do even better?
  - Remember, in the best-case the height of every node is at most 1
- Idea: Path compression
  - Once we do find-set(x), we *can* directly attach x to its root
  - Not just x, all the nodes in the path from root to x can be updated to point to the root directly

Path Compression

- Path Compression does not change the worst-case find-set time (the first find-set is the same as without path compression)
- But if there are several find-set queries, the amortised time is much less
- Let us consider the case when a sequence of k find-set queries are made on n items, with no intervening unions
  - Time taken in a find-set = O(#nodes moved to under the root) + O(1)
    - Even if no node is moved, we take O(1) time to find the root
  - Each item can be moved up to under its root at most once
  - So total time for k find-set queries = O(n+k)
- If $k = \Omega(n)$, the amortised time is O(1)

Path Compression

- Amortised time for Ω(n) find-set queries without intervening unions is O(1)
- What if there are intervening unions?
  - A node moved to under its root can again be moved, if that root got connected to the root of another tree due to a union
  - Cannot bound the total number of moves by O(n), or the amortised cost by O(1), any more
  - Turns out, it is practically O(1) still
    - m operations, involving n make-sets, takes overall time O(m α(n)), where α is the inverse Ackermann function
    - For all practical n, $\alpha(n) \leq 4$

Representation of Nodes

- In union-find, the elements are never deleted
- Allows representing all the nodes in the forest as a vector `A`
  - Once a node is added to `A`, its index in `A` never changes
  - Can use the index in `A` instead of a pointer
- For each node we should store: if it is not a root node, its parent's index, and if it is a root node, the size of its tree (for union by size)
  - A trick: `A[i]` is a signed integer s.t. if `A[i] ≥ 0`, then `parent(i) = A[i]` and if `A[i] < 0`, then node i is a root of a tree with size `-A[i]`
  - The forest is just `std::vector<int>`

An Extension

- Suppose we would also like to support enumeration of a set (returned by find-set) in worst-case time proportional to the size of that set (independent of n)
  - Our trees only carry parent pointers: cannot traverse the tree from the root
  - Option 1: Maintain child pointers and traverse the tree from the root
    - Since arity is unbounded, maintain children as a linked list at each node. While flattening, move all the siblings of each ancestor also directly under the root.
  - Option 2 (simpler): Maintain a circular linked list (unrelated to the tree's topology) tying all the nodes in a set together
    - Root appears somewhere in this ring, from where all elements can be discovered. During union, merging the rings of two sets takes O(1) time. The rings are not changed during find-set operations.

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 32

Graphs

Representation & Exploration Template

Graphs

- A graph is a representation of binary *relations*
  - A set of entities V and a relation $E \subseteq V \times V$ that contains all pairs (u,v) such that "u is related to v"
  - An element in E is called an *edge*, and it "connects" a pair of *nodes* in V
- E.g., Webpage u points to webpage v; profile u is a friend of profile v; point u is connected to point v by a road; game state u can evolve into game state v in one step; package u must be installed before installing package v; etc.
- Several graph algorithms and data structures enable a large number of applications that require answering questions about large graphs quickly

Graphs

- A *path* in a graph is a sequence of distinct nodes u_0, u_1, ..., u_k such that every pair $(u_i,u_{i+1}) \in E$
- Note that in general, $(u,v) \in E$ doesn't necessarily mean $(v,u) \in E$ (e.g., package dependencies)
  - But in some graphs, we have $(u,v) \in E \Leftrightarrow (v,u) \in E$. They are symmetric or *undirected* graphs (unless otherwise stated, our graphs are directed)
- In some applications, the edges have a "weight": e.g., the distance of a road, network latency or bandwidth, probability of state transition, etc.

Representing Graphs

- Basic requirements from a data structure storing a graph
  - Enumerate outgoing edges: Given a node u, enumerate all nodes in its *neighbourhood*: i.e., all v s.t. $(u,v) \in E$
    - Less often: Enumerate incoming edges
  - Given a candidate edge (u,v), check if $(u,v) \in E$
  - Add/delete a node/edge
- Nodes are typically labeled {0,...,n-1}
  - If labeled by other keys, can use an auxiliary hash table to map keys to indices
- There are different data structures with different trade-offs in time, space and features
  - E.g., if fast neighbourhood enumeration not required, can store all (u,v) pairs in a standard set data structure

Attendance Break!

Login using CSE LDAP id at

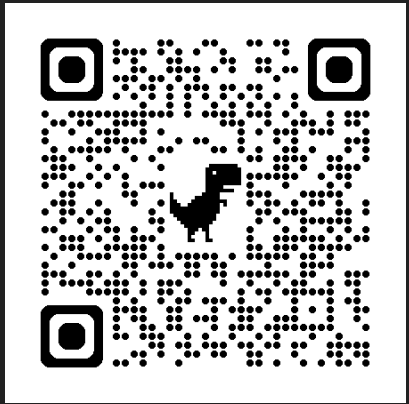
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



Adjacency Matrix

- V = {0,..,n-1}. Store E as an n×n 2D-matrix, of 0s and 1s
  - `A[u][v] == 0` means $(u,v) \notin E$, `A[u][v] == 1` means $(u,v) \in E$
  - Implemented as an array where $(u,v)^{th}$ index is at location $u \cdot n+v$
- Can instantly check if $(u,v) \in E$
- Can quickly enumerate outgoing edges: $\Theta(n)$ time, but elements will be in cache
  - Can enumerate incoming edges too (but less cache friendly)
- Main drawback: $\Theta(n^2)$ space. Not suitable for large, sparse graphs
  - Somewhat mitigated by packing the matrix entries as bits (`std::bitset<64>` stores 64 bits in one int rather than in 64 bools). Enables exploiting parallelisation of certain operations (e.g., intersection or union of neighbourhoods)

Adjacency List

- Store E as an array of lists: A[u] has a list of all v s.t. $(u,v) \in E$
  - Total space used is $\Theta(m+n)$ rather than $\Theta(n^2)$, where $|V|=n$, $|E|=m$
- To check if a given $(u,v) \in E$, check the list stored in A[u]
- Suited only for sparse graphs where all nodes have small degree
- The "list" of neighbours should be stored as a vector, not a linked-list
  - More cache-friendly, and no overhead from the links
  - Can add a neighbour using push_back in O(1) time. Can delete a neighbour (after having found it) in O(1) time by popping the last one and writing it back on top of the neighbour to be deleted.

Adjacency Map

- When using an adjacency list, if a few nodes have high degree, looking up their neighbours is slow
  - In many real-life applications, graphs have widely varying degrees
- Idea: For such large-degree nodes, instead of a list (vector), use a *set* data structure
  - Nodes with small degrees will still use a vector, to be more cache friendly
  - If neighbourhood needs to be explored in sorted order, use a balanced BST; otherwise a hash table gives O(1) amortised access time (w.h.p.)

Compressed Sparse Row

- Recall adjacency list representation: A[u] has a vector of all neighbours of u
  - To be more cache-friendly, we can concatenate all these vectors into a single vector, `nbrs[0,..,m-1]` (m is the total number of edges)
  - Will maintain boundaries of the original vectors in `offset[0,..,n]`
    - Neighbours of node u in `nbrs[offset[u],..,offset[u+1]-1]`
    - If u has no outgoing edges, empty range: `offset[u]==offset[u+1]`,
    - `offset[n]` is set to `m`
  - Like adjacency list, not good for high-degree nodes. Further, not suited for addition/deletion of edges.

Exploring a Graph

- In many applications using graphs, important to be able to start at a "source node" and discover all nodes reachable from s
- A general template

```
// frontier: a container for nodes; discovered[v] false for all nodes v
frontier.insert(s); discovered[s] = true;
while (!frontier.empty()) {
  u = frontier.extract();           // then may process u as necessary
  for ( v : graph.neighbours(u) ) {
    if(discovered[v]) continue;
    frontier.insert(v); discovered[v] = true;
  }
}
```

- Main variation: Which node will be extracted from frontier (LIFO/FIFO/...)

Exploring a Graph

- No matter what container is used for frontier, as long as extraction on a non-empty container removes and returns some element, the algorithm terminates (every node is inserted into frontier at most once, and in each iteration one node is extracted), and the set of nodes discovered $s=u_0, u_1, \ldots, u_t$ are exactly all the nodes reachable from s
- Any node discovered is reachable
  - Inductively, u_k is reachable using nodes $u_0,\ldots,u_{k-1}$: u_k was discovered from some node u_i, $i<k$, and inductively there is a path from s to u_i; add to this path the new edge (u_i,u_k) to get a valid path to u_k
- Any reachable node is discovered
  - Suppose there is a path $s=v_0, v_1, \ldots, v_t=v$, but v is not discovered. Let v_i ($i>0$) be the first undiscovered node in this path. So v_{i-1} was discovered and added to the frontier. Before the frontier becomes empty, v_{i-1} must be extracted. But then v_i will be discovered!

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 33

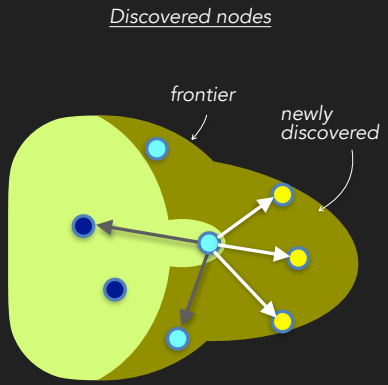
Graphs

Breadth-First Search

Exploring a Graph

- Recall the template for graph exploration:

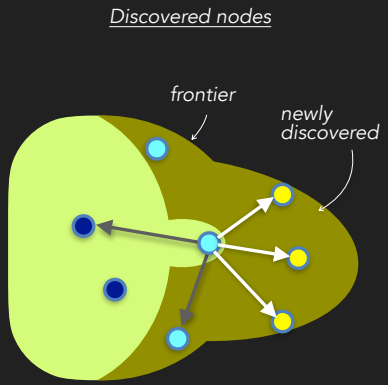
```
// discovered[v] false for all nodes v
frontier.insert(s); discovered[s] = true;
while (!frontier.empty()) {
  u = frontier.extract();
  for ( v : graph.neighbours(u) ) {
    if(discovered[v]) continue;
    frontier.insert(v); discovered[v] = true;
  }
}
```



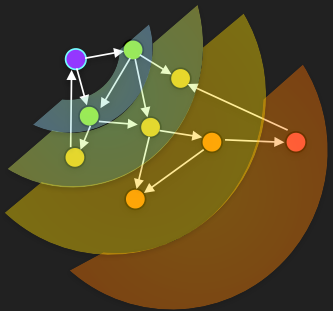
- With any container (which guarantees that every inserted item will be extracted before turning empty) will reach all reachable nodes

Exploring a Graph

- Time?
  - Each reachable node gets into and out of the frontier exactly once. ≤ min(|V|,|E|) reachable nodes.
    - ≤ min(|V|,|E|) iterations of the loop
  - In each iteration, examine each incident edge (inner loop). Each reachable edge examined once: O(|E|) time examining edges
    - If using adjacency matrix, |V| time to discover incident edges in each iteration
  - Total time: O(|E|) or $O(|V|^2)$ if using adjacency matrix



Breadth-First Search (BFS)

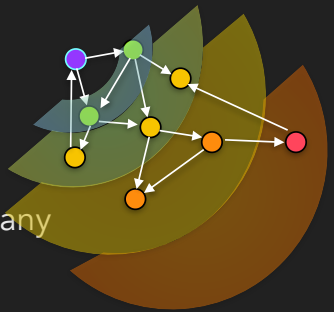


- BFS if frontier implemented as a FIFO structure (queue)
- To study BFS, we first define *level sets* in a graph
- Let d(s,v), for *distance* from s to v, denote the shortest length of a path from s to v (with d(s,s)=0).
- *Level-set* $L_{s,k} = \{ v \mid d(s,v) = k \}$ (when s understood, will call it simply L_k)

1. All edges reachable from s are from $L_{s,j}$ to $L_{s,k}$ where $k \leq j+1$ **[Why?]**
2. If $v \in L_k$ ($k>0$), there is an edge (in any shortest s-v path) (b,v) s.t. $b \in L_{k-1}$ **[Why?]**

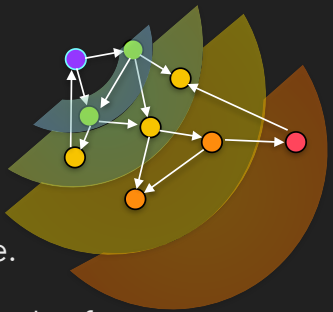
- Claim: In BFS, nodes enter the frontier queue ordered by levels

Breadth-First Search (BFS)



- Claim: For all j, all reachable nodes in L_j are enqueued before any in node in $L_{>j}$ is enqueued
- Base case: j=0: only s is in L_0 and it is enqueued first
- Suppose holds for up to j=k-1 (k>0). To prove that for any node $v \in L_k$ no node in $L_{>k}$ enqueued before it.
  - Let z be the first node in $L_{>k}$ enqueued, discovered through edge (x,z).
  - Choice of $z \Rightarrow x \in L_{\leq k}$, $z \in L_{>k}$. Since (x,z) is an edge, by 1, $x \in L_k$, $z \in L_{k+1}$
  - By 2, there is a node $u \in L_{k-1}$ with an edge (u,v). By IH, u enqueued before x enqueued. Queue $\Rightarrow$ u extracted before x extracted.
  - when v enqueued $\leq$ when u extracted < when x extracted = when z enqueued

Breadth-First Search (BFS)



- Node $v \in L_k$ must be discovered from a node $u \in L_{k-1}$
  - While extracting from the queue, level-by-level, v not seen from any node in $L_{<k-1}$ (1) and is seen from at least one node in L_{k-1} (2): u is first of those.
- Aside: so at any point, the frontier queue has all its nodes from at most two adjacent levels (or has all the reachable nodes from one level)
- The levels of nodes entering the queue can be inductively kept track of
  - If level of the dequeued node = k-1, level of enqueued nodes = k

Attendance Break!

Login using CSE LDAP id at

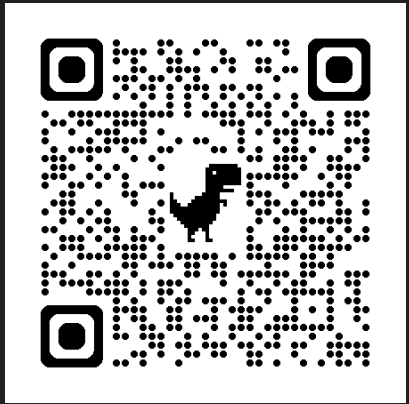
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



BFS and Shortest-Path

- An important application in a graph: Given a pair of source-target nodes (s,t), check if t reachable from s, and if so, find the distance d(s,t)
  - Run BFS from source s and if t is reached, report the level k s.t. $t \in L_{s,k}$
- Can we also find a shortest path from s to t?
  - Yes! When a node v is discovered from u, record `discovered[v]=u` (instead of just a boolean flag); use, say, `-1` to indicate not discovered, and let `discovered[s]=s`
  - Let $u_0 = t$, u_{i+1} = `discovered[`u_i`]`, until $u_k == s$. Path is $u_k, u_{k-1}, \ldots, u_0$
  - In the traversal template, with any container , gives some s-t path

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 34

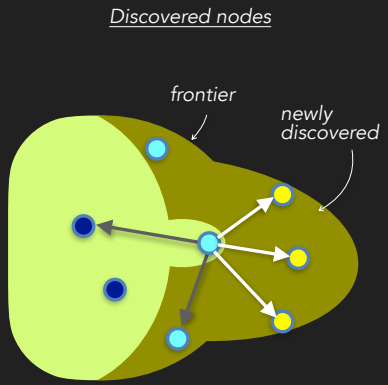
Graphs

Depth-First Search

Exploring a Graph: Template

- Recall the template for graph exploration:

```
// discovered[v] initially false for all v
frontier.insert(s); discovered[s] = true;
while (!frontier.empty()) {
  u = frontier.extract();
  for ( v : graph.neighbours(u) ) {
    if(discovered[v]) continue;
    discovered[v] = true;
    frontier.insert(v);
  }
}
```



- Marking on discovery prevents inserting a node twice into the frontier

Exploring a Graph: Template 2

- Can instead just prevent the same node from being *processed* twice

```
// activated[v] initially false for all v
frontier.insert(s);
while (!frontier.empty()) {
  u = frontier.extract();
  if(activated[u]) continue;
  activated[u] = true;
  for ( v : graph.neighbours(u) )
    frontier.insert(v);
}
```

O(|E|) iterations instead of O(|V|).
Still total time is O(|E|)

- Allows a node to get inserted into the frontier multiple times (as much as its incoming degree), possibly allowing it to be activated earlier
  - Not relevant to BFS, where first insertion will determine the extraction.
    Nor in a tree, where a node has in-degree 1.

Depth-First Search (DFS)

- DFS: Derived from our Template 2 (mark-on-activation), if frontier is implemented as a LIFO structure (stack)
- We have already seen it!
  - For (binary search) trees, with root as the source node
  - Pre-order, post-order and in-order traversal, all used DFS
    (but printed out the node at different points)
- What is new?
  - A graph is more general than a tree. In particular, multiple paths between a pair of nodes, or cycles.

Standard DFS

- Standard DFS is a recursive process

```
DFS(u) {
  activated[u] = true;
  for ( v : graph.neighbours(u) ) {
    if(activated[v]) continue;
    DFS(v);
  }
}
```

- In practice, for efficiency and for avoiding stack overflow on "deep" graphs, implemented without recursion, using an explicit stack
- This version behaves identically (in the order in which nodes are visited) to our mark-on-activation template, but the stack contents are slightly different

Standard DFS vs. DFS From Template 2

```
DFS(u) {
  if(activated[u]) return;
  activated[u] = true;
  for ( v : graph.neighbours(u) )
    DFS(v);
}
```

Recursive DFS

- u stays on in the (call-)stack, until all its neighbours are popped from the stack

```
frontier.insert(s);
while (!frontier.empty()) {
  u = frontier.extract();
  if(activated[u]) continue;
  activated[u] = true;
  for ( v : graph.neighbours(u) )
    frontier.insert(v);
}
```

Template DFS

- u popped from the stack, and all its neighbours pushed onto the stack.

- Say u has children $[v_1 \ldots v_d]$, and stack has (H,u) with history H

  In template: $(H,u) \rightarrow (H,v_d \ldots v_1) \dashrightarrow (H,v_d \ldots v_2) \dashrightarrow (H,v_d) \rightarrow (H)$

  In recursive: $(H,u) \rightarrow (H,(u,1),v_1) \dashrightarrow (H,(u,2),v_2) \dashrightarrow (H,(u,d)) \rightarrow (H)$ (stack frame has for-loop counter)

Equivalent executions with stack entries represented differently

Standard DFS

- Standard DFS obtained by a further minor optimisation

```
DFS(u) {
  if(activated[u]) return;
  activated[u] = true;
  for ( v : graph.neighbours(u) )
    DFS(v);
}
```

```
DFS(u) {
  activated[u] = true;
  for ( v : graph.neighbours(u) ) {
    if(activated[v]) continue;
    DFS(v);
  }
}
```

- Do not make the recursive call if it is just going to return
  - Still O(|E|) time, but the number of function calls is O(min(|V|,|E|)) instead of O(|E|), as DFS is called on each reachable node exactly once
- For further efficiency/avoiding stack overflow often implemented without recursion, using an explicit stack

Active Nodes in DFS

- To study DFS (recursive version) consider the "active lifetime" of a node u, from the point when DFS(u) is invoked till it returns
  - Define activation time A[u] and exit time X[u]

1. At any point all active nodes lie in a path from the source to the last activated
   - These are the nodes in the call-stack
2. If v reachable from u, and not activated when u activated (i.e., A[u] < A[v]) then v gets activated while u is still active. Further, then v must exit while u is still active (i.e., A[u] < A[v] < X[v] < X[u])
   - If v not reachable from u, and A[u] < A[v], then A[u] < X[u] < A[v] < X[v]

Cycle Detection

- A cycle: $u_0,\ldots,u_d$ s.t. $(u_{i-1},u_i) \in E$ for i=1,...,d, and u_i's distinct except $u_0=u_d$
- DFS (recursive version) can be easily adapted to check if a graph has a cycle.
- W.l.o.g, assume there is a source node from which all nodes are reachable
  - If necessary, add a new source node with an edge to every node: this cannot add or remove cycles. $|E_{new}| = |V| + |E|$, $|V_{new}| = |V|+1$.
- To detect cycles, run the following variant of DFS from this all-reaching source

```
DFS(u) {  // uses external boolean vectors activated and active
  active[u] = true; activated[u] = true;
  for ( v : graph.neighbours(u) ) {
    if(active[v]) report_cycle();
    if(activated[v]) continue;
    DFS(v);
  }
  active[u] = false;
}
```

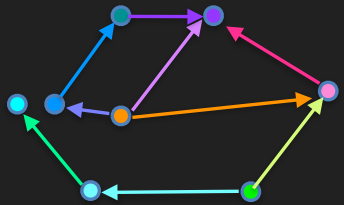
Saw active v from active u

Cycle Detection

- Claim: If it reports a cycle, there is a cycle
  - Reports a cycle: saw an edge (u,v) when v is active
    - u is the last activated node. So there is a path s ⇢ v ⇢ u (by 1). Edge (u,v) completes the cycle v ⇢ u → v
- Claim: If there is a cycle C (reachable), it will be found
  - Let v be the first node in C that is activated. Let u be s.t. C is v ⇢ u → v.
    - If u=v (C is just a self-loop), DFS(v) will see the edge to active v.
    - Suppose u≠v. Then u not activated when v activated. So DFS(v) will activate u eventually (by 2); then DFS(u) will see active v and report cycle.

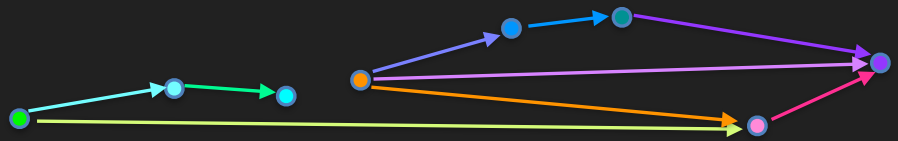
Topological Sorting

- A topological sort is an ordering of all the nodes in a directed graph such that all the edges point forward



Topological Sorting

- A topological sort is an ordering of all the nodes in a directed graph such that all the edges point forward



- Not possible if there is a cycle
- Turns out, otherwise, always possible!
  - A directed graph without cycles is called a Directed Acyclic Graph (DAG)

Topological Sorting

- Claim: Every DAG can be topologically sorted
  - In a DAG, the cycle detection DFS never reports a cycle
  - Claim: In a DAG, for any edge (u,v) , X[v] < X[u]. Consider cases:
    - A[v] < A[u]: Can't have A[v] < A[u] < X[v], because then cycle reported on exploring edge (u,v). So X[v] < A[u] and hence X[v] < X[u]
    - A[u] < A[v]: Since v reachable from u, by 2️⃣, A[u] < A[v] < X[v] < X[u]
  - Ordering nodes by decreasing X time-stamp gives a topological sorting!
- We may have added an artificial source s for DFS. X[s] is the maximum, and hence the first node in the topological ordering. Drop it if artificial node.

Attendance Break!

Login using CSE LDAP id at

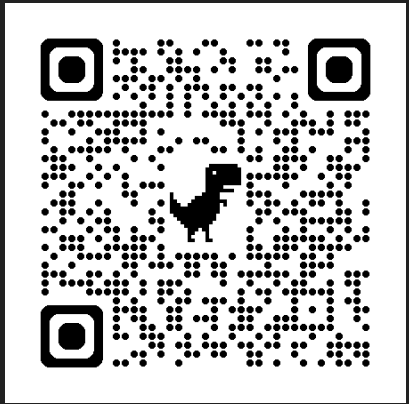
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 35

Graphs

Shortest-Path in Weighted Graphs

Recall: BFS and Shortest-Path

- An important application in a graph: Given a pair of source-target nodes (s,t), check if t reachable from s, and if so, find the distance d(s,t)
  - Run BFS from source s and if t is reached, report the level k s.t. $t \in L_{s,k}$
- Can we also find a shortest path from s to t?
  - Yes! When a node v is discovered from u, record `discovered[v]=u` (instead of just a boolean flag); use, say, `-1` to indicate not discovered, and let `discovered[s]=s`
  - Let $u_0 = t$, u_{i+1} = `discovered[`u_i`]`, until $u_k == s$. Path is $u_k, u_{k-1}, \ldots, u_0$
  - In the traversal template, with any container , gives some s-t path

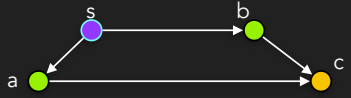
Weighted Graph

- We had defined d(s,v) = minimum number of edges in a path from s to v
  - Each edge contributes equally to the length of a path
- But in many applications, not all edges are equal
  - Cost of a step, e.g., distance on a map, latency in a network link, unlikelihood in a probabilistic model, ...
  - If the costs are all small integers, can model an edge with cost k by a path of length k (involving k-1 dummy nodes). But not feasible/efficient for arbitrary costs.
- Weighted Graph: A graph with non-negative costs, or weights, on the edges. Now, path-length adds up edge weights. d(s,v) = minimum length of a path from s to v.

Shortest Path in a Weighted Graph

Dijkstra's Algorithm

- As in BFS, we will explore the nodes nearest to the source first
- Main difference: in BFS we find the distance to a node when we discover it. In exploring a weighted graph this is not true
  - E.g., c discovered through path s→a→c since a is closest to s, but s→b→c can be shorter!



- Idea: Allow distance to be updated each time a node is discovered
  - This will be enough to ensure that we know the next nearest node to explore at any point!

Attendance Break!

Login using CSE LDAP id at

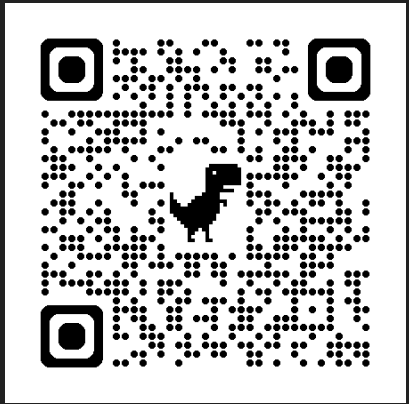
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



Dijkstra's Algorithm

- We shall use a "priority queue" as the frontier
  - holds (u,k) pairs (k possibly ∞); we call k = D(u), the recorded distance of u.
  - (u,k) can be updated to (u,k') only if k' < k
  - extraction yields the node with the minimum recorded distance

```
distance(s,t) {
  for (v : V) frontier.insert(v,∞);
  frontier.decrease(s,0);
  while(true) {
    (u,k) = frontier.extract_min();   // guaranteed that d(s,u) = k
    if(u==t) return k;
    if(k==∞) return ∞;                // all remaining nodes unreachable
    for ( v: graph.neighbours(u) ) {
      duv = k + graph.weight(u,v);    // distance from s to v using (u,v)
      frontier.decrease(v,duv);       // updates (v,j) → (v,duv) iff duv < j
    }
  }
}
```

If k == ∞, then D(w) == ∞ for all unexplored w. So no edge from any explored node to unexplored nodes: i.e., a *cut* between explored and unexplored

Dijkstra's Algorithm

- Claim: When extracting any reachable node u, recorded distance D(u) = d(s,u)
- Base case: first extracted item is (s,0) and d(s,s)=0.
- Induction: S be the set of nodes extracted so far, and T = V \ S.
  IH: $\forall u \in S$, D(u) = d(s,u). Next extracted node be w $\in$ T (assumed reachable).
  - Consider a shortest s-w path P, and v be the first node in T on P
    Then P upto v is a shortest s-v path (else can find a shorter walk, and hence path, to w) and $d(s,v) \le d(s,w) \le D(w) \le D(v)$.
  - Let s ⇢ u → v be this path. By choice of v, we have $u \notin T$. i.e., $u \in S$. Hence, by IH, when u was extracted, D(u) = d(s,u) and we updated D(v) = D(u) + w(u,v) = d(s,v). (D(v) cannot increase after that.)
  - So $d(s,v) \le d(s,w) \le D(w) \le D(v) \le d(s,v)$. Hence D(w) = d(s,w)

D(w) is the length of an s-w path we have seen

non-negative edge weights

Dijkstra's Algorithm

- How do we implement the priority queue?
- Practical choice: A binary (min-)heap
  - Additional feature needed: locate an entry for a node, and decreasing its key
    - Once node located on the heap, decreasing a key can be done in O(log |V|) time, by bubbling up
    - How do you locate a node on the heap?
      - Use a directory (vector) to map each node to a position in the heap
        - While bubbling up, the directory should be updated
  - Alternative: Instead of decreasing a key, just and a new entry with the new key. On extracting, check if the same node was extracted earlier, and if so ignore it and continue. Slightly more expensive (|E| items stored instead of |V|), but can use std::priority_queue as a blackbox

Can be made more efficient asymptotically, using "Fibonacci heaps," but rarely used in practice

Dijkstra's Algorithm: Time

- Initialisation of priority queue with (u,∞) for all u
  - Can actually be omitted by just not inserting them into the heap, and treating items not in the heap as having key ∞
- Every reachable node is extracted exactly once: $O(|V_{rch}|)$ iterations
  - In each iteration: extract an element from the priority queue with at most $O(|V_{rch}|)$ items, in $O(\log |V_{rch}|)$ time.
- Inner loop, across all iterations of the outer loop, explores reachable edges: $|E_{rch}|$ iterations, and in each iteration decrease a key: $O(|E_{rch}| \log |V_{rch}|)$ time
- Total time: $O(|E_{rch}| \log |V_{rch}|)$ since $|V_{rch}| \leq |E_{rch}| + 1$

With Fibonacci heaps decrease-key is O(1) amortised time: $O(|E_{rch}| + |V_{rch}| \log |V_{rch}|)$

Dijkstra's Algorithm: Variations

- We proved that for every node u, when it is extracted, recorded distance D(u) equals the distance d(s,u)
  - If we don't use a specific target t, Dijkstra computes d(s,u) for all u (with the same time complexity): Single-Source Shortest Path (SSSP)
- We can extract the actual shortest paths by recording not just D(u) but also the predecessor from which D(u) was realised
  - In SSSP, this gives a shortest-paths tree, with s as root
- In many applications (like routing in a road network), additional information beyond the graph topology is available to guide the search
  - A* algorithm is a variation of Dijkstra (single target version) where the priority queue uses key(u) = D(u) + estimate(u,t)
- Negative edge weights? Shortest path not well-defined if negative cycles reachable.
  - But even if no negative cycles, Dijkstra doesn't work. (Slower "Bellman-Ford" works.)

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 36

Graphs

Spanning Trees

Spanning Arborescences

- A *spanning arborescence* of a directed graph G = (V,E) is a subgraph A=(V,E') (i.e., E' ⊆ E) which is a "directed tree": i.e., one node is a root with no incoming edges, while every other node is reachable from the root and has in-degree 1.
  - Number of edges is in E' exactly |V| - 1
- BFS, DFS, and Dijkstra all give spanning arborescences of the subgraph reachable from the source
  - For each reachable v, E' includes exactly one edge (u,v) such that v is discovered from u
    - If v discovered multiple times, one of them is selected: the first in BFS, the last in DFS, the one in a shortest path in Dijkstra

Minimum Spanning Arborescences

- In a weighted directed graph, Dijkstra gives a spanning arborescence A that minimises the total root-to-node distance in A: $\sum_{v:\text{ reachable}} d_A(s,v)$
- But consider building a network connecting the source to all the nodes, and we seek to minimise the total network's cost: $\sum_{(u,v)\in A} w(u,v)$
  - This is called the minimum spanning arborescence (MSA)
  - Example application: in Natural Language Processing, for generating syntax trees (nodes are words, edge weights generated by neural networks)
- There is an algorithm to find the MSA: Chu-Liu/Edmonds
  - We shall not see this algorithm
- Instead we consider the same problem in the setting of *undirected graphs*
  - Minimum Spanning Tree

Attendance Break!

Login using CSE LDAP id at

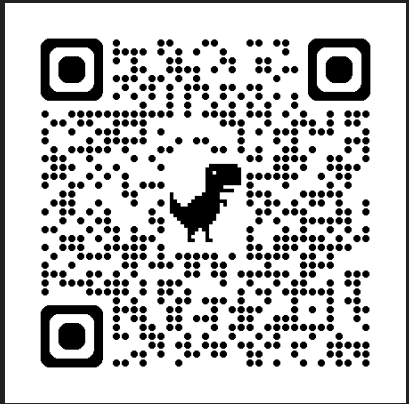
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!



Undirected Graphs & Spanning Trees

- In an undirected graph, the edge set E is a set of sets of 2 elements (not ordered pairs). The edge is not an "arrow".
- A spanning tree of G=(V,E) is a subgraph T=(V,E') such that in T there is a *unique path* between any two nodes.
  - A tree is a connected graph with no cycles. Spanning tree touches all of V.
  - Tree to arborescence: Pick *any* node as the root. Direct all edges "away from the root" (along the unique path from root to one of the two nodes)
  - $|E'| = |V|-1$
- In a weighted undirected graph, weights on the undirected edges
  - A Minimum Spanning Tree is a spanning tree with minimum total weight

Finding MST

- Finding a minimum spanning tree has many applications:
  - Examples: Wiring in a VLSI circuit to minimise conductor length (total heat dissipated due to resistance). Cheapest power grid to connect an electricity substation to homes.
- We'll see two "greedy" algorithms, both selecting one edge at a time
  - Prim's algorithm: Grow a single tree
    - Like Dijkstra: uses a priority queue
  - Kruskal's algorithm: Grow many trees and connect them
    - Uses Union-Find to maintain a sets of nodes in each tree being grown
  - Both take $O(|E| \log |V|)$ time, but could be faster depending on how graph is given (e.g., as a list of edges sorted by weight)

Prim's Algorithm

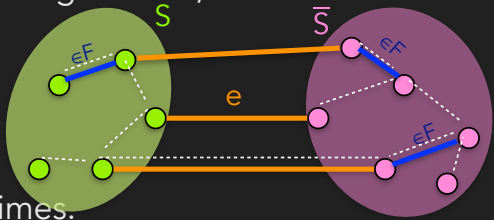
- Very similar to Dijkstra's algorithm: Grow a tree, incorporating one edge at a time (starting from an arbitrary source), selected using a priority-queue

```
Prim() {
  F = [];                              // F is a set of edges
  for (v:V) frontier.insert(v,⊥,∞); // store (v,u,wt({u,v})): v seen from u
  frontier.decrease(s,⊥,0);            // s is an arbitrary node
  while(!frontier.empty()) {
    (u,p) = frontier.extract_min(); // u seen from p is closest to F's nodes
    if(p != ⊥) F.add(p,u);
    for ( v: graph.neighbours(u) ) {
      wuv = graph.weight(u,v);
      frontier.decrease(v,u,wuv);    // update (v,x,j)→(v,u,wuv) iff wuv < j
    }
  }
  return F;
}
```

- Why does it work?

Underlying "Cut Observation"

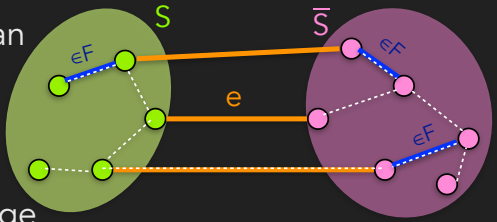
- Both algorithms will rely on the following observation for correctness
- For $S \subseteq V$, let $X_S = \{ \{u,v\} \in E \mid u \in S, v \notin S \}$ be the cut between S and $\overline{S}$
- Suppose $F \subseteq E'$ for some MST (V,E'), and e is a minimum weight edge in a cut X_S where $X_S \cap F = \emptyset$. Then there is an MST (V,E'') such that $F \cup \{e\} \subseteq E''$
- That is, it is *safe to add e to F* when constructing an MST, if e is a minimum weight edge in a cut that avoids F
  - Firstly, $E' \cap X_S \neq \emptyset$. If $e \in E'$ done (E''=E').
  - Else, $E' \cup \{e\}$ has a cycle through e (Why?)
  - A cycle crosses a cut an even number of times.
    So at least one more edge e' in the cycle. Take $E'' = E' \setminus \{e'\} \cup \{e\}$



connected graph with |V| edges: not a tree

Underlying "Cut Observation"

- It is *safe to add e to F* when constructing an MST, if e is a minimum weight edge in a cut that avoids F



- Take E'' = E' \ {e'} ∪ {e}, where e'≠e is an edge in the MST E' and is in the cut. e is a minimum weight edge in the same cut.
- Claim: (V,E'') is an MST
  - Total weight of E'' ≤ Total weight of E' (since wt(e) is min in the cut).
  - (V,E'') is connected (since the removed edge e' is from a cycle in E'∪{e}).
  - (V,E'') has no more cycles (since connected and |E''| = |E'| = |V|-1)
- F ∪ {e} ⊆ E'' (since F ⊆ E \ {e'}) ✓

Prim's Algorithm Revisited

- Maintain a tree (S,F), such that at any point there is an MST containing it
- Start with S={s} for an arbitrary node s and F = $\emptyset$
- Until S equals V, $X_S \neq \emptyset$ (some node in $\overline{S}$ has an edge to S) and $X_S \cap F = \emptyset$ (F is entirely within S)
  - At each iteration, pick a minimum weight edge $e \in X_S$.
  - Update S to $S \cup e$ (one of the end points is already in S) and F to $F \cup \{e\}$. By Cut Observation, safe to add e to F.
- When finished, S=V and, since we maintained the invariant, we have an MST!

Kruskal's Algorithm

- Maintain a *forest* (V,F) such that at any point there is an MST containing it
  - Forest: a graph with no cycles, but possibly not connected
- Start with F=∅. |V| trees, each with a single node.
- At each iteration, identify the minimum weight edge e which can be added between two trees in the forest.
- Continue until only one tree. This is an MST.
- Why? Suppose the edge identified is e, connecting two trees (A,T) and (A',T') in the forest. Every edge in X_A joins A with another tree in the forest. wt(e) is minimum among all such edges (and more). So by cut observation, F∪{e} is still contained in an MST.

Implementing Kruskal's Algorithm

- In each iteration, need the minimum weight edge that is between two trees
  - Go through the edges, in ascending order by weight (need to sort edges, in $O(|E| \log |E|)$ time unless the graph is already given with sorted weights)
  - For each edge {u,v}, we need to check if both u, v in the same tree
  - If not, we select the edge, and the two trees need to be united into a single tree
  - Union-Find! For each tree, maintain a set of all its nodes. Checking if {u,v} is within a tree or across two trees: Find. Adding {u,v} to the forest to combine two trees: Union.
    - $O(|E| \alpha(|V|)) \approx O(|E|)$ time once sorting is done

Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 37 (Short Lecture)

Introduction to Randomisation

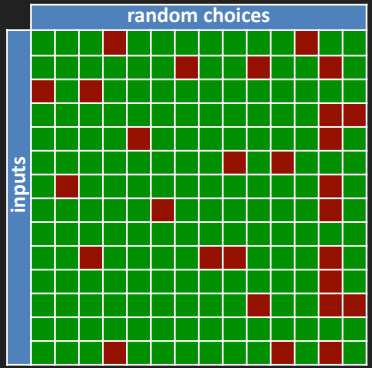
Examples already seen

Recall Two Examples

- Hash Tables
  - Recall central idea: A "universe of functions" $\mathscr{H}$ such that for any set of n keys chosen to be stored, *most* $H \in \mathscr{H}$ (say, more than 1-ε fraction) lead to low collisions
  - Without randomness, an adversary can send inputs which hash to the same bucket
- Quicksort
  - Deterministic strategies for picking pivots either result in $\Theta(n^2)$ worst-case time, or is $\Theta(n \log n)$ but with a large constant
  - Randomised pivot strategy: pick uniformly at random. $O(n \log n)$ with high probability.

Role of Randomness

- Suppose for each setting of random choice, there are bad inputs: then the worst-case performance is bad for each random choice
  - But still for each input, most of the random choices can be good!
- A deterministic algorithm obtained by fixing some random choice will be good for most inputs too
  - But it is not enough to design an algorithm to work well only when inputs are drawn uniformly or from a certain distribution



Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!

